JARVIS AI

Personal AI Assistant — Full Source Code

Next.js 16 · TypeScript · Tailwind · z.ai · Supabase-ready

Generated 2026-09-28

Table of Contents

1. **Project Configuration** (6 files)
2. **Database Schemas** (2 files)
3. **Core Libraries** (11 files)
4. **State Stores** (2 files)
5. **React Hooks** (6 files)
6. **App Shell & Layout** (15 files)
7. **Feature Views** (9 files)
8. **API Routes — Auth & Status** (3 files)
9. **API Routes — Chat (streaming z.ai)** (4 files)
10. **API Routes — Tasks / Notes / Projects** (8 files)
11. **API Routes — Memory / Briefing / Tools** (4 files)
12. **API Routes — Advanced AI (images, search, vision, voice)** (4 files)
13. **API Routes — Supabase Setup** (2 files)

Appendix A. shadcn/ui Components (48 components — standard boilerplate)

About this document. This PDF contains the complete, application-specific source code of the JARVIS AI project — configuration, database schemas (Prisma + Supabase), core libraries (auth, z.ai SDK wrapper, Supabase data layer with Prisma fallback, permissioned tool registry), state stores, React hooks, the app shell, all feature views, and every API route. The shadcn/ui primitive components (Appendix A) are standard open-source boilerplate installed via the shadcn CLI and are listed but not reproduced in full to keep this document focused on the project's own code.

Project Configuration

package.json

```
{
  "name": "nextjs_tailwind_shadcn_ts",
  "version": "0.2.1",
  "private": true,
  "scripts": {
    "dev": "next dev -p 3000 2&gt;&amp;1 | tee dev.log",
    "build": "next build &amp;&amp; cp -r .next/static .next/standalone/.next/ &amp;&amp; cp -r
public .next/standalone/",
    "start": "NODE_ENV=production bun .next/standalone/server.js 2&gt;&amp;1 | tee server.log",
    "lint": "eslint .",
    "db:push": "prisma db push --accept-data-loss",
    "db:generate": "prisma generate",
    "db:migrate": "prisma migrate dev",
    "db:reset": "prisma migrate reset"
  },
  "dependencies": {
    "@dnd-kit/core": "^6.3.1",
    "@dnd-kit/sortable": "^10.0.0",
    "@dnd-kit/utilities": "^3.2.2",
    "@hookform/resolvers": "^5.1.1",
    "@mdxeditor/editor": "^3.39.1",
    "@prisma/client": "^6.11.1",
    "@radix-ui/react-accordion": "^1.2.11",
    "@radix-ui/react-alert-dialog": "^1.1.14",
    "@radix-ui/react-aspect-ratio": "^1.1.7",
    "@radix-ui/react-avatar": "^1.1.10",
    "@radix-ui/react-checkbox": "^1.3.2",
    "@radix-ui/react-collapsible": "^1.1.11",
    "@radix-ui/react-context-menu": "^2.2.15",
    "@radix-ui/react-dialog": "^1.1.14",
    "@radix-ui/react-dropdown-menu": "^2.1.15",
    "@radix-ui/react-hover-card": "^1.1.14",
    "@radix-ui/react-label": "^2.1.7",
    "@radix-ui/react-menubar": "^1.1.15",
    "@radix-ui/react-navigation-menu": "^1.2.13",
    "@radix-ui/react-popover": "^1.1.14",
    "@radix-ui/react-progress": "^1.1.7",
    "@radix-ui/react-radio-group": "^1.3.7",
    "@radix-ui/react-scroll-area": "^1.2.9",
    "@radix-ui/react-select": "^2.2.5",
    "@radix-ui/react-separator": "^1.1.7",
    "@radix-ui/react-slider": "^1.3.5",
    "@radix-ui/react-slot": "^1.2.3",
    "@radix-ui/react-switch": "^1.2.5",
    "@radix-ui/react-tabs": "^1.1.12",
    "@radix-ui/react-toast": "^1.2.14",
    "@radix-ui/react-toggle": "^1.1.9",
    "@radix-ui/react-toggle-group": "^1.1.10",
    "@radix-ui/react-tooltip": "^1.2.7",
    "@reactuses/core": "^6.0.5",
    "@supabase/supabase-js": "^2.117.2",
    "@tanstack/react-query": "^5.82.0",
    "@tanstack/react-table": "^8.21.3",
    "bcryptjs": "^3.0.3",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "cmdk": "^1.1.1",
    "date-fns": "^4.1.0",
    "embla-carousel-react": "^8.6.0",
    "framer-motion": "^12.23.2",
    "input-otp": "^1.4.2",
    "lucide-react": "^0.525.0",
    "next": "^16.1.1",
    "next-auth": "^4.24.11",
    "next-intl": "^4.3.4",
    "next-themes": "^0.4.6",
    "prisma": "^6.11.1",
    "react": "^19.0.0",
    "react-day-picker": "^9.8.0",
    "react-dom": "^19.0.0",
    "react-hook-form": "^7.60.0",
    "react-markdown": "^10.1.0",
```

```
    "react-resizable-panels": "^3.0.3",
    "react-syntax-highlighter": "^15.6.1",
    "recharts": "^2.15.4",
    "sharp": "^0.34.3",
    "sonner": "^2.0.6",
    "tailwind-merge": "^3.3.1",
    "tailwindcss-animate": "^1.0.7",
    "uuid": "^11.1.0",
    "vaul": "^1.1.2",
    "z-ai-web-dev-sdk": "^0.0.18",
    "zod": "^4.0.2",
    "zustand": "^5.0.6"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4",
    "@types/bcryptjs": "^3.0.0",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "bun-types": "^1.3.4",
    "eslint": "^9",
    "eslint-config-next": "^16.1.1",
    "tailwindcss": "^4",
    "tw-animate-css": "^1.3.5",
    "typescript": "^5"
  }
}
```

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": [
      "dom",
      "dom.iterable",
      "esnext"
    ],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "noImplicitAny": false,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [
      {
        "name": "next"
      }
    ],
    "paths": {
      "@/*": [
        "./src/*"
      ]
    }
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts"
  ],
  "exclude": [
    "node_modules"
  ]
}
```

next.config.ts

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
```

```
  output: "standalone",
  /* config options here */
  typescript: {
    ignoreBuildErrors: true,
  },
  reactStrictMode: false,
};

export default nextConfig;
```

tailwind.config.ts

```
import type { Config } from "tailwindcss";
import tailwindcssAnimate from "tailwindcss-animate";

const config: Config = {
    darkMode: "class",
    content: [
    "./pages/**/*.{js,ts,jsx,tsx,mdx}",
    "./components/**/*.{js,ts,jsx,tsx,mdx}",
    "./app/**/*.{js,ts,jsx,tsx,mdx}",
  ],
  theme: {
  extend: {
  colors: {
  background: 'hsl(var(--background))',
  foreground: 'hsl(var(--foreground))',
  card: {
  DEFAULT: 'hsl(var(--card))',
  foreground: 'hsl(var(--card-foreground))'
  },
  popover: {
  DEFAULT: 'hsl(var(--popover))',
  foreground: 'hsl(var(--popover-foreground))'
  },
  primary: {
  DEFAULT: 'hsl(var(--primary))',
  foreground: 'hsl(var(--primary-foreground))'
  },
  secondary: {
  DEFAULT: 'hsl(var(--secondary))',
  foreground: 'hsl(var(--secondary-foreground))'
  },
  muted: {
  DEFAULT: 'hsl(var(--muted))',
  foreground: 'hsl(var(--muted-foreground))'
  },
  accent: {
  DEFAULT: 'hsl(var(--accent))',
  foreground: 'hsl(var(--accent-foreground))'
  },
  destructive: {
  DEFAULT: 'hsl(var(--destructive))',
  foreground: 'hsl(var(--destructive-foreground))'
  },
  border: 'hsl(var(--border))',
  input: 'hsl(var(--input))',
  ring: 'hsl(var(--ring))',
  chart: {
  '1': 'hsl(var(--chart-1))',
  '2': 'hsl(var(--chart-2))',
  '3': 'hsl(var(--chart-3))',
  '4': 'hsl(var(--chart-4))',
  '5': 'hsl(var(--chart-5))'
  }
  },
  borderRadius: {
  lg: 'var(--radius)',
  md: 'calc(var(--radius) - 2px)',
  sm: 'calc(var(--radius) - 4px)'
  }
  }
  },
  plugins: [tailwindcssAnimate],
};
export default config;
```

components.json

```
{
  "$schema": "https://ui.shadcn.com/schema.json",
  "style": "new-york",
  "rsc": true,
  "tsx": true,
  "tailwind": {
    "config": "",
    "css": "src/app/globals.css",
    "baseColor": "neutral",
    "cssVariables": true,
    "prefix": ""
  },
  "aliases": {
    "components": "@/components",
    "utils": "@/lib/utils",
    "ui": "@/components/ui",
    "lib": "@/lib",
    "hooks": "@/hooks"
  },
  "iconLibrary": "lucide"
}
```

postcss.config.mjs

```
const config = {
  plugins: ["@tailwindcss/postcss"],
};

export default config;
```

Database Schemas

prisma/schema.prisma

```
// JARVIS AI — Prisma schema (SQLite)
// All user-owned tables carry a `userId` foreign key so the data-access layer can
// enforce per-user isolation on the server. RLS-equivalent behavior is provided
// by the API routes (which always filter by the authenticated user id) — see lib/auth.ts.

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model User {
  id           String   @id @default(cuid())
  email        String   @unique
  displayName  String
  passwordHash String
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt

  sessions      Session[]
  conversations Conversation[]
  tasks         Task[]
  notes         Note[]
  projects      Project[]
  preferences   Preference[]
  auditEvents   AuditEvent[]
}

model Session {
  id        String   @id @default(cuid())
  userId    String
  expiresAt DateTime
  createdAt DateTime @default(now())

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@index([expiresAt])
}

model Conversation {
  id        String   @id @default(cuid())
  userId    String
  title     String
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  user     User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  messages Message[]

  @@index([userId])
  @@index([updatedAt])
}

model Message {
  id             String   @id @default(cuid())
  conversationId String
  role           String // 'user' | 'assistant' | 'system'
  content        String
  createdAt      DateTime @default(now())

  conversation Conversation @relation(fields: [conversationId], references: [id], onDelete: Cascade)

  @@index([conversationId])
  @@index([createdAt])
}

model Task {
  id          String    @id @default(cuid())
```

```
  userId      String
  title       String
  description String    @default("")
  priority    String    @default("medium") // low | medium | high
  status      String    @default("todo") // todo | in_progress | done
  category    String    @default("")
  dueDate     DateTime?
  projectId   String?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
  completedAt DateTime?

  user    User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  project Project? @relation(fields: [projectId], references: [id], onDelete: SetNull)

  @@index([userId])
  @@index([status])
  @@index([dueDate])
  @@index([projectId])
}

model Note {
  id        String   @id @default(cuid())
  userId    String
  title     String
  content   String   @default("")
  projectId String?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  user    User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  project Project? @relation(fields: [projectId], references: [id], onDelete: SetNull)

  @@index([userId])
  @@index([updatedAt])
  @@index([projectId])
}

model Project {
  id          String   @id @default(cuid())
  userId      String
  name        String
  description String   @default("")
  goal        String   @default("")
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  user  User   @relation(fields: [userId], references: [id], onDelete: Cascade)
  tasks Task[]
  notes Note[]

  @@index([userId])
}

// Key/value preference store. Used both for "memory" entries the user explicitly
// saves and for app settings (responseStyle, theme, reducedMotion, voice prefs, ...).
// `category` distinguishes 'memory' | 'settings'.
model Preference {
  id        String   @id @default(cuid())
  userId    String
  category  String   @default("settings") // settings | memory
  key       String
  value     String
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@unique([userId, category, key])
  @@index([userId, category])
}

model AuditEvent {
  id        String   @id @default(cuid())
  userId    String
  toolName  String
  input     String   @default("")
  result    String   @default("")
  status    String   @default("success") // success | denied | error
  createdAt DateTime @default(now())
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
```

```
  @@index([userId])
  @@index([createdAt])
}
```

supabase/schema.sql

```
-- ============================================================================
-- JARVIS AI — Supabase schema
-- Run this in the Supabase SQL editor (Dashboard → SQL Editor → New query).
-- It is idempotent: safe to re-run.
--
-- Model: single-user "Operator" app. All rows carry user_id = the fixed
-- OPERATOR_ID below. RLS is ENABLED on every table, with a permissive policy
-- that allows the anon key (and service role) to read/write all rows. This is
-- honest for a single-user local app where the database is the owner's own
-- Supabase project. For a true multi-user deployment, replace the
-- `USING (true)` policies with `USING (user_id = auth.uid())` and wire up
-- Supabase Auth.
-- ============================================================================

-- The fixed Operator UUID (must match src/lib/supabase.ts OPERATOR_ID).
-- 00000000-0000-0000-0000-000000000001

create extension if not exists "pgcrypto";

-- ----------------------------------------------------------------------------
-- Profiles
-- ----------------------------------------------------------------------------
create table if not exists public.profiles (
  id          uuid primary key,
  email       text not null,
  display_name text not null default 'Operator',
  created_at  timestamptz not null default now(),
  updated_at  timestamptz not null default now()
);

insert into public.profiles (id, email, display_name)
values ('00000000-0000-0000-0000-000000000001', 'operator@jarvis.local', 'Operator')
on conflict (id) do nothing;

-- ----------------------------------------------------------------------------
-- Conversations &amp;amp; messages
-- ----------------------------------------------------------------------------
create table if not exists public.conversations (
  id         uuid primary key default gen_random_uuid(),
  user_id    uuid not null references public.profiles(id) on delete cascade,
  title      text not null default 'New conversation',
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
create index if not exists conversations_user_id_idx on public.conversations(user_id);
create index if not exists conversations_updated_at_idx on public.conversations(updated_at desc);

create table if not exists public.messages (
  id              uuid primary key default gen_random_uuid(),
  conversation_id uuid not null references public.conversations(id) on delete cascade,
  role            text not null check (role in ('user','assistant','system')),
  content         text not null,
  created_at      timestamptz not null default now()
);
create index if not exists messages_conversation_id_idx on public.messages(conversation_id);
create index if not exists messages_created_at_idx on public.messages(created_at);

-- ----------------------------------------------------------------------------
-- Tasks
-- ----------------------------------------------------------------------------
create table if not exists public.tasks (
  id           uuid primary key default gen_random_uuid(),
  user_id      uuid not null references public.profiles(id) on delete cascade,
  title        text not null,
  description  text not null default '',
  priority     text not null default 'medium' check (priority in ('low','medium','high')),
  status       text not null default 'todo' check (status in ('todo','in_progress','done')),
  category     text not null default '',
  due_date     timestamptz,
  project_id   uuid,
  created_at   timestamptz not null default now(),
  updated_at   timestamptz not null default now(),
  completed_at timestamptz
```

```
);
create index if not exists tasks_user_id_idx on public.tasks(user_id);
create index if not exists tasks_status_idx on public.tasks(status);
create index if not exists tasks_due_date_idx on public.tasks(due_date);
create index if not exists tasks_project_id_idx on public.tasks(project_id);

-- ---------------------------------------------------------------------------
-- Notes
-- ---------------------------------------------------------------------------
create table if not exists public.notes (
  id         uuid primary key default gen_random_uuid(),
  user_id    uuid not null references public.profiles(id) on delete cascade,
  title      text not null,
  content    text not null default '',
  project_id uuid,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
create index if not exists notes_user_id_idx on public.notes(user_id);
create index if not exists notes_updated_at_idx on public.notes(updated_at desc);
create index if not exists notes_project_id_idx on public.notes(project_id);

-- ---------------------------------------------------------------------------
-- Projects
-- ---------------------------------------------------------------------------
create table if not exists public.projects (
  id          uuid primary key default gen_random_uuid(),
  user_id     uuid not null references public.profiles(id) on delete cascade,
  name        text not null,
  description text not null default '',
  goal        text not null default '',
  created_at  timestamptz not null default now(),
  updated_at  timestamptz not null default now()
);
create index if not exists projects_user_id_idx on public.projects(user_id);

-- FK from tasks/notes to projects (added after table creation so it survives
-- re-runs). Drop first to avoid "already exists" on re-run.
do $$
begin
  if not exists (
    select 1 from information_schema.table_constraints
    where constraint_name = 'tasks_project_id_fkey' and table_name = 'tasks'
  ) then
    alter table public.tasks
      add constraint tasks_project_id_fkey
      foreign key (project_id) references public.projects(id) on delete set null;
  end if;
  if not exists (
    select 1 from information_schema.table_constraints
    where constraint_name = 'notes_project_id_fkey' and table_name = 'notes'
  ) then
    alter table public.notes
      add constraint notes_project_id_fkey
      foreign key (project_id) references public.projects(id) on delete set null;
  end if;
end $$;

-- ---------------------------------------------------------------------------
-- Preferences (settings + memory)
-- ---------------------------------------------------------------------------
create table if not exists public.preferences (
  id         uuid primary key default gen_random_uuid(),
  user_id    uuid not null references public.profiles(id) on delete cascade,
  category   text not null default 'settings' check (category in ('settings','memory')),
  key        text not null,
  value      text not null default '',
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  unique (user_id, category, key)
);
create index if not exists preferences_user_category_idx on public.preferences(user_id, category);

-- ---------------------------------------------------------------------------
-- Audit events
-- ---------------------------------------------------------------------------
create table if not exists public.audit_events (
  id         uuid primary key default gen_random_uuid(),
  user_id    uuid not null references public.profiles(id) on delete cascade,
  tool_name  text not null,
  input      text not null default '',
```

```
  result     text not null default '',
  status     text not null default 'success' check (status in ('success','denied','error')),
  created_at timestamptz not null default now()
);
create index if not exists audit_events_user_id_idx on public.audit_events(user_id);
create index if not exists audit_events_created_at_idx on public.audit_events(created_at desc);

-- ------------------------------------------------------------------------
-- Row Level Security
-- ------------------------------------------------------------------------
alter table public.profiles     enable row level security;
alter table public.conversations enable row level security;
alter table public.messages     enable row level security;
alter table public.tasks         enable row level security;
alter table public.notes         enable row level security;
alter table public.projects     enable row level security;
alter table public.preferences  enable row level security;
alter table public.audit_events  enable row level security;

-- Permissive policies for the single-user Operator model. The anon key (and
-- service role) can read/write all rows. Replace with
-- `using (user_id = auth.uid())` for multi-user + Supabase Auth.
do $$
declare t text;
begin
  foreach t in array array['profiles','conversations','messages','tasks','notes','projects',
'preferences','audit_events'] loop
    begin
      execute format('create policy %I on public.%I for all using (true) with check (true);', t ||
'_all', t);
    exception when duplicate_object then null;
    end;
  end loop;
end $$;

-- updated_at trigger helper
create or replace function public.touch_updated_at()
returns trigger language plpgsql as $$
begin new.updat

… (truncated for brevity; see the full file in the project repository) …
```

Core Libraries

src/lib/types.ts

```
// Shared TypeScript types for JARVIS AI.

export type Role = "user" | "assistant" | "system";

export interface SafeUser {
  id: string;
  email: string;
  displayName: string;
  createdAt: Date;
}

export interface ChatMessage {
  id?: string;
  role: Role;
  content: string;
  createdAt?: string;
}

export interface Conversation {
  id: string;
  title: string;
  createdAt: string;
  updatedAt: string;
}

export type Priority = "low" | "medium" | "high";
export type TaskStatus = "todo" | "in_progress" | "done";

export interface Task {
  id: string;
  title: string;
  description: string;
  priority: Priority;
  status: TaskStatus;
  category: string;
  dueDate: string | null;
  projectId: string | null;
  createdAt: string;
  updatedAt: string;
  completedAt: string | null;
}

export interface Note {
  id: string;
  title: string;
  content: string;
  projectId: string | null;
  createdAt: string;
  updatedAt: string;
}

export interface Project {
  id: string;
  name: string;
  description: string;
  goal: string;
  createdAt: string;
  updatedAt: string;
  taskCount?: number;
  completedTaskCount?: number;
  noteCount?: number;
  progress?: number; // 0..100
}

export type ResponseStyle = "concise" | "balanced" | "detailed";
export type ThemeMode = "dark" | "light";

export interface Settings {
  displayName: string;
  model: string; // "auto" or a specific model id
  responseStyle: ResponseStyle;
  voiceEnabled: boolean;
```

```
  ttsVoice: string;
  ttsRate: number;
  theme: ThemeMode;
  reducedMotion: boolean;
  memoryEnabled: boolean;
}

export interface MemoryEntry {
  id: string;
  key: string;
  value: string;
  createdAt: string;
  updatedAt: string;
}

export interface AuditEvent {
  id: string;
  toolName: string;
  input: string;
  result: string;
  status: "success" | "denied" | "error";
  createdAt: string;
}

export interface ConnectionStatus {
  id: string;
  name: string;
  description: string;
  status: "connected" | "not_configured" | "error";
  detail?: string;
}

// ---- Permissioned tool registry types ----
export type ToolStatus = "success" | "denied" | "error";

export interface ToolResult {
  ok: boolean;
  status: ToolStatus;
  message: string;
  data?: unknown;
}

export interface ToolDef&lt;TInput = Record&lt;string, unknown&gt;&gt; {
  name: string;
  label: string;
  description: string;
  requiresConfirmation: boolean;
  // Zod-style schema description (lightweight, for UI + validation)
  schema: ToolField[];
  // Server-side execution. Receives validated input + authenticated userId.
  execute: (input: TInput, userId: string) =&gt; Promise&lt;ToolResult&gt;;
}

export interface ToolField {
  name: string;
  type: "string" | "number" | "boolean" | "date";
  label: string;
  required: boolean;
  options?: string[];
  placeholder?: string;
}

// In-stream tool call emitted by the model (parsed from fenced blocks).
export interface ParsedToolCall {
  tool: string;
  input: Record&lt;string, unknown&gt;;
}

export type AssistantState = "idle" | "listening" | "processing" | "speaking" | "error";

export type ImageSize = "1024x1024" | "768x1344" | "864x1152" | "1344x768" | "1152x864" |
"1440x720" | "720x1440";
```

src/lib/utils.ts

```
import { clsx, type ClassValue } from "clsx"
import { twMerge } from "tailwind-merge"

export function cn(...inputs: ClassValue[]) {
```

```
  return twMerge(clsx(inputs))
}
```

src/lib/db.ts

```
import { PrismaClient } from '@prisma/client'

const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined
}

export const db =
  globalForPrisma.prisma ??
  new PrismaClient({
    log: ['query'],
  })

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = db
```

src/lib/auth.ts

```
import { cookies } from "next/headers";
import { OPERATOR_ID } from "@/lib/supabase";

export const SESSION_COOKIE = "jarvis_session";
const SESSION_TTL_DAYS = 30;

// With the sign-in page removed, JARVIS is a single-user app. Every request
// acts as the fixed local "Operator" whose profile + data live in Supabase.
// The cookie is just an "initialized" marker for the client; there is no
// server-side session table.

export interface SafeUser {
  id: string;
  email: string;
  displayName: string;
}

const OPERATOR: SafeUser = {
  id: OPERATOR_ID,
  email: "operator@jarvis.local",
  displayName: "Operator",
};

function isProd() {
  return process.env.NODE_ENV === "production";
}

async function setSessionCookie(): Promise&lt;void&gt; {
  (await cookies()).set(SESSION_COOKIE, "operator", {
    httpOnly: true,
    sameSite: "lax",
    secure: isProd(),
    path: "/",
    maxAge: SESSION_TTL_DAYS * 24 * 60 * 60,
  });
}

/** Always returns the Operator; sets the init cookie on first contact. */
export async function ensureDefaultUser(): Promise&lt;SafeUser&gt; {
  await setSessionCookie();
  return OPERATOR;
}

/** Returns the Operator if the init cookie is present, else null. */
export async function getCurrentUser(): Promise&lt;SafeUser | null&gt; {
  const c = (await cookies()).get(SESSION_COOKIE)?.value;
  if (!c) return null;
  return OPERATOR;
}

/** Every protected route calls this — always the Operator. */
export async function requireUser(): Promise&lt;SafeUser&gt; {
  return OPERATOR;
}
```

src/lib/supabase.ts

```
import { createClient, type SupabaseClient } from "@supabase/supabase-js";

// The fixed single-user "Operator" id under which all rows are stored. With
// the sign-in page removed and Supabase Auth not used, every row is scoped to
// this UUID. RLS policies in supabase/schema.sql allow the anon key to access
// these rows (single-user local app). For multi-user, switch to Supabase Auth
// and change policies to `user_id = auth.uid()`.
export const OPERATOR_ID = "00000000-0000-0000-0000-000000000001";

function env(name: string): string {
  const v = process.env[name];
  if (!v) throw new Error(`Missing env var: ${name}`);
  return v;
}

let _client: SupabaseClient | null = null;

/**
 * Server-side Supabase client. Uses the anon key by default (subject to RLS).
 * If `SUPABASE_SERVICE_ROLE_KEY` is set, uses that to bypass RLS for trusted
 * server operations. The service-role key NEVER reaches the browser.
 */
export function supabaseServer(): SupabaseClient {
  if (_client) return _client;
  const url = env("NEXT_PUBLIC_SUPABASE_URL");
  const serviceKey = process.env.SUPABASE_SERVICE_ROLE_KEY;
  const anonKey = env("NEXT_PUBLIC_SUPABASE_ANON_KEY");
  // Prefer service role when configured (server-only).
  const key = serviceKey &amp;&amp; serviceKey.length &gt; 0 ? serviceKey : anonKey;
  _client = createClient(url, key, {
    auth: { persistSession: false, autoRefreshToken: false },
  });
  return _client;
}

/** True when Supabase env vars are configured. */
export function isSupabaseConfigured(): boolean {
  return !!process.env.NEXT_PUBLIC_SUPABASE_URL &amp;&amp; !!process.env.
NEXT_PUBLIC_SUPABASE_ANON_KEY;
}
```

src/lib/jarvis-ai.ts

```
import ZAI from "z-ai-web-dev-sdk";

// Server-only z.ai SDK wrapper. Config is auto-loaded from
// /etc/.z-ai-config (or ./z-ai-config / ~/.z-ai-config). No API keys in code.

let cached: ZAI | null = null;
let initError: string | null = null;

export async function getZAI(): Promise&lt;ZAI&gt; {
  if (cached) return cached;
  try {
    cached = await ZAI.create();
    initError = null;
    return cached;
  } catch (e) {
    initError = e instanceof Error ? e.message : String(e);
    cached = null;
    throw new Error(`z.ai SDK initialization failed: ${initError}`);
  }
}

export function getZaiInitError(): string | null {
  return initError;
}

// Curated list of model ids the user can pick in Settings. "auto" omits the
// model parameter so the provider uses its default. Availability depends on the
// configured provider; an invalid choice surfaces a real API error in the chat.
export const AVAILABLE_MODELS: { id: string; label: string }[] = [
  { id: "auto", label: "Auto (provider default)" },
  { id: "glm-4.6", label: "GLM-4.6" },
  { id: "glm-4.5", label: "GLM-4.5" },
  { id: "glm-4-plus", label: "GLM-4-Plus" },
  { id: "glm-4-flash", label: "GLM-4-Flash (fast)" },
```

```
];

export interface ZaiChatMessage {
  role: "system" | "user" | "assistant";
  content: string;
}

export interface StreamCallbacks {
  onDelta: (text: string) =&gt; void;
  onDone?: () =&gt; void;
  onError?: (err: Error) =&gt; void;
  signal?: AbortSignal;
}

/**
 * Streams a chat completion from z.ai. Parses the OpenAI-style SSE stream
 * server-side and invokes `onDelta` with each content chunk. Resolves when the
 * stream completes. Throws on non-2xx responses or parse failures.
 */
export async function streamChatCompletion(
  messages: ZaiChatMessage[],
  opts: { model?: string; temperature?: number } = {},
  cb: StreamCallbacks
): Promise&lt;void&gt; {
  const zai = await getZAI();
  const body: Record&lt;string, unknown&gt; = {
    messages,
    stream: true,
    thinking: { type: "disabled" },
  };
  if (opts.model &amp;&amp; opts.model !== "auto") body.model = opts.model;
  if (typeof opts.temperature === "number") body.temperature = opts.temperature;

  // The SDK returns the raw Response.body (Web ReadableStream&lt;Uint8Array&gt;) when
  // stream:true and the response is event-stream/plain.
  const stream = (await zai.chat.completions.create(body as never)) as ReadableStream&lt;
Uint8Array&gt;;
  if (!stream || typeof (stream as any).getReader !== "function") {
    throw new Error("z.ai did not return a readable stream.");
  }

  const reader = stream.getReader();
  const decoder = new TextDecoder();
  let buffer = "";
  try {
    while (true) {
      if (cb.signal?.aborted) {
        await reader.cancel().catch(() =&gt; {});
        break;
      }
      const { done, value } = await reader.read();
      if (done) break;
      buffer += decoder.decode(value, { stream: true });

      let nl: number;
      while ((nl = buffer.indexOf("\n")) !== -1) {
        const line = buffer.slice(0, nl).trim();
        buffer = buffer.slice(nl + 1);
        if (!line) continue;
        if (!line.startsWith("data:")) continue;
        const payload = line.slice(5).trim();
        if (payload === "[DONE]") {
          cb.onDone?.();
          return;
        }
        try {
          const json = JSON.parse(payload);
          const delta: string | undefined =
            json?.choices?.[0]?.delta?.content ??
            json?.choices?.[0]?.message?.content;
          if (delta) cb.onDelta(delta);
        } catch {
          // Ignore malformed SSE lines (keep-aliv etc.).
        }
      }
    }
    cb.onDone?.();
  } catch (err) {
    cb.onError?.(err instanceof Error ? err : new Error(String(err)));
    throw err;
  } finally {
```

```
    try {
      await reader.releaseLock();
    } catch {
      /* noop */
    }
  }
}

/** Non-streaming completion for short utility calls (summarize, plan, etc). */
export async function chatCompletion(
  messages: ZaiChatMessage[],
  opts: { model?: string; temperature?: number; maxTokens?: number } = {}
): Promise&lt;string&gt; {
  const zai = await getZAI();
  const body: Record&lt;string, unknown&gt; = {
    messages,
    thinking: { type: "disabled" },
  };
  if (opts.model &amp;&amp; opts.model !== "auto") body.model = opts.model;
  if (typeof opts.temperature === "number") body.temperature = opts.temperature;
  if (typeof opts.maxTokens === "number") body.max_tokens = opts.maxTokens;

  const res = (await zai.chat.completions.create(body as never)) as {
    choices?: { message?: { content?: string } }[];
  };
  return res?.choices?.[0]?.message?.content ?? "";
}

/** Lightweight connectivity probe used by /api/status. */
export async function pingZai(): Promise&lt;{ ok: boolean; detail?: string }&gt; {
  try {
    const txt = await chatCompletion(
      [{ role: "user", content: "Reply with the single word: online" }],
      { maxTokens: 8 }
    );
    return { ok: true, detail: txt.slice(0, 60) };
  } catch (e) {
    return { ok: false, detail: e instanceof Error ? e.message : String(e) };
  }
}

// ---------------------------------------------------------------------------
// Advanced z.ai capabilities (image generation, web search, page reader,
// vision). These unlock a much richer assistant experience.
// ---------------------------------------------------------------------------

export type ImageSize = "1024x1024" | "768x1344" | "864x1152" | "1344x768" | "1152x864" |
"1440x720" | "720x1440";

/** Generate an image from a text prompt. Returns base64 PNG data URLs. */
export async function generateImage(prompt: string, size: ImageSize = "1024x1024"): Promise&lt;
string[]&gt; {
  const zai = await getZAI();
  const res = await zai.images.generations.create({ prompt: prompt.slice(0, 1000), size });
  const data = (res as { data?: { base64?: string }[] }).data ?? [];
  return data.filter((d) =&gt; d.base64).map((d) =&gt; `${d.base64}`);
}

export interface WebSearchResult {
  url: string;
  name: string;
  snippet: string;
  host_name: string;
  date: string;
  favicon: string;
}

/** Live web search via z.ai's web_search function. */
export async function webSearch(query: string, num = 5, recencyDays?: number): Promise&lt;
WebSearchResult[]&gt; {
  const zai = await getZAI();
  const args: { query: string; num?: number; recency_days?: number } = { query: query.slice(0, 200),
 num };
  if (recencyDays) args.recency_days = recencyDays;
  const results = await zai.functions.invoke("web_search", args);
  return (results as WebSearchResult[]) ?? [];
}

export interface PageContent {
  title: string;
  url: string;
```

```
  html: string;
  publishedTime?: string;
}

/** Fetch + parse a web page's content via z.ai's page_reader function. */
export async function readPage(url: string): Promise&lt;PageContent&gt; {
  const zai = await getZAI();
  const res = await zai.functions.invoke("page_reader", { url });
  const data = (res as { data?: PageContent }).data;
  if (!data) throw new Error("page_reader returned no data.");
  return data;
}

export interface VisionMessage {
  role: "system" | "user" | "assistant";
  content: string | VisionContentItem[];
}
export interface VisionContentItem {
  type: "text" | "image_url";
  text?: string;
  image_url?: { url: string };
}

/** Describe/analyze an image (URL or base64 data URL) using z.ai vision. */
export async function visionChat(prompt: string, imageUrl: string): Promise&lt;string&gt; {
  const zai = await getZAI();
  const body: Record&lt;string, unknown&gt; = {
    model: "glm-4.5v",
    messages: [
      {
        role: "user",
        content: [
          { type: "text", text: prompt.slice(0, 2000) },
          { type: "image_url", image_url: { url: imageUrl } },
        ],
      },
    ] as VisionMessage[],
    thinking: { type: "disabled" },
  };
  const res = (await zai.chat.completions.createVision(body as never)) as {
    choices?: { message?: { content?: string } }[];
  };
  return res?.choices?.[0]?.message?.content ?? "";
}
```

src/lib/data.ts

```
import { supabaseServer, OPERATOR_ID } from "@/lib/supabase";
import type { Task, Note, Project, Conversation, ChatMessage, Priority, TaskStatus, Settings,
MemoryEntry } from "@/lib/types";
import * as P from "@/lib/data-prisma";

// ============================================================================
// Data-access layer. Tries Supabase first; on SCHEMA_NOT_READY (tables not yet
// applied — the anon key can't run DDL) it transparently falls back to the
// local Prisma+SQLite store. So the app is fully functional immediately, and
// once the user runs supabase/schema.sql, Supabase takes over automatically.
// All rows are scoped to the fixed OPERATOR_ID (single-user app).
// ============================================================================

const UID = OPERATOR_ID;

class DbError extends Error {
  constructor(public code: string, message: string) { super(message); }
}

function assertReady(res: { error: unknown | null; data: unknown }) {
  if (res.error &amp;&amp; typeof res.error === "object" &amp;&amp; "code" in res.error) {
    const code = String((res.error as { code?: string }).code ?? "");
    if (code === "PGRST205" || code === "42P01" || code === "404") {
      throw new DbError("SCHEMA_NOT_READY", "Supabase schema is not set up. Run supabase/schema.sql
in the Supabase SQL editor (link in the yellow banner at the top).");
    }
  }
}

function isNotReady(e: unknown): boolean {
  return e instanceof DbError &amp;&amp; e.code === "SCHEMA_NOT_READY";
}
```

```
async function withFallback&lt;T&gt;(sb: () =&gt; Promise&lt;T&gt;, prisma: () =&gt; Promise&lt;
T&gt;): Promise&lt;T&gt; {
  try {
    return await sb();
  } catch (e) {
    if (isNotReady(e)) return prisma();
    throw e;
  }
}

// ---- mappers -----------------------------------------------------------
interface TaskRow {
  id: string; user_id: string; title: string; description: string;
  priority: Priority; status: TaskStatus; category: string; due_date: string | null;
  project_id: string | null; created_at: string; updated_at: string; completed_at: string | null;
}
function taskFromRow(r: TaskRow): Task {
  return {
    id: r.id, title: r.title, description: r.description ?? "", priority: r.priority, status: r.
status,
    category: r.category ?? "", dueDate: r.due_date, projectId: r.project_id,
    createdAt: r.created_at, updatedAt: r.updated_at, completedAt: r.completed_at,
  };
}
interface NoteRow { id: string; user_id: string; title: string; content: string; project_id: string
| null; created_at: string; updated_at: string; }
function noteFromRow(r: NoteRow): Note {
  return { id: r.id, title: r.title, content: r.content ?? "", projectId: r.project_id, createdAt:
r.created_at, updatedAt: r.updated_at };
}
interface ProjectRow { id: string; user_id: string; name: string; description: string; goal: string;
 created_at: string; updated_at: string; }
function projectBase(r: ProjectRow): Project {
  return { id: r.id, name: r.name, description: r.description ?? "", goal: r.goal ?? "", createdAt:
r.created_at, updatedAt: r.updated_at };
}
function convFromRow(r: { id: string; title: string; created_at: string; updated_at: string }):
Conversation {
  return { id: r.id, title: r.title, createdAt: r.created_at, updatedAt: r.updated_at };
}

// ============================ TASKS ======================================
export async function listTasks(opts: { view?: string; q?: string; projectId?: string | null }):
Promise&lt;Task[]&gt; {
  return withFallback(async () =&gt; {
    const sb = supabaseServer();
    let q = sb.from("tasks").select("*").eq("user_id", UID);
    const view = opts.view ?? "all";
    if (opts.projectId) q = q.eq("project_id", opts.projectId);
    if (opts.q) { const s = `%${opts.q}%`; q = q.or(`title.ilike.${s},description.ilike.${s},
category.ilike.${s}`); }
    const now = new Date();
    const startOfDay = new Date(now); startOfDay.setHours(0, 0, 0, 0);
    const endOfDay = new Date(now); endOfDay.setHours(23, 59, 59, 999);
    const in7 = new Date(now.getTime() + 7 * 86400000);
    if (view === "today") q = q.gte("due_date", startOfDay.toISOString()).lte("due_date", endOfDay.
toISOString()).neq("status", "done");
    else if (view === "upcoming") q = q.gte("due_date", now.toISOString()).lte("due_date", in7.
toISOString()).neq("status", "done");
    else if (view === "completed") q = q.eq("status", "done");
    else if (view === "active") q = q.neq("status", "done");
    if (view === "completed") q = q.order("completed_at", { ascending: false, nullsFirst: false });
    else q = q.order("due_date", { ascending: true, nullsFirst: true }).order("created_at", {
ascending: false });
    const res = await q.limit(200);
    assertReady(res);
    if (res.error) throw new Error(res.error.message);
    return (res.data as TaskRow[]).map(taskFromRow);
  }, () =&gt; P.listTasksPrisma(opts));
}

export async function getTask(id: string): Promise&lt;Task | null&gt; {
  return withFallback(async () =&gt; {
    const sb = supabaseServer();
    const res = await sb.from("tasks").select("*").eq("id", id).eq("user_id", UID).maybeSingle();
    assertReady(res);
    if (res.error) throw new Error(res.error.message);
    return res.data ? taskFromRow(res.data as TaskRow) : null;
  }, () =&gt; P.getTaskPrisma(id));
}
export async function createTask(data: { title: string; description?: string; priority?: Priority;
```

```
status?: TaskStatus; category?: string; dueDate?: string | null; projectId?: string | null }):
Promise&lt;Task&gt; {
  return withFallback(async () =&gt; {
    const sb = supabaseServer();
    const insert: Record&lt;string, unknown&gt; = {
      user_id: UID, title: String(data.title).slice(0, 200),
      description: String(data.description ?? "").slice(0, 4000),
      priority: data.priority ?? "medium", status: data.status ?? "todo",
      category: String(data.category ?? "").slice(0, 60),
      due_date: data.dueDate ? new Date(data.dueDate).toISOString() : null,
      project_id: data.projectId ?? null,
      completed_at: data.status === "done" ? new Date().toISOString() : null,
    };
    const res = await sb.from("tasks").insert(insert).select("*").single();
    assertReady(res);
    if (res.error) throw new Error(res.error.message);
    return taskFromRow(res.data as TaskRow);
  }, () =&gt; P.createTaskPrisma(data));
}

export async function updateTask(id: string, patch: Partial&lt;Task&gt;): Promise&lt;Task | null&gt;
 {
  return withFallback(async () =&gt; {
    const sb = supabaseServer();
    const existing = await getTask(id);
    if (!existing) return null;
    const upd: Record&lt;string, unknown&gt; = {};
    if (patch.title !== undefined) upd.title = String(patch.title).slice(0, 200);
    if (patch.description !== undefined) upd.description = String(patch.description).slice(0, 4000);
    if (patch.priority !== undefined &amp;&amp; ["low", "medium", "high"].includes(patch.priority))
upd.priority = patch.priority;
    if (patch.category !== undefined) upd.category = String(patch.category).slice(0, 60);
    if (patch.dueDate !== undefined) upd.due_date = patch.dueDate ? new Date(patch.dueDate).
toISOString() : null;
    if (patch.projectId !== undefined) upd.project_id = patch.projectId || null;
    if (patch.status !== undefined &amp;&amp; ["todo", "in_progress", "done"].includes(patch.
status)) {
      upd.status = patch.status;
      if (patch.status === "done" &amp;&amp; !existing.completedAt) upd.completed_at = new Date().
toISOString();
      if (patch.status !== "done") upd.completed_at = null;
    }
    const res = await sb.from("tasks").update(upd).eq("id", id).eq("user_id", UID).select("*").
single();
    assertReady(res);
    if (res.error) throw new Error(res.error.message);
    return taskFromRow(res.data as TaskRow);
  }, () =&gt; P.updateTaskPrisma(id, patch));
}

export async function deleteTask(id: string): Promise&lt;void&gt; {
  return withFallback(async () =&gt; {
    const sb = supabaseServer();
    const res = await sb.from("tasks").delete().eq("id", id).eq("user_id", UID);
    assertReady(res);
    if (res.error) throw new Error(res.error.message);
  }, () =&gt; P.deleteTaskPrisma(id));
}

export async function countTasks(opts: { status?: TaskStatus; statusNot?: TaskStatus; dueLt?: Date;
dueGte?: Date; dueLte?: Date }): Promise&lt;number&gt; {
  return withFallback(async () =&gt; {
    const sb = supabaseServer();
    // Use a real select (not head:true) so a missing table surfaces PGRST205
    // (head:true returns 204 even for absent tables → false-positive 0).
    let q = sb.from("tasks").select("id", { count: "exact" }).eq("user_id", UID);
    if (opts.status) q = q.eq("status", opts.status);
    if (opts.statusNot) q = q.neq("status", opts.statusNot);
    if (opts.dueGte) q = q.gte("due_date", opts.dueGte.toISOString());
    if (opts.dueLte) q = q.lte("due_date", opts.dueLte.toISOString());
    if (opts.dueLt) q = q.lt("due_date", opts.dueLt.toISOString());
    const res = await q.limit(1);
    assertReady(res);
    if (res.error) throw new Error(res.error.message);
    return res.count ?? 0;
  }, () =&gt; P.countTasksPrisma(opts));
}

// ============================

… (truncated for brevity; see the full file in the project repository) …
```

src/lib/data-prisma.ts

```
import { db } from "@/lib/db";
import type { Task, Note, Project, Conversation, ChatMessage, Priority, TaskStatus, Settings,
MemoryEntry } from "@/lib/types";

// ============================================================================
// Prisma+SQLite fallback for the data layer. Used automatically when the
// Supabase schema isn't applied yet (anon key can't run DDL). Once the user
// runs supabase/schema.sql, the Supabase path takes over and this is dormant.
// All data is scoped to a single local Operator account (find-or-create).
// ============================================================================

const OPERATOR_EMAIL = "operator@jarvis.local";

async function operatorId(): Promise&lt;string&gt; {
  let u = await db.user.findUnique({ where: { email: OPERATOR_EMAIL } });
  if (!u) {
    u = await db.user.create({
      data: { email: OPERATOR_EMAIL, displayName: "Operator", passwordHash: "local-only:no-
password" },
    });
    // seed default settings
    const defaults: { key: string; value: string }[] = [
      { key: "model", value: "auto" },
      { key: "responseStyle", value: "balanced" },
      { key: "voiceEnabled", value: "true" },
      { key: "ttsVoice", value: "default" },
      { key: "ttsRate", value: "1" },
      { key: "theme", value: "dark" },
      { key: "reducedMotion", value: "false" },
      { key: "memoryEnabled", value: "true" },
    ];
    for (const d of defaults) {
      await db.preference.upsert({
        where: { userId_category_key: { userId: u.id, category: "settings", key: d.key } },
        update: {},
        create: { userId: u.id, category: "settings", key: d.key, value: d.value },
      });
    }
  }
  return u.id;
}

function taskSer(t: any): Task {
  return {
    id: t.id, title: t.title, description: t.description ?? "", priority: t.priority, status: t.
status,
    category: t.category ?? "", dueDate: t.dueDate ? t.dueDate.toISOString() : null, projectId: t.
projectId ?? null,
    createdAt: t.createdAt.toISOString(), updatedAt: t.updatedAt.toISOString(),
    completedAt: t.completedAt ? t.completedAt.toISOString() : null,
  };
}
function noteSer(n: any): Note {
  return { id: n.id, title: n.title, content: n.content ?? "", projectId: n.projectId ?? null,
createdAt: n.createdAt.toISOString(), updatedAt: n.updatedAt.toISOString() };
}
function projSer(p: any, taskCount = 0, done = 0, noteCount = 0): Project {
  return { id: p.id, name: p.name, description: p.description ?? "", goal: p.goal ?? "", createdAt:
p.createdAt.toISOString(), updatedAt: p.updatedAt.toISOString(), taskCount, completedTaskCount:
done, noteCount, progress: taskCount ? Math.round((done / taskCount) * 100) : 0 };
}

// ---- Tasks ------------------------------------------------------------
export async function listTasksPrisma(opts: { view?: string; q?: string; projectId?: string | null
}): Promise&lt;Task[]&gt; {
  const uid = await operatorId();
  const where: any = { userId: uid };
  const view = opts.view ?? "all";
  if (opts.projectId) where.projectId = opts.projectId;
  if (opts.q) where.OR = [{ title: { contains: opts.q } }, { description: { contains: opts.q } }, {
category: { contains: opts.q } }];
  const now = new Date();
  const startOfDay = new Date(now); startOfDay.setHours(0, 0, 0, 0);
  const endOfDay = new Date(now); endOfDay.setHours(23, 59, 59, 999);
  const in7 = new Date(now.getTime() + 7 * 86400000);
  if (view === "today") { where.dueDate = { gte: startOfDay, lte: endOfDay }; where.status = { not:
```

```
"done" }; }
  else if (view === "upcoming") { where.dueDate = { gte: now, lte: in7 }; where.status = { not:
"done" }; }
  else if (view === "completed") { where.status = "done"; }
  else if (view === "active") { where.status = { not: "done" }; }
  const orderBy = view === "completed" ? [{ completedAt: "desc" as const }] : [{ dueDate: "asc" as
const }, { createdAt: "desc" as const }];
  const tasks = await db.task.findMany({ where, orderBy, take: 200 });
  return tasks.map(taskSer);
}

export async function getTaskPrisma(id: string): Promise&lt;Task | null&gt; {
  const uid = await operatorId();
  const t = await db.task.findFirst({ where: { id, userId: uid } });
  return t ? taskSer(t) : null;
}

export async function createTaskPrisma(data: { title: string; description?: string; priority?:
Priority; status?: TaskStatus; category?: string; dueDate?: string | null; projectId?: string |
null }): Promise&lt;Task&gt; {
  const uid = await operatorId();
  const t = await db.task.create({
    data: {
      userId: uid,
      title: String(data.title).slice(0, 200),
      description: String(data.description ?? "").slice(0, 4000),
      priority: (data.priority as Priority) ?? "medium",
      status: (data.status as TaskStatus) ?? "todo",
      category: String(data.category ?? "").slice(0, 60),
      dueDate: data.dueDate ? new Date(data.dueDate) : null,
      projectId: data.projectId ?? null,
      completedAt: data.status === "done" ? new Date() : null,
    },
  });
  return taskSer(t);
}

export async function updateTaskPrisma(id: string, patch: Partial&lt;Task&gt;): Promise&lt;Task |
null&gt; {
  const uid = await operatorId();
  const existing = await db.task.findFirst({ where: { id, userId: uid } });
  if (!existing) return null;
  const data: any = {};
  if (patch.title !== undefined) data.title = String(patch.title).slice(0, 200);
  if (patch.description !== undefined) data.description = String(patch.description).slice(0, 4000);
  if (patch.priority !== undefined &amp;&amp; ["low", "medium", "high"].includes(patch.priority))
data.priority = patch.priority;
  if (patch.category !== undefined) data.category = String(patch.category).slice(0, 60);
  if (patch.dueDate !== undefined) data.dueDate = patch.dueDate ? new Date(patch.dueDate) : null;
  if (patch.projectId !== undefined) data.projectId = patch.projectId || null;
  if (patch.status !== undefined &amp;&amp; ["todo", "in_progress", "done"].includes(patch.status))
{
    data.status = patch.status;
    if (patch.status === "done" &amp;&amp; !existing.completedAt) data.completedAt = new Date();
    if (patch.status !== "done") data.completedAt = null;
  }
  const t = await db.task.update({ where: { id }, data });
  return taskSer(t);
}

export async function deleteTaskPrisma(id: string): Promise&lt;void&gt; {
  const uid = await operatorId();
  await db.task.deleteMany({ where: { id, userId: uid } });
}

export async function countTasksPrisma(opts: { status?: TaskStatus; statusNot?: TaskStatus; dueLt?:
Date; dueGte?: Date; dueLte?: Date }): Promise&lt;number&gt; {
  const uid = await operatorId();
  const where: any = { userId: uid };
  if (opts.status) where.status = opts.status;
  if (opts.statusNot) where.status = { not: opts.statusNot };
  if (opts.dueGte || opts.dueLte || opts.dueLt) {
    where.dueDate = {};
    if (opts.dueGte) where.dueDate.gte = opts.dueGte;
    if (opts.dueLte) where.dueDate.lte = opts.dueLte;
    if (opts.dueLt) where.dueDate.lt = opts.dueLt;
  }
  return db.task.count({ where });
}

// ---- Notes -----------------------------------------------------------------
```

```
export async function listNotesPrisma(opts: { q?: string; projectId?: string | null }): Promise&lt;
Note[]&gt; {
  const uid = await operatorId();
  const where: any = { userId: uid };
  if (opts.projectId) where.projectId = opts.projectId;
  if (opts.q) where.OR = [{ title: { contains: opts.q } }, { content: { contains: opts.q } }];
  const notes = await db.note.findMany({ where, orderBy: { updatedAt: "desc" }, take: 200 });
  return notes.map(noteSer);
}

export async function getNotePrisma(id: string): Promise&lt;Note | null&gt; {
  const uid = await operatorId();
  const n = await db.note.findFirst({ where: { id, userId: uid } });
  return n ? noteSer(n) : null;
}

export async function createNotePrisma(data: { title: string; content?: string; projectId?: string
| null }): Promise&lt;Note&gt; {
  const uid = await operatorId();
  const n = await db.note.create({
    data: { userId: uid, title: String(data.title).slice(0, 200), content: String(data.content ??
"").slice(0, 20000), projectId: data.projectId ?? null },
  });
  return noteSer(n);
}

export async function updateNotePrisma(id: string, patch: Partial&lt;Note&gt;): Promise&lt;Note |
null&gt; {
  const uid = await operatorId();
  const existing = await db.note.findFirst({ where: { id, userId: uid } });
  if (!existing) return null;
  const data: any = {};
  if (patch.title !== undefined) data.title = String(patch.title).slice(0, 200);
  if (patch.content !== undefined) data.content = String(patch.content).slice(0, 20000);
  if (patch.projectId !== undefined) data.projectId = patch.projectId || null;
  const n = await db.note.update({ where: { id }, data });
  return noteSer(n);
}

export async function deleteNotePrisma(id: string): Promise&lt;void&gt; {
  const uid = await operatorId();
  await db.note.deleteMany({ where: { id, userId: uid } });
}

export async function countNotesPrisma(opts: { projectId?: string }): Promise&lt;number&gt; {
  const uid = await operatorId();
  const where: any = { userI

… (truncated for brevity; see the full file in the project repository) …
```

src/lib/preferences.ts

```
import {
  getSettings as getSettingsMap,
  saveSetting as saveSettingRow,
  getMemory,
  saveMemory as saveMemoryRow,
  deleteMemory as deleteMemoryRow,
  clearMemory,
} from "@/lib/data";
import type { Settings, MemoryEntry } from "@/lib/types";

export async function getSettings(): Promise&lt;Settings&gt; {
  const map = await getSettingsMap();
  return {
    displayName: "Operator",
    model: map.model ?? "auto",
    responseStyle: (map.responseStyle as Settings["responseStyle"]) ?? "balanced",
    voiceEnabled: map.voiceEnabled !== "false",
    ttsVoice: map.ttsVoice ?? "default",
    ttsRate: Number(map.ttsRate ?? 1),
    theme: (map.theme as Settings["theme"]) ?? "dark",
    reducedMotion: map.reducedMotion === "true",
    memoryEnabled: map.memoryEnabled !== "false",
  };
}

export async function saveSetting(userId: string, key: string, value: string): Promise&lt;void&gt; {
  void userId; // single-operator; id baked into the data layer
```

```
  await saveSettingRow(key, value);
}

export async function updateDisplayName(_userId: string, name: string): Promise&lt;void&gt; {
  // Display name is fixed to "Operator" in the single-user model; store it as
  // a setting so Settings can still show/edit it.
  await saveSettingRow("displayName", name.trim().slice(0, 60) || "Operator");
}

export async function getMemoryList(): Promise&lt;MemoryEntry[]&gt; {
  return getMemory();
}

export async function saveMemory(userId: string, key: string, value: string): Promise&lt;
MemoryEntry&gt; {
  void userId;
  const k = key.trim().slice(0, 80);
  const v = value.trim().slice(0, 2000);
  if (!k || !v) throw new Error("Key and value are required.");
  return saveMemoryRow(k, v);
}

export async function deleteMemory(userId: string, id: string): Promise&lt;void&gt; {
  void userId;
  await deleteMemoryRow(id);
}

export async function clearAllMemory(userId: string): Promise&lt;void&gt; {
  void userId;
  await clearMemory();
}

// Returns memory entries as a compact string for inclusion in the system prompt.
export async function getMemoryContext(userId: string): Promise&lt;string&gt; {
  void userId;
  const settings = await getSettings();
  if (!settings.memoryEnabled) return "";
  const mem = await getMemory();
  if (mem.length === 0) return "";
  const lines = mem.map((m) =&gt; `- ${m.key}: ${m.value}`).join("\n");
  return `\n\nSaved user preferences (respect these; the user may edit or delete them):\n${lines}`;
}
```

src/lib/prompt.ts

```
import { listTools } from "@/lib/tools/registry";
import { getSettings, getMemoryContext } from "@/lib/preferences";
import type { Settings } from "@/lib/types";
import type { ParsedToolCall } from "@/lib/types";

const TOOL_CALL_RE = /&lt;&lt;&lt;TOOL_CALL:([a-z_]+)&gt;&gt;&gt;\s*([\s\S]*?)\s*&lt;&lt;&lt;
END_TOOL_CALL&gt;&gt;&gt;/g;

/** Extract tool-call blocks from a model output. Returns prose + parsed calls. */
export function parseToolCalls(text: string): { prose: string; calls: ParsedToolCall[] } {
  const calls: ParsedToolCall[] = [];
  let prose = text;
  let match: RegExpExecArray | null;
  TOOL_CALL_RE.lastIndex = 0;
  const cleaned = text.replace(TOOL_CALL_RE, (_full, name: string, body: string) =&gt; {
    try {
      const input = JSON.parse(body.trim());
      calls.push({ tool: String(name), input });
      return "";
    } catch {
      return "";
    }
  });
  prose = cleaned.replace(/\n{3,}/g, "\n\n").trim();
  return { prose, calls };
}

function styleInstruction(style: Settings["responseStyle"]): string {
  switch (style) {
    case "concise":
      return "Keep answers short and to the point. Prefer 1-3 sentences unless the user asks for
depth.";
    case "detailed":
      return "Provide thorough, well-structured answers with context and examples where useful.";
```

```
    default:
      return "Balance brevity and depth. Be clear and complete without padding.";
  }
}

const BASE_SYSTEM = `You are JARVIS, a calm, capable, and honest personal AI assistant.

Core principles:
- Be genuinely helpful: answer questions, brainstorm, explain, help with code, and assist with
study.
- Be honest. If you are not certain of a live fact, say so plainly. Never fabricate dates, prices,
news, weather, or live data.
- You cannot access the user's email, files, contacts, financial accounts, or devices. Do not
pretend to.
- You cannot control the operating system, install software, or change security settings. If asked,
explain the limitation and offer a safe alternative.
- When the user asks you to take a consequential action (create a task, save a note, open a link),
use the tool-call protocol below. The user will be asked to confirm before anything is saved or
changed.
- Respect the user's saved preferences. Do not store new preferences silently; ask first.
- Use Markdown for formatting. Use fenced code blocks with language tags for code.

Tool-call protocol:
When the user asks you to perform a supported action, emit a single fenced block in EXACTLY this
format and nothing else inside it:

&lt;&lt;&lt;TOOL_CALL:TOOL_NAME&gt;&gt;&gt;
{ "field": "value" }
&lt;&lt;&lt;END_TOOL_CALL&gt;&gt;&gt;

Rules for tool calls:
- Use only the tool names listed below.
- All dates must be ISO 8601 strings (e.g. "2025-10-05T10:00:00.000Z"). For relative terms like
"tomorrow", compute the actual date. Never output words like "tomorrow" in a date field.
- For create_task, include title (required), and optionally priority (low|medium|high), category,
description, and dueDate (ISO).
- For save_note, include title and content (markdown). projectId is optional.
- For web_search, include query (required) and optionally num (1-10) and recencyDays. Use this when
the user asks about current events, recent news, live prices, or facts you cannot verify from
memory. After the search completes, summarize the results for the user and always cite the source
URLs.
- For generate_image, include prompt (required, detailed and visual) and optionally size (one of
1024x1024, 1344x768, 768x1344, 1440x720, 720x1440). The user must approve before generation runs.
- You may add a short sentence before or after the block to explain what you propose. The system
will parse the block, show the user a confirmation card, and execute only after approval. Do not
claim the action is done — only that you have proposed it.`;

export async function buildSystemPrompt(_userId: string): Promise&lt;{ system: string; model:
string; style: Settings["responseStyle"] }&gt; {
  const settings = await getSettings();
  const tools = listTools();
  const toolList = tools
    .map((t) =&gt; `- ${t.name}: ${t.description} (fields: ${t.schema.map((f) =&gt; f.name).join(",
")})`)
    .join("\n");
  const memory = await getMemoryContext();
  const system = [
    BASE_SYSTEM,
    `\nResponse style: ${styleInstruction(settings.responseStyle)}`,
    `\nAvailable tools:\n${toolList}`,
    memory,
  ].join("\n");
  return { system, model: settings.model, style: settings.responseStyle };
}
```

src/lib/tools/registry.ts

```
import { chatCompletion, generateImage, webSearch, type ImageSize } from "@/lib/jarvis-ai";
import { createTask, createNote, listTasks, listNotes, addAudit } from "@/lib/data";
import type { ToolDef, ToolField, ToolResult } from "@/lib/types";

// ---- Field schemas ----------------------------------------------------
const taskFields: ToolField[] = [
  { name: "title", type: "string", label: "Title", required: true, placeholder: "Call the
optometrist" },
  { name: "description", type: "string", label: "Description", required: false, placeholder:
"Optional details" },
  { name: "priority", type: "string", label: "Priority", required: false, options: ["low", "medium",
 "high"] },
```

```
  { name: "category", type: "string", label: "Category", required: false, placeholder: "e.g.
health" },
  { name: "dueDate", type: "date", label: "Due date", required: false },
];

const noteFields: ToolField[] = [
  { name: "title", type: "string", label: "Title", required: true, placeholder: "Meeting notes" },
  { name: "content", type: "string", label: "Content (Markdown)", required: true, placeholder:
"Write in markdown..." },
  { name: "projectId", type: "string", label: "Project (optional)", required: false, placeholder:
"Project id" },
];

const searchFields: ToolField[] = [
  { name: "query", type: "string", label: "Query", required: true, placeholder: "dentist" },
  { name: "scope", type: "string", label: "Scope", required: false, options: ["all", "tasks",
"notes"] },
];

const summarizeFields: ToolField[] = [
  { name: "text", type: "string", label: "Text to summarize", required: true, placeholder: "Paste
content..." },
];

const URL_ALLOWLIST = [
  "https://en.wikipedia.org",
  "https://www.google.com",
  "https://github.com",
  "https://developer.mozilla.org",
];

const openUrlFields: ToolField[] = [
  { name: "url", type: "string", label: "URL", required: true, placeholder: "https://..." },
];

// ---- Helpers -----------------------------------------------------------
function bad(msg: string): ToolResult {
  return { ok: false, status: "error", message: msg };
}

function validateFields(input: Record&lt;string, unknown&gt;, fields: ToolField[]): { ok: boolean;
error?: string; value?: Record&lt;string, unknown&gt; } {
  const out: Record&lt;string, unknown&gt; = {};
  for (const f of fields) {
    const raw = input[f.name];
    if (raw === undefined || raw === null || raw === "") {
      if (f.required) return { ok: false, error: `Missing required field: ${f.label}` };
      continue;
    }
    if (f.type === "string") {
      const v = String(raw).slice(0, 5000);
      if (f.options &amp;&amp; !f.options.includes(v)) return { ok: false, error: `Invalid option
for ${f.label}` };
      out[f.name] = v;
    } else if (f.type === "number") {
      const n = Number(raw);
      if (!isFinite(n)) return { ok: false, error: `${f.label} must be a number` };
      out[f.name] = n;
    } else if (f.type === "boolean") {
      out[f.name] = Boolean(raw);
    } else if (f.type === "date") {
      const d = new Date(String(raw));
      if (isNaN(d.getTime())) return { ok: false, error: `${f.label} is not a valid date` };
      out[f.name] = d.toISOString();
    }
  }
  return { ok: true, value: out };
}

// ---- Tool definitions --------------------------------------------------
const createTaskTool: ToolDef = {
  name: "create_task",
  label: "Create a task",
  description: "Adds a new task to the user's task list.",
  requiresConfirmation: true,
  schema: taskFields,
  async execute(input) {
    const v = validateFields(input, taskFields);
    if (!v.ok || !v.value) return bad(v.error || "Invalid input");
    const val = v.value;
    const task = await createTask({
```

```
      title: String(val.title),
      description: String(val.description ?? ""),
      priority: (val.priority as "low" | "medium" | "high") ?? "medium",
      category: String(val.category ?? ""),
      dueDate: val.dueDate ? String(val.dueDate) : null,
      projectId: val.projectId ? String(val.projectId) : null,
    });
    return { ok: true, status: "success", message: "Task created.", data: { id: task.id, title:
task.title } };
  },
};

const saveNoteTool: ToolDef = {
  name: "save_note",
  label: "Save a note",
  description: "Creates a new note with markdown content.",
  requiresConfirmation: true,
  schema: noteFields,
  async execute(input) {
    const v = validateFields(input, noteFields);
    if (!v.ok || !v.value) return bad(v.error || "Invalid input");
    const val = v.value;
    const note = await createNote({
      title: String(val.title),
      content: String(val.content).slice(0, 20000),
      projectId: val.projectId ? String(val.projectId) : null,
    });
    return { ok: true, status: "success", message: "Note saved.", data: { id: note.id, title: note.
title } };
  },
};

const searchTool: ToolDef = {
  name: "search_user_data",
  label: "Search your tasks &amp;amp; notes",
  description: "Read-only search across the user's own tasks and notes.",
  requiresConfirmation: false,
  schema: searchFields,
  async execute(input) {
    const v = validateFields(input, searchFields);
    if (!v.ok || !v.value) return bad(v.error || "Invalid input");
    const val = v.value;
    const q = String(val.query).trim();
    if (q.length &amp;lt; 2) return bad("Query too short.");
    const scope = String(val.scope ?? "all");
    const tasks = (scope === "all" || scope === "tasks") ? await listTasks({ q }) : [];
    const notes = (scope === "all" || scope === "notes") ? await listNotes({ q }) : [];
    return { ok: true, status: "success", message: `Found ${tasks.length} tasks, ${notes.length}
notes.`, data: { tasks, notes } };
  },
};

const summarizeTool: ToolDef = {
  name: "summarize_text",
  label: "Summarize content",
  description: "Produces a concise summary of the provided text using z.ai.",
  requiresConfirmation: false,
  schema: summarizeFields,
  async execute(input) {
    const v = validateFields(input, summarizeFields);
    if (!v.ok || !v.value) return bad(v.error || "Invalid input");
    const text = String(v.value.text).slice(0, 12000);
    if (text.length &amp;lt; 20) return bad("Text too short to summarize.");
    try {
      const summary = await chatCompletion(
        [
          { role: "system", content: "You are JARVIS. Summarize the user's text in 3-5 concise
bullet points. Use markdown." },
          { role: "user", content: text },
        ],
        { maxTokens: 600 }
      );
      return { ok: true, status: "success", message: "Summary ready.", data: { summary } };
    } catch (e) {
      return bad(e instanceof Error ? e.message : "Summarization failed.");
    }
  },
};

const openUrlTool: ToolDef = {
  name: "open_url",
```

```
  label: "Open an allowlisted URL",
  description: "Returns a URL from a strict allowlist for the client to open in a new tab.",
  requiresConfirmation: true,
  schema: openUrlFields,
  async execute(input) {
    const v = validateFields(input, openUrlFields);
    if (!v.ok || !v.value) return bad(v.error || "Invalid input");
    const url = String(v.value.url);
    const allowed = URL_ALLOWLIST.some((base) =&gt; url.startsWith(base));
    if (!allowed) {
      return { ok: false, status: "denied", message: `URL not in allowlist. Allowed bases: ${
URL_ALLOWLIST.join(", ")}` };
    }
    return { ok: true, status: "success", message: "URL approved to open.", data: { url } };
  },
};

// ---- Advanced AI tools (web search + image generation) -------------------
const webSearchFields: ToolField[] = [
  { name: "query", type: "string", label: "Search query", required: true, placeholder: "latest
TypeScript features" },
  { name: "num", type: "number", label: "Number of results", required: false, placeholder: "5" },
  { name: "recencyDays", type: "number", label: "Recency (days)", required: false, placeholder:
"30" },
];

const imageGenFields: ToolField[] = [
  { name: "prompt", type: "string", label: "Image prompt", required: true, placeholder: "A serene
mountain lake at sunset" },
  { name: "size", type: "string", label: "Size", required: false, options: ["1024x1024", "1344x768",
 "768x1344", "1440x720", "720x1440"] },
];

const webSearchTool: ToolDef = {
  name: "web_search",
  label: "Search the web",
  description: "Fetches live web results via z.ai web_search. Use for current events, docs, facts
you can't verify from memory.",
  requiresConfirmation: false,
  schema: webSearchFields,
  async execute(input) {
    const v = validateFields(input, webSearchFields);
    if (!v.ok || !v.value) return bad(v.error || "Invalid input");
    const query = String(v.value.query).trim();
    const num = v.value.num ? Math.min(Math.max(Number(v.value.num), 1), 10) : 5;
    const recencyDays = v.value.recencyDays ? Number(v.value.recencyDays) : undefined;
    try {
      const results = await webSearch(query, num, recencyDays);
      return { ok: true, status: "success", message: `Found ${results.length} results for "${
query}".`, data: { results } };
    } catch (e) {
      return bad(e instanceo

… (truncated for brevity; see the full file in the project repository) …
```

State Stores

src/store/app-store.ts

```
import { create } from "zustand";

export type View =
  | "dashboard"
  | "chat"
  | "voice"
  | "tasks"
  | "notes"
  | "projects"
  | "studio"
  | "memory"
  | "settings";

interface AppState {
  view: View;
  setView: (v: View) =&gt; void;
  // Quick-action launcher state so Dashboard "New Task" can jump into Tasks view
  // with the create dialog pre-opened.
  quickAction: { target: View; action: string | null } | null;
  launchQuick: (target: View, action: string) =&gt; void;
  clearQuick: () =&gt; void;
  // Selected conversation id for the chat view.
  activeConversationId: string | null;
  setActiveConversation: (id: string | null) =&gt; void;
  // Selected project id for the projects view.
  activeProjectId: string | null;
  setActiveProject: (id: string | null) =&gt; void;
  // Sidebar collapse on desktop.
  sidebarCollapsed: boolean;
  toggleSidebar: () =&gt; void;
  // Mobile sidebar open state.
  mobileNavOpen: boolean;
  setMobileNav: (open: boolean) =&gt; void;
}

export const useApp = create&lt;AppState&gt;((set) =&gt; ({
  view: "dashboard",
  setView: (view) =&gt; set({ view }),
  quickAction: null,
  launchQuick: (target, action) =&gt; set({ quickAction: { target, action }, view: target }),
  clearQuick: () =&gt; set({ quickAction: null }),
  activeConversationId: null,
  setActiveConversation: (id) =&gt; set({ activeConversationId: id }),
  activeProjectId: null,
  setActiveProject: (id) =&gt; set({ activeProjectId: id }),
  sidebarCollapsed: false,
  toggleSidebar: () =&gt; set((s) =&gt; ({ sidebarCollapsed: !s.sidebarCollapsed })),
  mobileNavOpen: false,
  setMobileNav: (open) =&gt; set({ mobileNavOpen: open }),
}));
```

src/store/images-store.ts

```
import { create } from "zustand";

export interface GeneratedImage {
  id: string;
  src: string; // ...
  prompt: string;
  size: string;
  createdAt: string;
}

interface ImagesState {
  images: GeneratedImage[];
  addImage: (img: Omit&lt;GeneratedImage, "id" | "createdAt"&gt;) =&gt; void;
  removeImage: (id: string) =&gt; void;
  clear: () =&gt; void;
}

export const useImagesStore = create&lt;ImagesState&gt;((set) =&gt; ({
```

```
  images: [],
  addImage: (img) =&gt;
    set((s) =&gt; ({
      images: [
        { id: crypto.randomUUID(), src: img.src, prompt: img.prompt, size: img.size, createdAt: new
Date().toISOString() },
        ...s.images,
      ].slice(0, 30),
    })),
  removeImage: (id) =&gt; set((s) =&gt; ({ images: s.images.filter((i) =&gt; i.id !== id) })),
  clear: () =&gt; set({ images: [] }),
}));
```

React Hooks

src/hooks/use-mobile.ts

```
import * as React from "react"

const MOBILE_BREAKPOINT = 768

export function useIsMobile() {
  const [isMobile, setIsMobile] = React.useState&lt;boolean | undefined&gt;(undefined)

  React.useEffect(() =&gt; {
    const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT - 1}px)`)
    const onChange = () =&gt; {
      setIsMobile(window.innerWidth &lt; MOBILE_BREAKPOINT)
    }
    mql.addEventListener("change", onChange)
    setIsMobile(window.innerWidth &lt; MOBILE_BREAKPOINT)
    return () =&gt; mql.removeEventListener("change", onChange)
  }, [])

  return !!isMobile
}
```

src/hooks/use-toast.ts

```
"use client"

// Inspired by react-hot-toast library
import * as React from "react"

import type {
  ToastActionElement,
  ToastProps,
} from "@/components/ui/toast"

const TOAST_LIMIT = 1
const TOAST_REMOVE_DELAY = 1000000

type ToasterToast = ToastProps &amp;amp; {
  id: string
  title?: React.ReactNode
  description?: React.ReactNode
  action?: ToastActionElement
}

const actionTypes = {
  ADD_TOAST: "ADD_TOAST",
  UPDATE_TOAST: "UPDATE_TOAST",
  DISMISS_TOAST: "DISMISS_TOAST",
  REMOVE_TOAST: "REMOVE_TOAST",
} as const

let count = 0

function genId() {
  count = (count + 1) % Number.MAX_SAFE_INTEGER
  return count.toString()
}

type ActionType = typeof actionTypes

type Action =
  | {
    type: ActionType["ADD_TOAST"]
    toast: ToasterToast
  }
  | {
    type: ActionType["UPDATE_TOAST"]
    toast: Partial&lt;ToasterToast&gt;
  }
  | {
    type: ActionType["DISMISS_TOAST"]
    toastId?: ToasterToast["id"]
  }
```

```
  | {
    type: ActionType["REMOVE_TOAST"]
    toastId?: ToasterToast["id"]
  }

interface State {
  toasts: ToasterToast[]
}

const toastTimeouts = new Map&lt;string, ReturnType&lt;typeof setTimeout&gt;&gt;()

const addToRemoveQueue = (toastId: string) =&gt; {
  if (toastTimeouts.has(toastId)) {
    return
  }

  const timeout = setTimeout(() =&gt; {
    toastTimeouts.delete(toastId)
    dispatch({
      type: "REMOVE_TOAST",
      toastId: toastId,
    })
  }, TOAST_REMOVE_DELAY)

  toastTimeouts.set(toastId, timeout)
}

export const reducer = (state: State, action: Action): State =&gt; {
  switch (action.type) {
    case "ADD_TOAST":
      return {
        ...state,
        toasts: [action.toast, ...state.toasts].slice(0, TOAST_LIMIT),
      }

    case "UPDATE_TOAST":
      return {
        ...state,
        toasts: state.toasts.map((t) =&gt;
          t.id === action.toast.id ? { ...t, ...action.toast } : t
        ),
      }

    case "DISMISS_TOAST": {
      const { toastId } = action

      // ! Side effects ! - This could be extracted into a dismissToast() action,
      // but I'll keep it here for simplicity
      if (toastId) {
        addToRemoveQueue(toastId)
      } else {
        state.toasts.forEach((toast) =&gt; {
          addToRemoveQueue(toast.id)
        })
      }

      return {
        ...state,
        toasts: state.toasts.map((t) =&gt;
          t.id === toastId || toastId === undefined
            ? {
              ...t,
              open: false,
            }
            : t
        ),
      }
    }
    case "REMOVE_TOAST":
      if (action.toastId === undefined) {
        return {
          ...state,
          toasts: [],
        }
      }
      return {
        ...state,
        toasts: state.toasts.filter((t) =&gt; t.id !== action.toastId),
      }
  }
}
```

```
const listeners: Array&lt;(state: State) =&gt; void&gt; = []

let memoryState: State = { toasts: [] }

function dispatch(action: Action) {
  memoryState = reducer(memoryState, action)
  listeners.forEach((listener) =&gt; {
    listener(memoryState)
  })
}

type Toast = Omit&lt;ToasterToast, "id"&gt;

function toast({ ...props }: Toast) {
  const id = genId()

  const update = (props: ToasterToast) =&gt;
    dispatch({
      type: "UPDATE_TOAST",
      toast: { ...props, id },
    })
  const dismiss = () =&gt; dispatch({ type: "DISMISS_TOAST", toastId: id })

  dispatch({
    type: "ADD_TOAST",
    toast: {
      ...props,
      id,
      open: true,
      onOpenChange: (open) =&gt; {
        if (!open) dismiss()
      },
    },
  })

  return {
    id: id,
    dismiss,
    update,
  }
}

function useToast() {
  const [state, setState] = React.useState&lt;State&gt;(memoryState)

  React.useEffect(() =&gt; {
    listeners.push(setState)
    return () =&gt; {
      const index = listeners.indexOf(setState)
      if (index &gt; -1) {
        listeners.splice(index, 1)
      }
    }
  }, [state])

  return {
    ...state,
    toast,
    dismiss: (toastId?: string) =&gt; dispatch({ type: "DISMISS_TOAST", toastId }),
  }
}

export { useToast, toast }
```

src/hooks/use-settings.ts

```
"use client";

import { useQuery, useQueryClient } from "@tanstack/react-query";
import { useCallback } from "react";
import type { Settings } from "@/lib/types";

export function useSettings() {
  const qc = useQueryClient();
  const q = useQuery&lt;{ settings: Settings; models: { id: string; label: string }[] }&gt;({
    queryKey: ["settings"],
    queryFn: async () =&gt; {
      const r = await fetch("/api/settings", { cache: "no-store" });
      const j = await r.json();
```

```
      if (!j.ok) throw new Error(j.error || "Failed to load settings");
      return j;
    },
  });
  const update = useCallback(
    async (patch: Partial&lt;Settings&gt; &amp; { displayName?: string }) =&gt; {
      const r = await fetch("/api/settings", {
        method: "PATCH",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(patch),
      });
      const j = await r.json();
      if (!j.ok) throw new Error(j.error || "Failed to save setting");
      qc.setQueryData(["settings"], j);
      return j.settings as Settings;
    },
    [qc]
  );
  return { settings: q.data?.settings, models: q.data?.models, update, ...q };
}
```

src/hooks/use-data.ts

```
"use client";

import { useQueryClient, useMutation, useQuery } from "@tanstack/react-query";
import type { Task, Note, Project } from "@/lib/types";

export function useTasks(view: string = "all", q = "", projectId?: string | null) {
  const params = new URLSearchParams();
  if (view) params.set("view", view);
  if (q) params.set("q", q);
  if (projectId) params.set("projectId", projectId);
  return useQuery&lt;{ tasks: Task[] }&gt;({
    queryKey: ["tasks", view, q, projectId ?? ""],
    queryFn: async () =&gt; {
      const r = await fetch(`/api/tasks?${params}`, { cache: "no-store" });
      const j = await r.json();
      if (!j.ok) throw new Error(j.error);
      return j;
    },
  });
}

export function useCreateTask() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async (payload: Partial&lt;Task&gt;) =&gt; {
      const r = await fetch("/api/tasks", { method: "POST", headers: { "Content-Type": "application/
json" }, body: JSON.stringify(payload) });
      const j = await r.json();
      if (!j.ok) throw new Error(j.error);
      return j.task as Task;
    },
    onSuccess: () =&gt; qc.invalidateQueries({ queryKey: ["tasks"] }),
  });
}

export function useUpdateTask() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async ({ id, patch }: { id: string; patch: Partial&lt;Task&gt; }) =&gt; {
      const r = await fetch(`/api/tasks/${id}`, { method: "PATCH", headers: { "Content-Type":
"application/json" }, body: JSON.stringify(patch) });
      const j = await r.json();
      if (!j.ok) throw new Error(j.error);
      return j.task as Task;
    },
    onSuccess: () =&gt; qc.invalidateQueries({ queryKey: ["tasks"] }),
  });
}

export function useDeleteTask() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async (id: string) =&gt; {
      const r = await fetch(`/api/tasks/${id}`, { method: "DELETE" });
      const j = await r.json();
      if (!j.ok) throw new Error(j.error);
```

```
    },
    onSuccess: () =&gt; qc.invalidateQueries({ queryKey: ["tasks"] }),
  });
}

export function useNotes(q = "", projectId?: string | null) {
  const params = new URLSearchParams();
  if (q) params.set("q", q);
  if (projectId) params.set("projectId", projectId);
  return useQuery&lt;{ notes: Note[] }&gt;({
    queryKey: ["notes", q, projectId ?? ""],
    queryFn: async () =&gt; {
      const r = await fetch(`/api/notes?${params}`, { cache: "no-store" });
      const j = await r.json();
      if (!j.ok) throw new Error(j.error);
      return j;
    },
  });
}

export function useCreateNote() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async (payload: Partial&lt;Note&gt;) =&gt; {
      const r = await fetch("/api/notes", { method: "POST", headers: { "Content-Type": "application/
json" }, body: JSON.stringify(payload) });
      const j = await r.json();
      if (!j.ok) throw new Error(j.error);
      return j.note as Note;
    },
    onSuccess: () =&gt; qc.invalidateQueries({ queryKey: ["notes"] }),
  });
}

export function useUpdateNote() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async ({ id, patch }: { id: string; patch: Partial&lt;Note&gt; }) =&gt; {
      const r = await fetch(`/api/notes/${id}`, { method: "PATCH", headers: { "Content-Type":
"application/json" }, body: JSON.stringify(patch) });
      const j = await r.json();
      if (!j.ok) throw new Error(j.error);
      return j.note as Note;
    },
    onSuccess: () =&gt; qc.invalidateQueries({ queryKey: ["notes"] }),
  });
}

export function useDeleteNote() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async (id: string) =&gt; {
      const r = await fetch(`/api/notes/${id}`, { method: "DELETE" });
      const j = await r.json();
      if (!j.ok) throw new Error(j.error);
    },
    onSuccess: () =&gt; qc.invalidateQueries({ queryKey: ["notes"] }),
  });
}

export function useProjects(q = "") {
  const params = new URLSearchParams();
  if (q) params.set("q", q);
  return useQuery&lt;{ projects: Project[] }&gt;({
    queryKey: ["projects", q],
    queryFn: async () =&gt; {
      const r = await fetch(`/api/projects?${params}`, { cache: "no-store" });
      const j = await r.json();
      if (!j.ok) throw new Error(j.error);
      return j;
    },
  });
}

export function useCreateProject() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async (payload: { name: string; description?: string; goal?: string }) =&gt; {
      const r = await fetch("/api/projects", { method: "POST", headers: { "Content-Type":
"application/json" }, body: JSON.stringify(payload) });
      const j = await r.json();
```

```
      if (!j.ok) throw new Error(j.error);
      return j.project as Project;
    },
    onSuccess: () =&gt; qc.invalidateQueries({ queryKey: ["projects"] }),
  });
}
```

src/hooks/use-tools.ts

```
"use client";

import { useQuery } from "@tanstack/react-query";
import type { ToolField } from "@/lib/types";

export function useTools() {
  return useQuery&lt;{ tools: { name: string; label: string; description: string;
requiresConfirmation: boolean; schema: ToolField[] }[] }&gt;({
    queryKey: ["tools"],
    queryFn: async () =&gt; {
      const r = await fetch("/api/tools", { cache: "no-store" });
      const j = await r.json();
      if (!j.ok) throw new Error(j.error);
      return j;
    },
    staleTime: 60_000,
  });
}

export async function runTool(name: string, input: Record&lt;string, unknown&gt;) {
  const r = await fetch("/api/tools", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name, input }),
  });
  return r.json();
}
```

src/hooks/use-supabase-setup.ts

```
"use client";

import { useQuery } from "@tanstack/react-query";

interface SetupState {
  ok: boolean;
  configured: boolean;
  ready: boolean;
  tables: string[];
  error?: string;
}

export function useSupabaseSetup() {
  return useQuery&lt;SetupState&gt;({
    queryKey: ["supabase-setup"],
    queryFn: async () =&gt; {
      const r = await fetch("/api/supabase/setup", { cache: "no-store" });
      return r.json();
    },
    staleTime: 15_000,
    refetchInterval: (q) =&gt; {
      // Keep polling until the schema is ready, then stop.
      return q.state.data?.ready ? false : 10000;
    },
  });
}
```

App Shell & Layout

src/app/layout.tsx

```
import type { Metadata } from "next";
import { Geist, Geist_Mono } from "next/font/google";
import "./globals.css";
import { Toaster } from "@/components/ui/toaster";
import { Toaster as SonnerToaster } from "@/components/ui/sonner";
import { Providers } from "@/components/providers/providers";

const geistSans = Geist({ variable: "--font-geist-sans", subsets: ["latin"] });
const geistMono = Geist_Mono({ variable: "--font-geist-mono", subsets: ["latin"] });

export const metadata: Metadata = {
  title: "JARVIS AI",
  description: "A premium AI personal assistant with voice, chat, tasks, notes, and projects.",
  icons: { icon: "https://z-cdn.chatglm.cn/z-ai/static/logo.svg" },
};

export default function RootLayout({ children }: Readonly&lt;{ children: React.ReactNode }&gt;) {
  return (
    &lt;html lang="en" suppressHydrationWarning className="dark"&gt;
      &lt;body className={`${geistSans.variable} ${geistMono.variable} antialiased`}&gt;
        &lt;Providers&gt;
          {children}
          &lt;Toaster /&gt;
          &lt;SonnerToaster position="bottom-right" theme="dark" /&gt;
        &lt;/Providers&gt;
      &lt;/body&gt;
    &lt;/html&gt;
  );
}
```

src/app/page.tsx

```
"use client";

import { useAuth } from "@/components/providers/auth-provider";
import { AppShell } from "@/components/jarvis/app-shell";
import { JarvisOrb } from "@/components/jarvis/orb";

export default function Home() {
  const { user, loading } = useAuth();

  // With the sign-in page removed, the only "loading" state is the brief
  // moment while /api/auth/me auto-provisions the local Operator session.
  if (loading || !user) {
    return (
      &lt;div className="min-h-screen grid place-items-center"&gt;
        &lt;div className="flex flex-col items-center gap-4"&gt;
          &lt;JarvisOrb state="processing" size={160} /&gt;
          &lt;p className="text-sm text-muted-foreground animate-pulse"&gt;Initializing JARVIS…&lt;/
p&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    );
  }
  return &lt;AppShell /&gt;;
}
```

src/app/globals.css

```
@import "tailwindcss";
@import "tw-animate-css";

@custom-variant dark (&amp;:is(.dark *));

@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: var(--font-geist-sans);
  --font-mono: var(--font-geist-mono);
```

```
  --color-sidebar-ring: var(--sidebar-ring);
  --color-sidebar-border: var(--sidebar-border);
  --color-sidebar-accent-foreground: var(--sidebar-accent-foreground);
  --color-sidebar-accent: var(--sidebar-accent);
  --color-sidebar-primary-foreground: var(--sidebar-primary-foreground);
  --color-sidebar-primary: var(--sidebar-primary);
  --color-sidebar-foreground: var(--sidebar-foreground);
  --color-sidebar: var(--sidebar);
  --color-chart-5: var(--chart-5);
  --color-chart-4: var(--chart-4);
  --color-chart-3: var(--chart-3);
  --color-chart-2: var(--chart-2);
  --color-chart-1: var(--chart-1);
  --color-ring: var(--ring);
  --color-input: var(--input);
  --color-border: var(--border);
  --color-destructive: var(--destructive);
  --color-accent-foreground: var(--accent-foreground);
  --color-accent: var(--accent);
  --color-muted-foreground: var(--muted-foreground);
  --color-muted: var(--muted);
  --color-secondary-foreground: var(--secondary-foreground);
  --color-secondary: var(--secondary);
  --color-primary-foreground: var(--primary-foreground);
  --color-primary: var(--primary);
  --color-popover-foreground: var(--popover-foreground);
  --color-popover: var(--popover);
  --color-card-foreground: var(--card-foreground);
  --color-card: var(--card);
  --radius-sm: calc(var(--radius) - 4px);
  --radius-md: calc(var(--radius) - 2px);
  --radius-lg: var(--radius);
  --radius-xl: calc(var(--radius) + 4px);
}

/* ---------------------------------------------------------------------------
   JARVIS theme. Dark graphite + midnight blue base, cyan / electric-blue
   accents. A light variant is provided for the Settings toggle but the dark
   variant is the canonical JARVIS look.
   --------------------------------------------------------------------------- */
:root, .dark {
  --radius: 0.75rem;
  --background: oklch(0.16 0.02 255);          /* midnight graphite */
  --foreground: oklch(0.96 0.01 220);
  --card: oklch(0.2 0.025 255);
  --card-foreground: oklch(0.97 0.01 220);
  --popover: oklch(0.2 0.03 255);
  --popover-foreground: oklch(0.97 0.01 220);
  --primary: oklch(0.78 0.16 210);             /* cyan/electric blue */
  --primary-foreground: oklch(0.16 0.03 255);
  --secondary: oklch(0.26 0.03 255);
  --secondary-foreground: oklch(0.95 0.01 220);
  --muted: oklch(0.24 0.02 255);
  --muted-foreground: oklch(0.7 0.02 220);
  --accent: oklch(0.3 0.05 215);
  --accent-foreground: oklch(0.97 0.01 220);
  --destructive: oklch(0.65 0.22 25);
  --border: oklch(1 0 0 / 8%);
  --input: oklch(1 0 0 / 12%);
  --ring: oklch(0.7 0.14 210);
  --chart-1: oklch(0.78 0.16 210);
  --chart-2: oklch(0.7 0.13 190);
  --chart-3: oklch(0.72 0.14 235);
  --chart-4: oklch(0.66 0.17 160);
  --chart-5: oklch(0.6 0.2 30);
  --sidebar: oklch(0.18 0.025 255);
  --sidebar-foreground: oklch(0.95 0.01 220);
  --sidebar-primary: oklch(0.78 0.16 210);
  --sidebar-primary-foreground: oklch(0.16 0.03 255);
  --sidebar-accent: oklch(0.26 0.04 215);
  --sidebar-accent-foreground: oklch(0.97 0.01 220);
  --sidebar-border: oklch(1 0 0 / 8%);
  --sidebar-ring: oklch(0.7 0.14 210);
}

.light {
  --background: oklch(0.98 0.005 220);
  --foreground: oklch(0.2 0.02 255);
  --card: oklch(1 0 0);
  --card-foreground: oklch(0.2 0.02 255);
  --popover: oklch(1 0 0);
```

```
  --popover-foreground: oklch(0.2 0.02 255);
  --primary: oklch(0.55 0.18 215);
  --primary-foreground: oklch(0.98 0.01 220);
  --secondary: oklch(0.96 0.01 220);
  --secondary-foreground: oklch(0.25 0.02 255);
  --muted: oklch(0.96 0.01 220);
  --muted-foreground: oklch(0.45 0.02 255);
  --accent: oklch(0.94 0.02 215);
  --accent-foreground: oklch(0.25 0.02 255);
  --destructive: oklch(0.55 0.24 27);
  --border: oklch(0.9 0.005 220);
  --input: oklch(0.9 0.005 220);
  --ring: oklch(0.55 0.18 215);
  --sidebar: oklch(0.97 0.006 220);
  --sidebar-foreground: oklch(0.2 0.02 255);
  --sidebar-primary: oklch(0.55 0.18 215);
  --sidebar-primary-foreground: oklch(0.98 0.01 220);
  --sidebar-accent: oklch(0.94 0.02 215);
  --sidebar-accent-foreground: oklch(0.25 0.02 255);
  --sidebar-border: oklch(0.9 0.005 220);
  --sidebar-ring: oklch(0.55 0.18 215);
}

@layer base {
  * {
    @apply border-border outline-ring/50;
  }
  html, body {
    @apply bg-background text-foreground;
    min-height: 100%;
  }
  body {
    background-image:
      radial-gradient(900px 600px at 12% -8%, oklch(0.28 0.06 215 / 0.55), transparent 60%),
      radial-gradient(700px 500px at 100% 0%, oklch(0.22 0.08 195 / 0.4), transparent 55%),
      radial-gradient(800px 700px at 50% 120%, oklch(0.2 0.05 240 / 0.45), transparent 60%);
    background-attachment: fixed;
  }
}

/* Subtle glass panel utility used across JARVIS surfaces. */
@layer components {
  .jarvis-panel {
    @apply rounded-xl border border-border/70 bg-card/70 backdrop-blur-xl;
    box-shadow: 0 1px 0 oklch(1 0 0 / 0.04) inset, 0 18px 50px -28px oklch(0.78 0.16 210 / 0.35);
  }
  .jarvis-panel-solid {
    @apply rounded-xl border border-border bg-card;
  }
  .jarvis-glow {
    box-shadow: 0 0 0 1px oklch(0.78 0.16 210 / 0.25), 0 0 32px -6px oklch(0.78 0.16 210 / 0.45);
  }
  .jarvis-divider {
    background: linear-gradient(90deg, transparent, oklch(0.78 0.16 210 / 0.5), transparent);
    height: 1px;
  }
  .jarvis-scrollbar::-webkit-scrollbar { width: 8px; height: 8px; }
  .jarvis-scrollbar::-webkit-scrollbar-track { background: transparent; }
  .jarvis-scrollbar::-webkit-scrollbar-thumb {
    background: oklch(0.78 0.16 210 / 0.25);
    border-radius: 9999px;
  }
  .jarvis-scrollbar::-webkit-scrollbar-thumb:hover { background: oklch(0.78 0.16 210 / 0.45); }

  .jarvis-orb-ring {
    background: conic-gradient(from 0deg, transparent, oklch(0.78 0.16 210 / 0.9), transparent 60%,
oklch(0.7 0.13 190 / 0.6), transparent);
  }
}

@keyframes jarvis-spin { to { transform: rotate(360deg); } }
@keyframes jarvis-spin-rev { to { transform: rotate(-360deg); } }
@keyframes jarvis-pulse-soft {
  0%, 100% { transform: scale(1); opacity: 0.85; }
  50% { transform: scale(1.04); opacity: 1; }
}
@keyframes jarvis-breathe {
  0%, 100% { opacity: 0.55; }
  50% { opacity: 1; }
}
.jarvis-spin { animation: jarvis-spin 8s linear infinite; }
```

```
.jarvis-spin-rev { animation: jarvis-spin-rev 12s linear infinite; }
.jarvis-pulse-soft { animation: jarvis-pulse-soft 2.6s ease-in-out infinite; }
.jarvis-breathe { animation: jarvis-breathe 3.2s ease-in-out infinite; }

/* Reduced motion: disable nonessential animation. */
@media (prefers-reduced-motion: reduce) {
  .jarvis-spin, .jarvis-spin-rev, .jarvis-pulse-soft, .jarvis-breathe { animation: none !important;
}
}
.reduced-motion .jarvis-spin,
.reduced-motion .jarvis-spin-rev,
.reduced-motion .jarvis-pulse-soft,
.reduced-motion .jarvis-breathe { animation: none !important; }

/* Markdown prose styling inside chat/notes. */
.jarvis-prose { @apply text-sm leading-relaxed; }
.jarvis-prose h1 { @apply text-lg font-semibold mt-3 mb-2; }
.jarvis-prose h2 { @apply text-base font-semibold mt-3 mb-2; }
.jarvis-prose h3 { @apply text-sm font-semibold mt-2 mb-1; }
.jarvis-prose p { @apply my-2; }
.jarvis-prose ul { @apply list-disc pl-5 my-2; }
.jarvis-prose ol { @apply list-decimal pl-5 my-2; }
.jarvis-prose li { @apply my-0.5; }
.jarvis-prose a { @apply text-primary underline underline-offset-2; }
.jarvis-prose blockquote { @apply border-l-2 border-primary/50 pl-3 italic text-muted-foreground my-
2; }
.jarvis-prose code:not(pre code) { @apply rounded bg-secondary/60 px-1 py-0.5 text-[0.85em] font-
mono; }
.jarvis-prose pre { @apply my-3 rounded-lg border border-border/70 bg-background/60 overflow-hidden;
 }
.jarvis-prose pre code { @apply block p-3 text-xs font-mono overflow-x-auto; }
.jarvis-prose table { @apply w-full my-3 text-xs border-collapse; }
.jarvis-prose th, .jarvis-prose td { @apply border border-border/60 px-2 py-1; }
```

src/components/providers/providers.tsx

```
"use client";

import { ThemeProvider as NextThemes } from "next-themes";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { useState } from "react";
import { AuthProvider } from "@/components/providers/auth-provider";

export function Providers({ children }: { children: React.ReactNode }) {
  const [qc] = useState(
    () =&gt;
      new QueryClient({
        defaultOptions: {
          queries: { staleTime: 30_000, retry: 1, refetchOnWindowFocus: false },
        },
      })
  );
  return (
    &lt;NextThemes attribute="class" defaultTheme="dark" enableSystem={false}
disableTransitionOnChange&gt;
      &lt;QueryClientProvider client={qc}&gt;
        &lt;AuthProvider&gt;{children}&lt;/AuthProvider&gt;
      &lt;/QueryClientProvider&gt;
    &lt;/NextThemes&gt;
  );
}
```

src/components/providers/auth-provider.tsx

```
"use client";

import { createContext, useCallback, useContext, useEffect, useMemo, useState } from "react";
import type { SafeUser } from "@/lib/types";

interface AuthCtx {
  user: SafeUser | null;
  loading: boolean;
  refresh: () =&gt; Promise&lt;void&gt;;
}

const Ctx = createContext&lt;AuthCtx&gt;(null!);
export function useAuth() {
```

```
  const c = useContext(Ctx);
  if (!c) throw new Error("useAuth must be used within AuthProvider");
  return c;
}

// With the sign-in page removed, this provider only reflects whether the
// auto-provisioned Operator session has loaded yet. There is no sign-in or
// sign-out UI; the server's /api/auth/me always returns a user.
export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [user, setUser] = useState&lt;SafeUser | null&gt;(null);
  const [loading, setLoading] = useState(true);

  const refresh = useCallback(async () =&gt; {
    try {
      const r = await fetch("/api/auth/me", { cache: "no-store" });
      const j = await r.json();
      setUser(j.user ?? null);
    } catch {
      setUser(null);
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() =&gt; {
    refresh();
  }, [refresh]);

  const value = useMemo(() =&gt; ({ user, loading, refresh }), [user, loading, refresh]);
  return &lt;Ctx.Provider value={value}&gt;{children}&lt;/Ctx.Provider&gt;;
}
```

src/components/jarvis/app-shell.tsx

```
"use client";

import { useEffect, useMemo, useState } from "react";
import { Sidebar } from "@/components/jarvis/sidebar";
import { Topbar } from "@/components/jarvis/topbar";
import { DashboardView } from "@/components/jarvis/views/dashboard";
import { ChatView } from "@/components/jarvis/views/chat";
import { VoiceView } from "@/components/jarvis/views/voice";
import { TasksView } from "@/components/jarvis/views/tasks";
import { NotesView } from "@/components/jarvis/views/notes";
import { ProjectsView } from "@/components/jarvis/views/projects";
import { ImageStudioView } from "@/components/jarvis/views/image-studio";
import { MemoryView } from "@/components/jarvis/views/memory";
import { SettingsView } from "@/components/jarvis/views/settings";
import { useApp } from "@/store/app-store";
import { useSettings } from "@/hooks/use-settings";
import { SupabaseSetupBanner, DbStatusDot } from "@/components/jarvis/supabase-setup-banner";
import { CommandPalette } from "@/components/jarvis/command-palette";

export function AppShell() {
  const { view, sidebarCollapsed } = useApp();
  const { settings } = useSettings();
  const [micGranted, setMicGranted] = useState&lt;"unknown" | "granted" | "denied" |
"unsupported"&gt;("unknown");

  // Apply reduced-motion + theme to &lt;html&gt;.
  useEffect(() =&gt; {
    const root = document.documentElement;
    if (settings?.reducedMotion) root.classList.add("reduced-motion");
    else root.classList.remove("reduced-motion");
    if (settings?.theme === "light") root.classList.remove("dark");
    else root.classList.add("dark");
  }, [settings?.reducedMotion, settings?.theme]);

  // Probe microphone permission status (non-invasive: only checks the existing
  // permission state; does not prompt).
  useEffect(() =&gt; {
    let cancelled = false;
    async function probe() {
      if (!("permissions" in navigator)) {
        setMicGranted("unknown");
        return;
      }
      try {
        const p = await (navigator as any).permissions.query({ name: "microphone" });
```

```
        const apply = (state: string) =&gt; {
          if (cancelled) return;
          if (state === "granted") setMicGranted("granted");
          else if (state === "denied") setMicGranted("denied");
          else setMicGranted("unknown");
        };
        apply(p.state);
        p.onchange = () =&gt; apply(p.state);
      } catch {
        setMicGranted("unknown");
      }
    }
    probe();
    return () =&gt; { cancelled = true; };
  }, []);

  const View = useMemo(() =&gt; {
    switch (view) {
      case "chat": return &lt;ChatView /&gt;;
      case "voice": return &lt;VoiceView /&gt;;
      case "tasks": return &lt;TasksView /&gt;;
      case "notes": return &lt;NotesView /&gt;;
      case "projects": return &lt;ProjectsView /&gt;;
      case "studio": return &lt;ImageStudioView /&gt;;
      case "memory": return &lt;MemoryView /&gt;;
      case "settings": return &lt;SettingsView micGranted={micGranted} /&gt;;
      default: return &lt;DashboardView /&gt;;
    }
  }, [view, micGranted]);

  return (
    &lt;div className="min-h-screen flex flex-col"&gt;
      &lt;SupabaseSetupBanner /&gt;
      &lt;CommandPalette /&gt;
      &lt;div className="flex flex-1 min-h-0"&gt;
        &lt;Sidebar /&gt;
        &lt;div className="flex-1 flex flex-col min-w-0"&gt;
          &lt;Topbar micGranted={micGranted} /&gt;
          &lt;main className="flex-1 min-h-0 overflow-hidden"&gt;
            &lt;div className={"h-full transition-all " + (sidebarCollapsed ? "lg:pl-0" : "")}&gt;
              {View}
            &lt;/div&gt;
          &lt;/main&gt;
        &lt;/div&gt;
      &lt;/div&gt;
      &lt;footer className="mt-auto border-t border-border/70 bg-background/70 backdrop-blur-xl"&gt;
        &lt;div className="px-4 py-2 text-[11px] text-muted-foreground flex flex-wrap items-center
justify-between gap-2"&gt;
          &lt;span&gt;JARVIS AI · local-first personal assistant&lt;/span&gt;
          &lt;span className="flex items-center gap-3"&gt;
            &lt;DbStatusDot /&gt;
            &lt;span className="size-1.5 rounded-full bg-emerald-400 animate-pulse" /&gt;
            Data stays on this machine
          &lt;/span&gt;
        &lt;/div&gt;
      &lt;/footer&gt;
    &lt;/div&gt;
  );
}
```

src/components/jarvis/sidebar.tsx

```
"use client";

import { LayoutDashboard, MessagesSquare, Mic, ListChecks, NotebookPen, FolderKanban, Wand2, Brain,
Settings } from "lucide-react";
import { useApp, type View } from "@/store/app-store";
import { useAuth } from "@/components/providers/auth-provider";
import { cn } from "@/lib/utils";
import { Button } from "@/components/ui/button";
import { Tooltip, TooltipContent, TooltipTrigger, TooltipProvider } from "@/components/ui/tooltip";

const NAV: { id: View; label: string; icon: React.ElementType }[] = [
  { id: "dashboard", label: "Dashboard", icon: LayoutDashboard },
  { id: "chat", label: "Chat", icon: MessagesSquare },
  { id: "voice", label: "Voice", icon: Mic },
  { id: "tasks", label: "Tasks", icon: ListChecks },
  { id: "notes", label: "Notes", icon: NotebookPen },
  { id: "projects", label: "Projects", icon: FolderKanban },
```

```
  { id: "studio", label: "Studio", icon: Wand2 },
  { id: "memory", label: "Memory", icon: Brain },
  { id: "settings", label: "Settings", icon: Settings },
];

export function Sidebar() {
  const { view, setView, sidebarCollapsed, mobileNavOpen, setMobileNav } = useApp();
  const { user } = useAuth();
  const collapsed = sidebarCollapsed;

  return (
    &lt;&gt;
      {/* Mobile overlay */}
      {mobileNavOpen &amp;&amp; (
        &lt;div className="fixed inset-0 z-40 bg-black/60 lg:hidden" onClick={() =&gt; setMobileNav(
false)} aria-hidden /&gt;
      )}
      &lt;aside
        className={cn(
          "z-50 flex flex-col border-r border-sidebar-border bg-sidebar/80 backdrop-blur-xl
transition-all duration-300",
          "lg:static lg:translate-x-0",
          "fixed inset-y-0 left-0 w-64",
          mobileNavOpen ? "translate-x-0" : "-translate-x-full lg:translate-x-0",
          collapsed &amp;&amp; "lg:w-16 lg:w-16"
        )}
      &gt;
        &lt;div className="flex items-center gap-3 h-16 px-4 border-b border-sidebar-border"&gt;
          &lt;div className="size-8 rounded-full jarvis-orb-ring jarvis-spin shrink-0" /&gt;
          {!collapsed &amp;&amp; (
            &lt;div className="leading-tight"&gt;
              &lt;div className="font-semibold tracking-wide"&gt;JARVIS&lt;/div&gt;
              &lt;div className="text-[10px] uppercase tracking-[0.2em] text-muted-foreground"&gt;
AI Assistant&lt;/div&gt;
            &lt;/div&gt;
          )}
        &lt;/div&gt;

        &lt;nav className="flex-1 p-2 jarvis-scrollbar overflow-y-auto"&gt;
          &lt;TooltipProvider delayDuration={collapsed ? 200 : 99999}&gt;
            &lt;ul className="flex flex-col gap-1"&gt;
              {NAV.map((item) =&gt; {
                const active = view === item.id;
                const Icon = item.icon;
                const btn = (
                  &lt;button
                    key={item.id}
                    onClick={() =&gt; {
                      setView(item.id);
                      setMobileNav(false);
                    }}
                    className={cn(
                      "group flex w-full items-center gap-3 rounded-lg px-3 py-2 text-sm transition-
all",
                      "hover:bg-sidebar-accent/60",
                      active &amp;&amp; "bg-sidebar-accent text-sidebar-accent-foreground jarvis-
glow",
                      !active &amp;&amp; "text-sidebar-foreground/80",
                      collapsed &amp;&amp; "justify-center px-0"
                    )}
                    aria-current={active ? "page" : undefined}
                  &gt;
                    &lt;Icon className="size-4 shrink-0" /&gt;
                    {!collapsed &amp;&amp; &lt;span&gt;{item.label}&lt;/span&gt;}
                  &lt;/button&gt;
                );
                if (collapsed) {
                  return (
                    &lt;li key={item.id}&gt;
                      &lt;Tooltip&gt;
                        &lt;TooltipTrigger asChild&gt;{btn}&lt;/TooltipTrigger&gt;
                        &lt;TooltipContent side="right"&gt;{item.label}&lt;/TooltipContent&gt;
                      &lt;/Tooltip&gt;
                    &lt;/li&gt;
                  );
                }
                return &lt;li key={item.id}&gt;{btn}&lt;/li&gt;;
              })}
            &lt;/ul&gt;
          &lt;/TooltipProvider&gt;
        &lt;/nav&gt;
```

```
        &lt;div className="border-t border-sidebar-border p-3"&gt;
          &lt;div className={cn("flex items-center gap-2", collapsed &amp;&amp; "justify-
center")}&gt;
            &lt;div className="size-8 rounded-full bg-primary/20 grid place-items-center text-xs
font-semibold text-primary shrink-0"&gt;
              {(user?.displayName?.[0] || "U").toUpperCase()}
            &lt;/div&gt;
            {!collapsed &amp;&amp; (
              &lt;div className="min-w-0 flex-1"&gt;
                &lt;div className="text-sm truncate"&gt;{user?.displayName}&lt;/div&gt;
                &lt;div className="text-[10px] text-muted-foreground truncate"&gt;{user?.email}&lt;/
div&gt;
              &lt;/div&gt;
            )}
          &lt;/div&gt;
        &lt;/div&gt;
      &lt;/aside&gt;
    &lt;/&gt;
  );
}
```

src/components/jarvis/topbar.tsx

```
"use client";

import { Menu, PanelLeftClose, PanelLeft, Mic, MicOff, Wifi, WifiOff, Command } from "lucide-react";
import { useApp, type View } from "@/store/app-store";
import { Button } from "@/components/ui/button";
import { useEffect, useState } from "react";

const TITLES: Record&lt;View, string&gt; = {
  dashboard: "Dashboard",
  chat: "Chat",
  voice: "Voice",
  tasks: "Tasks",
  notes: "Notes",
  projects: "Projects",
  memory: "Memory",
  settings: "Settings",
};

export function Topbar({ micGranted }: { micGranted: "unknown" | "granted" | "denied" |
"unsupported" }) {
  const { view, toggleSidebar, sidebarCollapsed, setMobileNav } = useApp();
  const [online, setOnline] = useState(true);

  useEffect(() =&gt; {
    const on = () =&gt; setOnline(true);
    const off = () =&gt; setOnline(false);
    window.addEventListener("online", on);
    window.addEventListener("offline", off);
    return () =&gt; {
      window.removeEventListener("online", on);
      window.removeEventListener("offline", off);
    };
  }, []);

  return (
    &lt;header className="sticky top-0 z-30 h-16 flex items-center gap-3 border-b border-border/70
bg-background/70 backdrop-blur-xl px-4"&gt;
      &lt;Button
        variant="ghost"
        size="icon"
        className="lg:hidden"
        onClick={() =&gt; setMobileNav(true)}
        aria-label="Open navigation"
      &gt;
        &lt;Menu className="size-5" /&gt;
      &lt;/Button&gt;
      &lt;Button
        variant="ghost"
        size="icon"
        className="hidden lg:inline-flex"
        onClick={toggleSidebar}
        aria-label="Toggle sidebar"
      &gt;
        {sidebarCollapsed ? &lt;PanelLeft className="size-5" /&gt; : &lt;PanelLeftClose
className="size-5" /&gt;}
      &lt;/Button&gt;
```

```
      &lt;h1 className="text-lg font-semibold tracking-tight"&gt;{TITLES[view]}&lt;/h1&gt;
      &lt;button
        onClick={() =&gt; { window.dispatchEvent(new KeyboardEvent("keydown", { key: "k", metaKey:
true })); }}
        className="hidden md:inline-flex items-center gap-1.5 rounded-md border border-border/60 bg-
background/40 px-2.5 py-1 text-xs text-muted-foreground hover:bg-accent/50 transition"
        title="Open command palette (■K)"
      &gt;
        &lt;Command className="size-3" /&gt;
        &lt;span&gt;Search&lt;/span&gt;
        &lt;kbd className="ml-1 rounded bg-muted/60 px-1 text-[10px]"&gt;■K&lt;/kbd&gt;
      &lt;/button&gt;
      &lt;div className="ml-auto flex items-center gap-2"&gt;
        &lt;StatusPill ok={online} okLabel="Online" badLabel="Offline" iconOn={Wifi} iconOff={
WifiOff} /&gt;
        &lt;StatusPill
          ok={micGranted === "granted"}
          okLabel="Mic ready"
          badLabel={micGranted === "unsupported" ? "No mic" : micGranted === "denied" ? "Mic
blocked" : "Mic idle"}
          iconOn={Mic}
          iconOff={MicOff}
        /&gt;
      &lt;/div&gt;
    &lt;/header&gt;
  );
}

function StatusPill({
  ok,
  okLabel,
  badLabel,
  iconOn: IconOn,
  iconOff: IconOff,
}: {
  ok: boolean;
  okLabel: string;
  badLabel: string;
  iconOn: React.ElementType;
  iconOff: React.ElementType;
}) {
  return (
    &lt;span
      className={
        "inline-flex items-center gap-1.5 rounded-full border px-2.5 py-1 text-[11px] font-medium "
+
        (ok
          ? "border-primary/40 bg-primary/10 text-primary"
          : "border-border bg-muted/40 text-muted-foreground")
      }
    &gt;
      {ok ? &lt;IconOn className="size-3" /&gt; : &lt;IconOff className="size-3" /&gt;}
      &lt;span className="hidden sm:inline"&gt;{ok ? okLabel : badLabel}&lt;/span&gt;
    &lt;/span&gt;
  );
}
```

src/components/jarvis/orb.tsx

```
"use client";

import { motion } from "framer-motion";
import { cn } from "@/lib/utils";
import type { AssistantState } from "@/lib/types";

interface OrbProps {
  state: AssistantState;
  size?: number;
  reducedMotion?: boolean;
}

/**
 * Central JARVIS orb. Purely decorative + state-reactive (no audio/telemetry).
 * Layers:
 *  - outer rotating ring (conic gradient)
 *  - counter-rotating ring
 *  - inner core that pulses when processing/listening/speaking
 */
export function JarvisOrb({ state, size = 220, reducedMotion = false }: OrbProps) {
```

```
  const palette = stateColor(state);
  const idle = state === "idle";
  const dim = reducedMotion;

  return (
    &lt;div
      className="relative grid place-items-center"
      style={{ width: size, height: size }}
      role="img"
      aria-label={`JARVIS ${state}`}
    &gt;
      {/* outer glow */}
      &lt;div
        className="absolute inset-0 rounded-full blur-2xl"
        style={{ background: `radial-gradient(circle, ${palette.glow}, transparent 70%)`, opacity:
idle ? 0.4 : 0.85 }}
      /&gt;
      {/* rotating ring */}
      &lt;div
        className={cn("absolute inset-0 rounded-full jarvis-orb-ring", !dim &amp;&amp; "jarvis-
spin")}
        style={{ opacity: idle ? 0.6 : 0.95 }}
      /&gt;
      {/* counter ring */}
      &lt;div
        className={cn("absolute rounded-full", !dim &amp;&amp; "jarvis-spin-rev")}
        style={{ inset: size * 0.12, background: palette.ring2, opacity: idle ? 0.35 : 0.7,
maskImage: "radial-gradient(transparent 60%, black 62%)", WebkitMaskImage: "radial-gradient(
transparent 60%, black 62%)" }}
      /&gt;
      {/* core */}
      &lt;motion.div
        className="absolute rounded-full"
        animate={dim ? {} : { scale: idle ? 1 : [1, 1.06, 1], opacity: idle ? 0.85 : [0.85, 1, 0.
85] }}
        transition={dim ? {} : { duration: 2.6, repeat: Infinity, ease: "easeInOut" }}
        style={{ inset: size * 0.22, background: palette.core, boxShadow: `0 0 60px -8px ${palette.
glow}` }}
      /&gt;
      {/* innermost node */}
      &lt;div
        className={cn("absolute rounded-full", !dim &amp;&amp; !idle &amp;&amp; "jarvis-breathe")}
        style={{ inset: size * 0.38, background: palette.node }}
      /&gt;
    &lt;/div&gt;
  );
}

function stateColor(s: AssistantState) {
  switch (s) {
    case "listening":
      return {
        core: "radial-gradient(circle at 35% 30%, oklch(0.85 0.16 190), oklch(0.45 0.14 200))",
        node: "oklch(0.95 0.05 190)",
        ring2: "conic-gradient(from 90deg, transparent, oklch(0.85 0.16 190 / 0.8), transparent)",
        glow: "oklch(0.7 0.16 190 / 0.6)",
      };
    case "processing":
      return {
        core: "radial-gradient(circle at 35% 30%, oklch(0.85 0.14 230), oklch(0.45 0.16 250))",
        node: "oklch(0.9 0.06 230)",
        ring2: "conic-gradient(from 90deg, transparent, oklch(0.8 0.16 235 / 0.8), transparent)",
        glow: "oklch(0.7 0.16 240 / 0.6)",
      };
    case "speaking":
      return {
        core: "radial-gradient(circle at 35% 30%, oklch(0.85 0.16 175), oklch(0.5 0.16 165))",
        node: "oklch(0.95 0.06 175)",
        ring2: "conic-gradient(from 90deg, transparent, oklch(0.8 0.16 170 / 0.8), transparent)",
        glow: "oklch(0.72 0.16 170 / 0.6)",
      };
    case "error":
      return {
        core: "radial-gradient(circle at 35% 30%, oklch(0.78 0.2 25), oklch(0.5 0.22 25))",
        node: "oklch(0.92 0.06 25)",
        ring2: "conic-gradient(from 90deg, transparent, oklch(0.78 0.2 25 / 0.8), transparent)",
        glow: "oklch(0.7 0.2 25 / 0.6)",
      };
    default:
      return {
        core: "radial-gradient(circle at 35% 30%, oklch(0.78 0.16 210), oklch(0.45 0.14 220))",
```

```
        node: "oklch(0.9 0.08 210)",
        ring2: "conic-gradient(from 90deg, transparent, oklch(0.78 0.16 210 / 0.8), transparent)",
        glow: "oklch(0.7 0.16 210 / 0.55)",
      };
  }
}
```

src/components/jarvis/markdown.tsx

```
"use client";

import ReactMarkdown from "react-markdown";
import { Prism as SyntaxHighlighter } from "react-syntax-highlighter";
import { oneDark } from "react-syntax-highlighter/dist/esm/styles/prism";
import { useState } from "react";
import { Check, Copy } from "lucide-react";
import { toast } from "sonner";
import { cn } from "@/lib/utils";

export function Markdown({ content, className }: { content: string; className?: string }) {
  return (
    &lt;div className={cn("jarvis-prose", className)}&gt;
      &lt;ReactMarkdown
        components={{
          code(props) {
            const { children, className, node, ...rest } = props as any;
            void node;
            const match = /language-(\w+)/.exec(className || "");
            const inline = !className;
            if (inline || !match) {
              return &lt;code className={className} {...rest}&gt;{children}&lt;/code&gt;;
            }
            const code = String(children).replace(/\n$/, "");
            return &lt;CodeBlock language={match[1]} code={code} /&gt;;
          },
          a(props) {
            const { children, href } = props as any;
            return &lt;a href={href} target="_blank" rel="noopener noreferrer"&gt;{children}&lt;/
a&gt;;
          },
        }}
      &gt;
        {content}
      &lt;/ReactMarkdown&gt;
    &lt;/div&gt;
  );
}

function CodeBlock({ language, code }: { language: string; code: string }) {
  const [copied, setCopied] = useState(false);
  return (
    &lt;div className="relative group"&gt;
      &lt;button
        onClick={() =&gt; {
          navigator.clipboard?.writeText(code);
          setCopied(true);
          setTimeout(() =&gt; setCopied(false), 1200);
          toast.success("Copied");
        }}
        className="absolute right-2 top-2 rounded-md border border-border/60 bg-background/60 p-1
opacity-0 group-hover:opacity-100 transition"
        aria-label="Copy code"
      &gt;
        {copied ? &lt;Check className="size-3 text-emerald-400" /&gt; : &lt;Copy className="size-3"
/&gt;}
      &lt;/button&gt;
      &lt;SyntaxHighlighter style={oneDark} language={language} PreTag="div" className="!bg-
background/60 !text-xs"&gt;
        {code}
      &lt;/SyntaxHighlighter&gt;
    &lt;/div&gt;
  );
}
```

src/components/jarvis/confirm-dialog.tsx

```
"use client";
```

```
import { AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent,
AlertDialogDescription, AlertDialogFooter, AlertDialogHeader, AlertDialogTitle } from "@/components/
ui/alert-dialog";
import { Button } from "@/components/ui/button";

export interface ConfirmRequest {
  title: string;
  description?: string;
  confirmText?: string;
  cancelText?: string;
  destructive?: boolean;
  onConfirm: () =&gt; void;
}

export function ConfirmDialog({ open, request, onClose }: { open: boolean; request: ConfirmRequest
| null; onClose: () =&gt; void }) {
  return (
    &lt;AlertDialog open={open} onOpenChange={(o) =&gt; { if (!o) onClose(); }}&gt;
      &lt;AlertDialogContent className="jarvis-panel"&gt;
        &lt;AlertDialogHeader&gt;
          &lt;AlertDialogTitle&gt;{request?.title ?? "Confirm"}&lt;/AlertDialogTitle&gt;
          {request?.description &amp;&amp; &lt;AlertDialogDescription&gt;{request.description}&lt;/
AlertDialogDescription&gt;}
        &lt;/AlertDialogHeader&gt;
        &lt;AlertDialogFooter&gt;
          &lt;AlertDialogCancel asChild&gt;
            &lt;Button variant="ghost"&gt;{request?.cancelText ?? "Cancel"}&lt;/Button&gt;
          &lt;/AlertDialogCancel&gt;
          &lt;AlertDialogAction asChild&gt;
            &lt;Button variant={request?.destructive ? "destructive" : "default"} onClick={() =&gt;
{ request?.onConfirm(); onClose(); }}&gt;
              {request?.confirmText ?? "Confirm"}
            &lt;/Button&gt;
          &lt;/AlertDialogAction&gt;
        &lt;/AlertDialogFooter&gt;
      &lt;/AlertDialogContent&gt;
    &lt;/AlertDialog&gt;
  );
}
```

src/components/jarvis/tool-call-card.tsx

```
"use client";

import { useEffect, useState } from "react";
import { useTools, runTool } from "@/hooks/use-tools";
import { Card } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Badge } from "@/components/ui/badge";
import { CheckCircle2, Loader2, ShieldCheck, XCircle, Download, ExternalLink, Save } from "lucide-
react";
import { toast } from "sonner";
import type { ToolField } from "@/lib/types";
import { useImagesStore } from "@/store/images-store";

export interface ToolCallCardProps {
  tool: string;
  input: Record&lt;string, unknown&gt;;
  onResolved?: (result: { ok: boolean; message: string }) =&gt; void;
}

interface WebSearchHit { url: string; name: string; snippet: string; host_name?: string; date?:
string }

export function ToolCallCard({ tool, input, onResolved }: ToolCallCardProps) {
  const { data } = useTools();
  const def = data?.tools.find((t) =&gt; t.name === tool);
  const [values, setValues] = useState&lt;Record&lt;string, string&gt;&gt;({});
  const [state, setState] = useState&lt;"idle" | "running" | "done" | "error" | "cancelled"&gt;(
"idle");
  const [resultMsg, setResultMsg] = useState("");
  const [resultData, setResultData] = useState&lt;Record&lt;string, unknown&gt; | null&gt;(null);
  const addImage = useImagesStore((s) =&gt; s.addImage);

  useEffect(() =&gt; {
    const init: Record&lt;string, string&gt; = {};
    for (const f of def?.schema ?? []) {
```

```
      const v = (input as Record&lt;string, unknown&gt;)[f.name];
      if (v !== undefined &amp;&amp; v !== null) init[f.name] = String(v);
    }
    // setValues is a sync-from-props init; the linter's set-state-in-effect
    // rule does not flag this pattern here.
    setValues(init);
  }, [def, input]);

  async function confirm() {
    setState("running");
    const res = await runTool(tool, values);
    if (res.ok) {
      setState("done");
      setResultMsg(res.result?.message ?? "Done.");
      setResultData((res.result?.data as Record&lt;string, unknown&gt;) ?? null);
      toast.success(res.result?.message ?? "Action completed.");
      // If image generation, add to the gallery.
      if (tool === "generate_image" &amp;&amp; res.result?.data) {
        const imgs = (res.result.data as { images?: string[] }).images ?? [];
        const prompt = (res.result.data as { prompt?: string }).prompt ?? "";
        for (const src of imgs) addImage({ src, prompt, size: values.size || "1024x1024" });
      }
    } else {
      setState("error");
      setResultMsg(res.result?.message ?? res.error ?? "Failed.");
      toast.error(res.result?.message ?? "Action failed.");
    }
    onResolved?.({ ok: res.ok, message: res.result?.message ?? "" });
  }

  if (!def) {
    return (
      &lt;Card className="p-4 border-dashed"&gt;
        &lt;div className="flex items-center gap-2 text-sm text-muted-foreground"&gt;
          &lt;ShieldCheck className="size-4" /&gt; Unknown tool: &lt;code className="text-
foreground"&gt;{tool}&lt;/code&gt;
        &lt;/div&gt;
      &lt;/Card&gt;
    );
  }

  const terminal = state === "done" || state === "error" || state === "cancelled";
  const images = (resultData?.images as string[] | undefined) ?? [];
  const searchResults = (resultData?.results as WebSearchHit[] | undefined) ?? [];

  return (
    &lt;Card className="p-4 jarvis-panel"&gt;
      &lt;div className="flex items-center justify-between gap-2 mb-3"&gt;
        &lt;div className="flex items-center gap-2"&gt;
          &lt;ShieldCheck className="size-4 text-primary" /&gt;
          &lt;span className="text-sm font-medium"&gt;{def.label}&lt;/span&gt;
          {def.requiresConfirmation &amp;&amp; &lt;Badge variant="secondary" className="text-[
10px]"&gt;Approval required&lt;/Badge&gt;}
        &lt;/div&gt;
        {state === "done" &amp;&amp; &lt;CheckCircle2 className="size-4 text-emerald-400" /&gt;}
        {state === "error" &amp;&amp; &lt;XCircle className="size-4 text-destructive" /&gt;}
        {state === "running" &amp;&amp; &lt;Loader2 className="size-4 animate-spin text-primary" /
&gt;}
      &lt;/div&gt;
      &lt;p className="text-xs text-muted-foreground mb-3"&gt;{def.description}&lt;/p&gt;

      {!terminal &amp;&amp; (
        &lt;div className="grid gap-3"&gt;
          {def.schema.map((f: ToolField) =&gt; (
            &lt;div key={f.name} className="grid gap-1"&gt;
              &lt;Label htmlFor={`tf-${f.name}`} className="text-xs"&gt;
                {f.label}{f.required &amp;&amp; &lt;span className="text-destructive"&gt; *&lt;/
span&gt;}
              &lt;/Label&gt;
              {f.type === "string" &amp;&amp; f.options ? (
                &lt;select
                  id={`tf-${f.name}`}
                  className="h-9 rounded-md border border-input bg-background px-3 text-sm"
                  value={values[f.name] ?? f.options[0]}
                  onChange={(e) =&gt; setValues((v) =&gt; ({ ...v, [f.name]: e.target.value }))}
                  disabled={state === "running"}
                &gt;
                  {f.options.map((o) =&gt; &lt;option key={o} value={o}&gt;{o}&lt;/option&gt;)}
                &lt;/select&gt;
              ) : (
                &lt;Input
```

```
                  id={`tf-${f.name}`}
                  type={f.type === "date" ? "date" : f.type === "number" ? "number" : "text"}
                  value={values[f.name] ?? ""}
                  placeholder={f.placeholder}
                  onChange={(e) =&gt; setValues((v) =&gt; ({ ...v, [f.name]: e.target.value }))}
                  disabled={state === "running"}
                /&gt;
              )}
            &lt;/div&gt;
          ))}
        &lt;/div&gt;
      )}

      {terminal &amp;&amp; (
        &lt;div className="grid gap-3"&gt;
          &lt;p className="text-xs text-muted-foreground"&gt;{resultMsg}&lt;/p&gt;
          {images.length &gt; 0 &amp;&amp; (
            &lt;div className="grid gap-2"&gt;
              {images.map((src, i) =&gt; (
                &lt;div key={i} className="relative group"&gt;
                  &lt;img src={src} alt={values.prompt || "Generated"} className="rounded-lg w-
full" /&gt;
                  &lt;div className="absolute top-2 right-2 flex gap-1 opacity-0 group-hover:
opacity-100 transition"&gt;
                    &lt;a href={src} download={`jarvis-${Date.now()}.png`} className="rounded-md
border border-border/60 bg-background/80 p-1.5"&gt;
                      &lt;Download className="size-3.5" /&gt;
                    &lt;/a&gt;
                  &lt;/div&gt;
                &lt;/div&gt;
              ))}
            &lt;/div&gt;
          )}
          {searchResults.length &gt; 0 &amp;&amp; (
            &lt;ul className="grid gap-2 text-xs"&gt;
              {searchResults.map((r, i) =&gt; (
                &lt;li key={i} className="rounded-md border border-border/60 p-2 bg-background/
40"&gt;
                  &lt;a href={r.url} target="_blank" rel="noopener noreferrer" className="font-
medium text-primary hover:underline flex items-center gap-1"&gt;
                    {r.name} &lt;ExternalLink className="size-3" /&gt;
                  &lt;/a&gt;
                  {r.host_name &amp;&amp; &lt;span className="text-[10px] text-muted-foreground ml-
1"&gt;{r.host_name}&lt;/span&gt;}
                  {r.snippet &amp;&amp; &lt;p className="text-muted-foreground mt-1 line-clamp-
2"&gt;{r.snippet}&lt;/p&gt;}
                &lt;/li&gt;
              ))}
            &lt;/ul&gt;
          )}
        &lt;/div&gt;
      )}

      {!terminal &amp;&amp; (
        &lt;div className="flex justify-end gap-2 mt-4"&gt;
          &lt;Button size="sm" variant="ghost" onClick={() =&gt; { setState("cancelled");
setResultMsg("Cancelled."); onResolved?.({ ok: false, message: "Cancelled" }); }}&gt;
            Cancel
          &lt;/Button&gt;
          &lt;Button size="sm" onClick={confirm} disabled={state === "running"}&gt;
            {state === "running" ? "Running…" : def.requiresConfirmation ? "Approve &amp; run" :
"Run"}
          &lt;/Button&gt;
        &lt;/div&gt;
      )}
    &lt;/Card&gt;
  );
}
```

src/components/jarvis/task-charts.tsx

```
"use client";

import { PieChart, Pie, Cell, ResponsiveContainer, Tooltip, BarChart, Bar, XAxis, YAxis,
CartesianGrid, LineChart, Line } from "recharts";
import { Card } from "@/components/ui/card";
import type { Task } from "@/lib/types";
import { CircleDashed, CircleDot, CircleCheckBig, TrendingUp } from "lucide-react";
const STATUS_COLORS = { todo: "#7c8db5", in_progress: "#f5a623", done: "#3ecf8e" } as const;
```

```
const PRIORITY_COLORS = { low: "#3ecf8e", medium: "#f5a623", high: "#ef4444" } as const;

export function TaskCharts({ tasks }: { tasks: Task[] }) {
  const statusData = (["todo", "in_progress", "done"] as const).map((k) =&gt; ({
    name: k === "todo" ? "To do" : k === "in_progress" ? "In progress" : "Done",
    value: tasks.filter((t) =&gt; t.status === k).length,
    key: k,
  }));
  const priorityData = (["low", "medium", "high"] as const).map((k) =&gt; ({
    name: k.charAt(0).toUpperCase() + k.slice(1),
    value: tasks.filter((t) =&gt; t.priority === k &amp;&amp; t.status !== "done").length,
    fill: PRIORITY_COLORS[k],
  }));
  // 7-day completion trend.
  const today = new Date(); today.setHours(0, 0, 0, 0);
  const trend = Array.from({ length: 7 }).map((_, i) =&gt; {
    const d = new Date(today); d.setDate(d.getDate() - (6 - i));
    const next = new Date(d); next.setDate(next.getDate() + 1);
    const done = tasks.filter((t) =&gt; t.completedAt &amp;&amp; new Date(t.completedAt) &gt;= d
&amp;&amp; new Date(t.completedAt) &lt; next).length;
    const created = tasks.filter((t) =&gt; new Date(t.createdAt) &gt;= d &amp;&amp; new Date(t.
createdAt) &lt; next).length;
    return { name: d.toLocaleDateString(undefined, { weekday: "short" }), created, done };
  });

  const total = tasks.length;
  const doneCount = tasks.filter((t) =&gt; t.status === "done").length;
  const completionRate = total ? Math.round((doneCount / total) * 100) : 0;

  if (total === 0) {
    return (
      &lt;Card className="jarvis-panel p-5"&gt;
        &lt;div className="flex items-center gap-2 mb-2"&gt;&lt;TrendingUp className="size-4 text-
primary" /&gt;&lt;h3 className="font-semibold text-sm"&gt;Productivity&lt;/h3&gt;&lt;/div&gt;
        &lt;p className="text-xs text-muted-foreground"&gt;No tasks yet — create one to see your
productivity charts.&lt;/p&gt;
      &lt;/Card&gt;
    );
  }

  return (
    &lt;div className="grid lg:grid-cols-3 gap-4"&gt;
      &lt;Card className="jarvis-panel p-4"&gt;
        &lt;div className="flex items-center justify-between mb-2"&gt;
          &lt;h3 className="font-semibold text-sm flex items-center gap-2"&gt;&lt;CircleDashed
className="size-4 text-primary" /&gt; Status&lt;/h3&gt;
          &lt;span className="text-xs text-muted-foreground"&gt;{completionRate}% done&lt;/span&gt;
        &lt;/div&gt;
        &lt;div className="h-40"&gt;
          &lt;ResponsiveContainer width="100%" height="100%"&gt;
            &lt;PieChart&gt;
              &lt;Pie data={statusData} dataKey="value" nameKey="name" innerRadius={42}
outerRadius={64} paddingAngle={3}&gt;
                {statusData.map((d) =&gt; &lt;Cell key={d.key} fill={STATUS_COLORS[d.key]}
stroke="transparent" /&gt;)}
              &lt;/Pie&gt;
              &lt;Tooltip contentStyle={{ background: "rgba(20,20,30,0.95)", border: "1px solid
rgba(255,255,255,0.1)", borderRadius: 8, fontSize: 12 }} /&gt;
            &lt;/PieChart&gt;
          &lt;/ResponsiveContainer&gt;
        &lt;/div&gt;
        &lt;div className="flex flex-wrap gap-3 justify-center text-[11px]"&gt;
          {statusData.map((d) =&gt; (
            &lt;span key={d.key} className="flex items-center gap-1"&gt;&lt;span className="size-2
rounded-full" style={{ background: STATUS_COLORS[d.key] }} /&gt;{d.name}: {d.value}&lt;/span&gt;
          ))}
        &lt;/div&gt;
      &lt;/Card&gt;

      &lt;Card className="jarvis-panel p-4"&gt;
        &lt;div className="flex items-center justify-between mb-2"&gt;
          &lt;h3 className="font-semibold text-sm flex items-center gap-2"&gt;&lt;CircleDot
className="size-4 text-primary" /&gt; Open by priority&lt;/h3&gt;
        &lt;/div&gt;
        &lt;div className="h-40"&gt;
          &lt;ResponsiveContainer width="100%" height="100%"&gt;
            &lt;BarChart data={priorityData} margin={{ top: 4, right: 8, left: -18, bottom: 0 }}&gt;
              &lt;CartesianGrid strokeDasharray="3 3" stroke="rgba(255,255,255,0.06)" vertical={
false} /&gt;
              &lt;XAxis dataKey="name" tick={{ fill: "#94a3b8", fontSize: 11 }} axisLine={false}
tickLine={false} /&gt;
```

```
              &lt;YAxis allowDecimals={false} tick={{ fill: "#94a3b8", fontSize: 11 }} axisLine={
false} tickLine={false} /&gt;
              &lt;Tooltip cursor={{ fill: "rgba(255,255,255,0.04)" }} contentStyle={{ background:
"rgba(20,20,30,0.95)", border: "1px solid rgba(255,255,255,0.1)", borderRadius: 8, fontSize: 12 }} /
&gt;
              &lt;Bar dataKey="value" radius={[4, 4, 0, 0]} /&gt;
            &lt;/BarChart&gt;
          &lt;/ResponsiveContainer&gt;
        &lt;/div&gt;
      &lt;/Card&gt;

      &lt;Card className="jarvis-panel p-4"&gt;
        &lt;div className="flex items-center justify-between mb-2"&gt;
          &lt;h3 className="font-semibold text-sm flex items-center gap-2"&gt;&lt;CircleCheckBig
className="size-4 text-primary" /&gt; 7-day trend&lt;/h3&gt;
        &lt;/div&gt;
        &lt;div className="h-40"&gt;
          &lt;ResponsiveContainer width="100%" height="100%"&gt;
            &lt;LineChart data={trend} margin={{ top: 4, right: 8, left: -18, bottom: 0 }}&gt;
              &lt;CartesianGrid strokeDasharray="3 3" stroke="rgba(255,255,255,0.06)" vertical={
false} /&gt;
              &lt;XAxis dataKey="name" tick={{ fill: "#94a3b8", fontSize: 11 }} axisLine={false}
tickLine={false} /&gt;
              &lt;YAxis allowDecimals={false} tick={{ fill: "#94a3b8", fontSize: 11 }} axisLine={
false} tickLine={false} /&gt;
              &lt;Tooltip contentStyle={{ background: "rgba(20,20,30,0.95)", border: "1px solid
rgba(255,255,255,0.1)", borderRadius: 8, fontSize: 12 }} /&gt;
              &lt;Line type="monotone" dataKey="created" name="Created" stroke="#7c8db5"
strokeWidth={2} dot={{ r: 2 }} /&gt;
              &lt;Line type="monotone" dataKey="done" name="Done" stroke="#3ecf8e" strokeWidth={2}
dot={{ r: 2 }} /&gt;
            &lt;/LineChart&gt;
          &lt;/ResponsiveContainer&gt;
        &lt;/div&gt;
        &lt;div className="flex gap-3 justify-center text-[11px] text-muted-foreground"&gt;
          &lt;span className="flex items-center gap-1"&gt;&lt;span className="size-2 rounded-full
bg-[#7c8db5]" /&gt;Created&lt;/span&gt;
          &lt;span className="flex items-center gap-1"&gt;&lt;span className="size-2 rounded-full
bg-[#3ecf8e]" /&gt;Done&lt;/span&gt;
        &lt;/div&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
}
```

src/components/jarvis/command-palette.tsx

```
"use client";

import { useEffect, useState } from "react";
import { Command, CommandEmpty, CommandGroup, CommandInput, CommandItem, CommandList,
CommandSeparator } from "@/components/ui/command";
import { Dialog, DialogContent } from "@/components/ui/dialog";
import { useApp, type View } from "@/store/app-store";
import {
  LayoutDashboard, MessagesSquare, Mic, ListChecks, NotebookPen, FolderKanban, Wand2, Brain,
Settings,
  Plus, Search, Sparkles, Home,
} from "lucide-react";
import { useQuery } from "@tanstack/react-query";
import { useTasks, useNotes, useProjects } from "@/hooks/use-data";

interface PaletteItem {
  id: string;
  label: string;
  hint?: string;
  icon: React.ElementType;
  group: string;
  run: () =&gt; void;
}

const VIEW_ITEMS: { id: View; label: string; icon: React.ElementType }[] = [
  { id: "dashboard", label: "Dashboard", icon: LayoutDashboard },
  { id: "chat", label: "Chat with JARVIS", icon: MessagesSquare },
  { id: "voice", label: "Voice assistant", icon: Mic },
  { id: "tasks", label: "Tasks", icon: ListChecks },
  { id: "notes", label: "Notes", icon: NotebookPen },
  { id: "projects", label: "Projects", icon: FolderKanban },
  { id: "studio", label: "Image Studio", icon: Wand2 },
```

```
  { id: "memory", label: "Memory", icon: Brain },
  { id: "settings", label: "Settings", icon: Settings },
];

export function CommandPalette() {
  const [open, setOpen] = useState(false);
  const { setView, launchQuick, setActiveConversation, setActiveProject } = useApp();

  // Global Cmd/Ctrl+K to open, plus view shortcuts Cmd+1..9, N for new task.
  useEffect(() =&gt; {
    function onKey(e: KeyboardEvent) {
      const mod = e.metaKey || e.ctrlKey;
      if (mod &amp;&amp; e.key.toLowerCase() === "k") {
        e.preventDefault();
        setOpen((o) =&gt; !o);
        return;
      }
      if (mod &amp;&amp; /^[1-9]$/.test(e.key)) {
        const idx = Number(e.key) - 1;
        const item = VIEW_ITEMS[idx];
        if (item) { e.preventDefault(); setView(item.id); }
        return;
      }
      // Plain "n" (no modifier, not typing) → new task.
      const el = document.activeElement as HTMLElement | null;
      const typing = el &amp;&amp; (el.tagName === "INPUT" || el.tagName === "TEXTAREA" || el.
isContentEditable);
      if (!typing &amp;&amp; !mod &amp;&amp; !e.altKey &amp;&amp; e.key.toLowerCase() === "n") {
        e.preventDefault();
        launchQuick("tasks", "new");
      }
    }
    window.addEventListener("keydown", onKey);
    return () =&gt; window.removeEventListener("keydown", onKey);
  }, [setView, launchQuick]);

  const tasks = useTasks("active", "");
  const notes = useNotes("");
  const projects = useProjects("");

  const items: PaletteItem[] = [
    ...VIEW_ITEMS.map((v) =&gt; ({
      id: `view-${v.id}`,
      label: v.label,
      hint: "Go to",
      icon: v.icon,
      group: "Navigation",
      run: () =&gt; setView(v.id),
    })),
    { id: "new-task", label: "Create a new task", icon: Plus, group: "Quick actions", run: () =&gt;
launchQuick("tasks", "new") },
    { id: "new-note", label: "Create a new note", icon: Plus, group: "Quick actions", run: () =&gt;
launchQuick("notes", "new") },
    { id: "new-project", label: "Create a new project", icon: Plus, group: "Quick actions", run: ()
=&gt; launchQuick("projects", "new") },
    { id: "ask-jarvis", label: "Ask JARVIS…", icon: Sparkles, group: "Quick actions", run: () =&gt;
setView("chat") },
    ...((tasks.data?.tasks ?? []).slice(0, 6).map((t) =&gt; ({
      id: `task-${t.id}`, label: t.title, hint: "Open task", icon: ListChecks, group: "Tasks",
      run: () =&gt; setView("tasks"),
    }))),
    ...((notes.data?.notes ?? []).slice(0, 6).map((n) =&gt; ({
      id: `note-${n.id}`, label: n.title, hint: "Open note", icon: NotebookPen, group: "Notes",
      run: () =&gt; setView("notes"),
    }))),
    ...((projects.data?.projects ?? []).slice(0, 6).map((p) =&gt; ({
      id: `proj-${p.id}`, label: p.name, hint: `${p.progress}%`, icon: FolderKanban, group:
"Projects",
      run: () =&gt; { setActiveProject(p.id); setView("projects"); },
    }))),
  ];

  return (
    &lt;Dialog open={open} onOpenChange={setOpen}&gt;
      &lt;DialogContent className="jarvis-panel p-0 overflow-hidden max-w-xl"&gt;
        &lt;Command className="bg-transparent"&gt;
          &lt;CommandInput placeholder="Type a command or search…" autoFocus /&gt;
          &lt;CommandList className="max-h-[60vh] jarvis-scrollbar"&gt;
            &lt;CommandEmpty&gt;No results.&lt;/CommandEmpty&gt;
            &lt;CommandGroup heading="Navigation"&gt;
              {items.filter((i) =&gt; i.group === "Navigation").map((i) =&gt; (
```

```
                &lt;CommandItem key={i.id} value={`${i.label} ${i.group}`} onSelect={() =&gt; { i.
run(); setOpen(false); }}&gt;
                  &lt;i.icon className="size-4 shrink-0 text-muted-foreground" /&gt;
                  &lt;span className="flex-1"&gt;{i.label}&lt;/span&gt;
                  &lt;span className="text-[10px] text-muted-foreground"&gt;{i.hint}&lt;/span&gt;
                &lt;/CommandItem&gt;
              ))}
            &lt;/CommandGroup&gt;
            &lt;CommandSeparator /&gt;
            &lt;CommandGroup heading="Quick actions"&gt;
              {items.filter((i) =&gt; i.group === "Quick actions").map((i) =&gt; (
                &lt;CommandItem key={i.id} value={i.label} onSelect={() =&gt; { i.run(); setOpen(
false); }}&gt;
                  &lt;i.icon className="size-4 shrink-0 text-primary" /&gt;
                  &lt;span className="flex-1"&gt;{i.label}&lt;/span&gt;
                &lt;/CommandItem&gt;
              ))}
            &lt;/CommandGroup&gt;
            {tasks.data?.tasks &amp;&amp; tasks.data.tasks.length &gt; 0 &amp;&amp; (
              &lt;CommandGroup heading="Tasks"&gt;
                {items.filter((i) =&gt; i.group === "Tasks").map((i) =&gt; (
                  &lt;CommandItem key={i.id} value={i.label} onSelect={() =&gt; { i.run(); setOpen(
false); }}&gt;
                    &lt;i.icon className="size-4 shrink-0 text-muted-foreground" /&gt;
                    &lt;span className="flex-1 truncate"&gt;{i.label}&lt;/span&gt;
                  &lt;/CommandItem&gt;
                ))}
              &lt;/CommandGroup&gt;
            )}
            {notes.data?.notes &amp;&amp; notes.data.notes.length &gt; 0 &amp;&amp; (
              &lt;CommandGroup heading="Notes"&gt;
                {items.filter((i) =&gt; i.group === "Notes").map((i) =&gt; (
                  &lt;CommandItem key={i.id} value={i.label} onSelect={() =&gt; { i.run(); setOpen(
false); }}&gt;
                    &lt;i.icon className="size-4 shrink-0 text-muted-foreground" /&gt;
                    &lt;span className="flex-1 truncate"&gt;{i.label}&lt;/span&gt;
                  &lt;/CommandItem&gt;
                ))}
              &lt;/CommandGroup&gt;
            )}
            {projects.data?.projects &amp;&amp; projects.data.projects.length &gt; 0 &amp;&amp; (
              &lt;CommandGroup heading="Projects"&gt;
                {items.filter((i) =&gt; i.group === "Projects").map((i) =&gt; (
                  &lt;CommandItem key={i.id} value={i.label} onSelect={() =&gt; { i.run(); setOpen(
false); }}&gt;
                    &lt;i.icon className="size-4 shrink-0 text-muted-foreground" /&gt;
                    &lt;span className="flex-1 truncate"&gt;{i.label}&lt;/span&gt;
                    &lt;span className="text-[10px] text-muted-foreground"&gt;{i.hint}&lt;/span&gt;
                  &lt;/CommandItem&gt;
                ))}
              &lt;/CommandGroup&gt;
            )}
          &lt;/CommandList&gt;
          &lt;div className="border-t border-border/60 px-3 py-2 text-[10px] text-muted-foreground
flex items-center justify-between"&gt;
            &lt;span className="flex items-center gap-1"&gt;&lt;Home className="size-3" /&gt; ■K to
toggle · ■1-9 for views · N for new task&lt;/span&gt;
            &lt;Search className="size-3" /&gt;
          &lt;/div&gt;
        &lt;/Command&gt;
      &lt;/DialogContent&gt;
    &lt;/Dialog&gt;
  );
}
```

src/components/jarvis/supabase-setup-banner.tsx

```
"use client";

import { useState } from "react";
import { useQuery } from "@tanstack/react-query";
import { Database, Copy, Check, ExternalLink, Loader2 } from "lucide-react";
import { Button } from "@/components/ui/button";
import { useSupabaseSetup } from "@/hooks/use-supabase-setup";
import { toast } from "sonner";

// Shown at the top of the app when Supabase is configured but the schema
// hasn't been applied yet. Guides the user to run the SQL once.
export function SupabaseSetupBanner() {
```

```
  const { data } = useSupabaseSetup();
  const [open, setOpen] = useState(false);
  const sql = useQuery&lt;{ sql: string; editorUrl: string }&gt;({
    queryKey: ["supabase-sql"],
    queryFn: async () =&gt; {
      const r = await fetch("/api/supabase/schema", { cache: "no-store" });
      const j = await r.json();
      if (!j.ok) throw new Error(j.error);
      return j;
    },
    enabled: false,
  });

  if (!data?.ok || !data.configured || data.ready) return null;

  return (
    &lt;div className="border-b border-amber-500/40 bg-amber-500/10 px-4 py-2.5"&gt;
      &lt;div className="max-w-6xl mx-auto flex flex-wrap items-center gap-3 text-sm"&gt;
        &lt;Database className="size-4 text-amber-400 shrink-0" /&gt;
        &lt;div className="flex-1 min-w-0"&gt;
          &lt;span className="font-medium text-amber-200"&gt;Supabase connected — schema not
applied yet.&lt;/span&gt;{" "}
          &lt;span className="text-amber-100/80 hidden sm:inline"&gt;
            The app is fully functional now (local store). Run the schema SQL once to store your
data in Supabase.
          &lt;/span&gt;
        &lt;/div&gt;
        &lt;Button size="sm" variant="outline" className="border-amber-500/40 text-amber-100 hover:
bg-amber-500/20" onClick={() =&gt; setOpen((o) =&gt; !o)}&gt;
          {open ? "Hide" : "Show setup"}
        &lt;/Button&gt;
      &lt;/div&gt;
      {open &amp;&amp; (
        &lt;div className="max-w-6xl mx-auto mt-3 grid gap-3"&gt;
          &lt;div className="flex flex-wrap gap-2"&gt;
            &lt;Button
              size="sm"
              onClick={async () =&gt; {
                const res = await sql.refetch();
                if (res.data) {
                  await navigator.clipboard?.writeText(res.data.sql);
                  toast.success("Schema SQL copied — paste it into the Supabase SQL editor and Run.
");
                }
              }}
            &gt;
              &lt;Copy className="size-4" /&gt; Copy schema SQL
            &lt;/Button&gt;
            &lt;a
              href={`https://supabase.com/dashboard/project/${process.env.NEXT_PUBLIC_SUPABASE_URL?.
replace("https://", "").replace(".supabase.co", "")}/sql/new`}
              target="_blank"
              rel="noopener noreferrer"
            &gt;
              &lt;Button size="sm" variant="outline"&gt;
                &lt;ExternalLink className="size-4" /&gt; Open SQL editor
              &lt;/Button&gt;
            &lt;/a&gt;
            &lt;span className="text-xs text-amber-100/70 self-center ml-2"&gt;
              After running the SQL, this banner disappears automatically.
            &lt;/span&gt;
          &lt;/div&gt;
          &lt;div className="rounded-lg border border-amber-500/30 bg-background/60 p-3 max-h-64
overflow-auto jarvis-scrollbar text-xs font-mono whitespace-pre"&gt;
            {sql.isFetching ? &lt;span className="flex items-center gap-2"&gt;&lt;Loader2
className="size-3 animate-spin" /&gt; Loading SQL…&lt;/span&gt; : (sql.data?.sql ?? "Click “Copy
schema SQL” to load.")}
          &lt;/div&gt;
        &lt;/div&gt;
      )}
    &lt;/div&gt;
  );
}

// Tiny inline indicator used in the footer to show live DB readiness.
export function DbStatusDot() {
  const { data, isLoading } = useSupabaseSetup();
  const ready = data?.ok &amp;&amp; data.configured &amp;&amp; data.ready;
  const partial = data?.ok &amp;&amp; data.configured &amp;&amp; !data.ready;
  const tone = ready ? "bg-emerald-400" : partial ? "bg-amber-400 animate-pulse" : "bg-muted-
foreground/40";
```

```
  const label = ready ? "Supabase ready" : partial ? "Local mode · Supabase pending" : isLoading ?
"Checking DB…" : "DB idle";
  return (
    &lt;span className="inline-flex items-center gap-1.5" title={label}&gt;
      &lt;span className={"size-1.5 rounded-full " + tone} /&gt;
      &lt;span className="hidden sm:inline"&gt;{label}&lt;/span&gt;
    &lt;/span&gt;
  );
}
```

Feature Views

src/components/jarvis/views/dashboard.tsx

```
"use client";

import { useEffect, useState } from "react";
import { JarvisOrb } from "@/components/jarvis/orb";
import { useApp } from "@/store/app-store";
import { useAuth } from "@/components/providers/auth-provider";
import { Card } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Skeleton } from "@/components/ui/skeleton";
import { ListChecks, NotebookPen, FolderKanban, Mic, Send, Sparkles, CalendarClock, Circle,
CheckCircle2 } from "lucide-react";
import { toast } from "sonner";
import { TaskCharts } from "@/components/jarvis/task-charts";
import { useTasks } from "@/hooks/use-data";
import type { AssistantState } from "@/lib/types";

interface Briefing {
  greeting: string;
  narrative: string | null;
  narrativeError: string | null;
  counts: { totalOpen: number; totalDone: number; overdue: number; today: number; upcoming: number
};
  tasksToday: { id: string; title: string; priority: string; dueDate: string | null }[];
  upcoming: { id: string; title: string; priority: string; dueDate: string | null }[];
  overdue: { id: string; title: string; priority: string; dueDate: string | null }[];
  projects: { id: string; name: string; progress: number }[];
}

export function DashboardView() {
  const { user } = useAuth();
  const { setView, launchQuick, setActiveConversation } = useApp();
  const [orb, setOrb] = useState&lt;AssistantState&gt;("idle");
  const [ask, setAsk] = useState("");
  const [briefing, setBriefing] = useState&lt;Briefing | null&gt;(null);
  const [briefingLoading, setBriefingLoading] = useState(true);
  const allTasks = useTasks("all", "");

  useEffect(() =&gt; {
    let alive = true;
    (async () =&gt; {
      try {
        const r = await fetch("/api/briefing", { cache: "no-store" });
        const j = await r.json();
        if (alive &amp;&amp; j.ok) setBriefing(j.briefing);
      } finally {
        if (alive) setBriefingLoading(false);
      }
    })();
    return () =&gt; { alive = false; };
  }, []);

  async function askJarvis(e: React.FormEvent) {
    e.preventDefault();
    if (!ask.trim()) return;
    setOrb("processing");
    try {
      // Send to chat. Parse the SSE stream so we can (a) wait for the assistant
      // reply to be persisted and (b) capture the new conversationId so the
      // Chat view opens directly on that conversation.
      const r = await fetch("/api/chat/send", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ message: ask }),
      });
      if (!r.ok || !r.body) throw new Error("Chat stream failed.");
      const reader = r.body.getReader();
      const decoder = new TextDecoder();
      let buffer = "";
      let convId: string | null = null;
      while (true) {
        const { done, value } = await reader.read();
```

```
        if (done) break;
        buffer += decoder.decode(value, { stream: true });
        let nl: number;
        while ((nl = buffer.indexOf("\n\n")) !== -1) {
          const block = buffer.slice(0, nl).trim();
          buffer = buffer.slice(nl + 2);
          if (!block.startsWith("data:")) continue;
          const payload = block.slice(5).trim();
          if (!payload) continue;
          try {
            const ev = JSON.parse(payload);
            if (ev.type === "conv" &amp;&amp; ev.conversationId) convId = ev.conversationId;
          } catch { /* ignore malformed */ }
        }
      }
      setAsk("");
      if (convId) setActiveConversation(convId);
      setView("chat");
    } catch (err) {
      toast.error(err instanceof Error ? err.message : "Failed to reach JARVIS.");
      setOrb("error");
      setTimeout(() =&gt; setOrb("idle"), 1500);
    } finally {
      setOrb("idle");
    }
  }

  return (
    &lt;div className="h-full overflow-y-auto jarvis-scrollbar"&gt;
      &lt;div className="max-w-6xl mx-auto px-4 sm:px-6 py-8 grid gap-6"&gt;
        {/* Greeting + orb */}
        &lt;section className="grid lg:grid-cols-[1fr_auto_1fr] items-center gap-6"&gt;
          &lt;div className="hidden lg:block" /&gt;
          &lt;div className="flex flex-col items-center gap-4"&gt;
            &lt;JarvisOrb state={orb} size={240} reducedMotion={false} /&gt;
            &lt;div className="text-center"&gt;
              &lt;p className="text-sm text-muted-foreground"&gt;{briefing?.greeting ?? `Welcome
back, ${user?.displayName ?? "User"}.`}&lt;/p&gt;
              &lt;h2 className="text-2xl font-semibold tracking-tight"&gt;How may I help you?&lt;/
h2&gt;
            &lt;/div&gt;
          &lt;/div&gt;
          &lt;div className="hidden lg:block" /&gt;
        &lt;/section&gt;

        {/* Ask JARVIS */}
        &lt;Card className="jarvis-panel p-4"&gt;
          &lt;form onSubmit={askJarvis} className="flex flex-col sm:flex-row gap-3"&gt;
            &lt;Input
              autoFocus
              value={ask}
              onChange={(e) =&gt; setAsk(e.target.value)}
              placeholder="Ask JARVIS anything, or ask me to create a task…"
              className="flex-1 bg-background/60"
            /&gt;
            &lt;div className="flex gap-2"&gt;
              &lt;Button type="button" variant="outline" size="icon" aria-label="Voice" onClick={()
=&gt; setView("voice")}&gt;
                &lt;Mic className="size-4" /&gt;
              &lt;/Button&gt;
              &lt;Button type="submit" size="icon" disabled={orb === "processing"} aria-
label="Send"&gt;
                &lt;Send className="size-4" /&gt;
              &lt;/Button&gt;
            &lt;/div&gt;
          &lt;/form&gt;
          &lt;p className="text-xs text-muted-foreground mt-2"&gt;
            Try: &lt;em&gt;“Remind me to call the dentist tomorrow”&lt;/em&gt; · JARVIS will draft
a task and ask before saving.
          &lt;/p&gt;
        &lt;/Card&gt;

        {/* Quick actions */}
        &lt;section className="grid sm:grid-cols-3 gap-3"&gt;
          &lt;QuickAction icon={ListChecks} label="New Task" desc="Capture something to do"
onClick={() =&gt; launchQuick("tasks", "new")} /&gt;
          &lt;QuickAction icon={NotebookPen} label="New Note" desc="Write down an idea" onClick={()
=&gt; launchQuick("notes", "new")} /&gt;
          &lt;QuickAction icon={FolderKanban} label="New Project" desc="Group tasks &amp; notes"
onClick={() =&gt; launchQuick("projects", "new")} /&gt;
        &lt;/section&gt;
```

```
        {/* Briefing + counts */}
        &lt;section className="grid lg:grid-cols-3 gap-4"&gt;
          &lt;Card className="jarvis-panel p-5 lg:col-span-2"&gt;
            &lt;div className="flex items-center gap-2 mb-3"&gt;
              &lt;Sparkles className="size-4 text-primary" /&gt;
              &lt;h3 className="font-semibold"&gt;Daily Briefing&lt;/h3&gt;
              &lt;span className="text-[10px] uppercase tracking-wider text-muted-foreground ml-
auto"&gt;Generated from your data&lt;/span&gt;
            &lt;/div&gt;
            {briefingLoading ? (
              &lt;div className="grid gap-2"&gt;
                &lt;Skeleton className="h-4 w-3/4" /&gt;
                &lt;Skeleton className="h-4 w-5/6" /&gt;
                &lt;Skeleton className="h-4 w-2/3" /&gt;
              &lt;/div&gt;
            ) : briefing?.narrativeError ? (
              &lt;p className="text-sm text-destructive"&gt;Briefing unavailable: {briefing.
narrativeError}&lt;/p&gt;
            ) : (
              &lt;p className="text-sm leading-relaxed whitespace-pre-wrap"&gt;{briefing?.narrative
?? "No briefing right now."}&lt;/p&gt;
            )}
            &lt;div className="grid grid-cols-2 sm:grid-cols-5 gap-2 mt-4"&gt;
              &lt;Stat label="Open" value={briefing?.counts.totalOpen ?? 0} /&gt;
              &lt;Stat label="Today" value={briefing?.counts.today ?? 0} /&gt;
              &lt;Stat label="Upcoming" value={briefing?.counts.upcoming ?? 0} /&gt;
              &lt;Stat label="Overdue" value={briefing?.counts.overdue ?? 0} tone={briefing?.counts.
overdue ? "warn" : "default"} /&gt;
              &lt;Stat label="Done" value={briefing?.counts.totalDone ?? 0} tone="ok" /&gt;
            &lt;/div&gt;
          &lt;/Card&gt;

          &lt;Card className="jarvis-panel p-5"&gt;
            &lt;div className="flex items-center gap-2 mb-3"&gt;
              &lt;CalendarClock className="size-4 text-primary" /&gt;
              &lt;h3 className="font-semibold"&gt;Today’s Tasks&lt;/h3&gt;
              &lt;button className="text-[11px] text-primary ml-auto hover:underline" onClick={()
=&gt; setView("tasks")}&gt;View all&lt;/button&gt;
            &lt;/div&gt;
            {briefingLoading ? (
              &lt;div className="grid gap-2"&gt;&lt;Skeleton className="h-4 w-3/4" /&gt;&lt;
Skeleton className="h-4 w-2/3" /&gt;&lt;/div&gt;
            ) : (briefing?.tasksToday ?? []).length === 0 ? (
              &lt;EmptyRow text="Nothing due today." /&gt;
            ) : (
              &lt;ul className="grid gap-2 text-sm"&gt;
                {briefing!.tasksToday.slice(0, 6).map((t) =&gt; &lt;TaskRow key={t.id} title={t.
title} priority={t.priority} /&gt;)}
              &lt;/ul&gt;
            )}
            {(briefing?.overdue ?? []).length &gt; 0 &amp;&amp; (
              &lt;div className="mt-3 pt-3 border-t border-border/60"&gt;
                &lt;p className="text-[11px] uppercase tracking-wider text-destructive mb-2"&gt;
Overdue&lt;/p&gt;
                &lt;ul className="grid gap-1.5 text-sm"&gt;
                  {briefing!.overdue.slice(0, 3).map((t) =&gt; &lt;TaskRow key={t.id} title={t.
title} priority

… (truncated for brevity; see the full file in the project repository) …
```

src/components/jarvis/views/chat.tsx

```
"use client";

import { useCallback, useEffect, useRef, useState } from "react";
import { useApp } from "@/store/app-store";
import { useAuth } from "@/components/providers/auth-provider";
import { Card } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Skeleton } from "@/components/ui/skeleton";
import { ScrollArea } from "@/components/ui/scroll-area";
import { Plus, Send, Square, Trash2, Pencil, RefreshCw, MessagesSquare } from "lucide-react";
import { toast } from "sonner";
import { Markdown } from "@/components/jarvis/markdown";
import { ToolCallCard } from "@/components/jarvis/tool-call-card";
import type { ChatMessage, Conversation } from "@/lib/types";

interface StreamEvent {
```

```
  type: "conv" | "delta" | "tool" | "done" | "error";
  conversationId?: string;
  text?: string;
  tool?: string;
  input?: Record&lt;string, unknown&gt;;
  message?: string;
}

interface RenderedMessage {
  id: string;
  role: "user" | "assistant";
  content: string;
  tools: { tool: string; input: Record&lt;string, unknown&gt; }[];
  streaming?: boolean;
  error?: boolean;
}

export function ChatView() {
  const { user } = useAuth();
  const { activeConversationId, setActiveConversation } = useApp();
  const [conversations, setConversations] = useState&lt;Conversation[]&gt;([]);
  const [messages, setMessages] = useState&lt;RenderedMessage[]&gt;([]);
  const [input, setInput] = useState("");
  const [loadingConv, setLoadingConv] = useState(false);
  const [loadingMsgs, setLoadingMsgs] = useState(false);
  const [streaming, setStreaming] = useState(false);
  const abortRef = useRef&lt;AbortController | null&gt;(null);
  const scrollRef = useRef&lt;HTMLDivElement&gt;(null);
  const renameRef = useRef&lt;HTMLInputElement | null&gt;(null);

  // Load conversation list.
  const loadConvs = useCallback(async () =&gt; {
    setLoadingConv(true);
    try {
      const r = await fetch("/api/chat/sessions", { cache: "no-store" });
      const j = await r.json();
      if (j.ok) setConversations(j.conversations);
    } finally {
      setLoadingConv(false);
    }
  }, []);
  useEffect(() =&gt; { loadConvs(); }, [loadConvs]);

  // Load messages for active conversation.
  useEffect(() =&gt; {
    if (!activeConversationId) {
      setMessages([]);
      return;
    }
    let alive = true;
    (async () =&gt; {
      setLoadingMsgs(true);
      try {
        const r = await fetch(`/api/chat/sessions/${activeConversationId}/messages`, { cache: "no-
store" });
        const j = await r.json();
        if (!j.ok) { toast.error(j.error); return; }
        if (!alive) return;
        const msgs: RenderedMessage[] = j.messages.map((m: ChatMessage) =&gt; ({
          id: m.id || crypto.randomUUID(),
          role: m.role === "user" ? "user" : "assistant",
          content: m.content,
          tools: [],
        }));
        setMessages(msgs);
      } finally {
        if (alive) setLoadingMsgs(false);
      }
    })();
    return () =&gt; { alive = false; };
  }, [activeConversationId]);

  // Auto-scroll on new content.
  useEffect(() =&gt; {
    const el = scrollRef.current;
    if (el) el.scrollTop = el.scrollHeight;
  }, [messages]);

  async function newConversation() {
    const r = await fetch("/api/chat/sessions", { method: "POST", headers: { "Content-Type":
"application/json" }, body: JSON.stringify({ title: "New conversation" }) });
```

```
    const j = await r.json();
    if (j.ok) {
      setConversations((c) =&gt; [j.conversation, ...c]);
      setActiveConversation(j.conversation.id);
    }
  }

  async function deleteConversation(id: string) {
    if (!confirm("Delete this conversation and all its messages?")) return;
    const r = await fetch(`/api/chat/sessions/${id}`, { method: "DELETE" });
    const j = await r.json();
    if (j.ok) {
      setConversations((c) =&gt; c.filter((x) =&gt; x.id !== id));
      if (activeConversationId === id) setActiveConversation(null);
      toast.success("Conversation deleted.");
    }
  }

  async function renameConversation(id: string) {
    const title = renameRef.current?.value?.trim();
    if (!title) return;
    const r = await fetch(`/api/chat/sessions/${id}`, { method: "PATCH", headers: { "Content-Type":
"application/json" }, body: JSON.stringify({ title }) });
    const j = await r.json();
    if (j.ok) {
      setConversations((c) =&gt; c.map((x) =&gt; x.id === id ? j.conversation : x));
      toast.success("Renamed.");
    }
  }

  async function send(text?: string, retryFrom?: number) {
    const content = (text ?? input).trim();
    if (!content || streaming) return;
    setInput("");

    const userMsgId = crypto.randomUUID();
    const assistantId = crypto.randomUUID();
    const baseMessages = retryFrom !== undefined ? messages.slice(0, retryFrom) : messages;
    setMessages([
      ...baseMessages,
      { id: userMsgId, role: "user", content, tools: [] },
      { id: assistantId, role: "assistant", content: "", tools: [], streaming: true },
    ]);
    setStreaming(true);

    const abort = new AbortController();
    abortRef.current = abort;

    try {
      const r = await fetch("/api/chat/send", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ conversationId: activeConversationId ?? undefined, message: content
}),
        signal: abort.signal,
      });
      if (!r.ok || !r.body) throw new Error("Chat stream unavailable.");
      // Set the active conversation from the server response.
      const reader = r.body.getReader();
      const decoder = new TextDecoder();
      let buffer = "";
      let prose = "";
      const tools: { tool: string; input: Record&lt;string, unknown&gt; }[] = [];

      while (true) {
        const { done, value } = await reader.read();
        if (done) break;
        buffer += decoder.decode(value, { stream: true });
        let nl: number;
        while ((nl = buffer.indexOf("\n\n")) !== -1) {
          const block = buffer.slice(0, nl).trim();
          buffer = buffer.slice(nl + 2);
          if (!block.startsWith("data:")) continue;
          const payload = block.slice(5).trim();
          if (!payload) continue;
          let ev: StreamEvent;
          try { ev = JSON.parse(payload); } catch { continue; }
          if (ev.type === "conv" &amp;&amp; ev.conversationId) {
            if (activeConversationId !== ev.conversationId) {
              setActiveConversation(ev.conversationId);
              loadConvs();
```

```
            }
          } else if (ev.type === "delta" &amp;&amp; ev.text) {
            prose += ev.text;
            setMessages((m) =&gt; m.map((x) =&gt; x.id === assistantId ? { ...x, content: prose } :
x));
          } else if (ev.type === "tool" &amp;&amp; ev.tool) {
            tools.push({ tool: ev.tool, input: ev.input ?? {} });
            setMessages((m) =&gt; m.map((x) =&gt; x.id === assistantId ? { ...x, content: prose,
tools: [...tools] } : x));
          } else if (ev.type === "error") {
            setMessages((m) =&gt; m.map((x) =&gt; x.id === assistantId ? { ...x, content: prose ||
ev.message!, error: true, streaming: false } : x));
            toast.error(ev.message || "The model returned an error.");
          } else if (ev.type === "done") {
            setMessages((m) =&gt; m.map((x) =&gt; x.id === assistantId ? { ...x, content: prose,
tools, streaming: false } : x));
          }
        }
      }
    } catch (e) {
      if (abort.signal.aborted) {
        setMessages((m) =&gt; m.map((x) =&gt; x.id === assistantId ? { ...x, streaming: false,
content: x.content || "_(stopped)_" } : x));
      } else {
        const msg = e instanceof Error ? e.message : "Chat failed.";
        setMessages((m) =&gt; m.map((x) =&gt; x.id === assistantId ? { ...x, content: msg, error:
true, streaming: false } : x));
        toast.error(msg);
      }
    } finally {
      setStreaming(false);
      abortRef.current = null;
    }
  }

  function stop() {
    abortRef.current?.abort();
  }

  return (
    &lt;div className="h-full grid md:grid-cols-[280px_1fr]"&gt;
      {/* Sessions */}
      &lt;div className="hidden md:flex flex-col border-r border-border/70 bg-card/40"&gt;
        &lt;div className="p-3 border-b border-border/70 flex items-center gap-2"&gt;
          &lt;Button size="sm" className="flex-1" onClick={newConversation}&gt;&lt;Plus
className="size-4" /&gt; New chat&lt;/Button&gt;
        &lt;/div&gt;
        &lt;ScrollArea className="flex-1 jarvis-scrollbar"&gt;
          &lt;ul className="p-2 grid gap-1"&gt;
            {loadingConv &amp;&amp; &lt;li className="p-2"&gt;&lt;Skeleton className="h-8 w-full" /
&gt;&lt;/li&gt;}
            {!loadingConv &amp;&amp; conversations.length === 0 &amp;&amp; (
              &lt;li className="text-xs text-muted-foreground p-3"&gt;No conversations yet.&lt;/
li&gt;
            )}
            {conversations.map((c) =&gt; (
              &lt;li key={c.id}&gt;
                &lt;div
                  className={"group flex items-center gap-2 rounded-lg px-2.5 py-2 text-sm cursor-
pointer transition " + (c.id === activeConversationId ? "bg-primary/15 text-primary-foreground" :
"hover:bg-accent/50")}
                  onClic

… (truncated for brevity; see the full file in the project repository) …
```

src/components/jarvis/views/voice.tsx

```
"use client";

import { useCallback, useEffect, useRef, useState } from "react";
import { JarvisOrb } from "@/components/jarvis/orb";
import { Card } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Slider } from "@/components/ui/slider";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/
select";
import { Mic, MicOff, Send, Square, Volume2, Pause, Play, AlertTriangle, Settings2 } from "lucide-
```

```
react";
import { toast } from "sonner";
import { useApp } from "@/store/app-store";
import { useSettings } from "@/hooks/use-settings";
import { Markdown } from "@/components/jarvis/markdown";
import { ToolCallCard } from "@/components/jarvis/tool-call-card";
import type { AssistantState } from "@/lib/types";

// Minimal types for the Web Speech API (vendor-prefixed in some browsers).
type SpeechRecognitionLike = {
  lang: string;
  continuous: boolean;
  interimResults: boolean;
  onresult: ((e: any) =&gt; void) | null;
  onerror: ((e: any) =&gt; void) | null;
  onend: (() =&gt; void) | null;
  start: () =&gt; void;
  stop: () =&gt; void;
};

export function VoiceView() {
  const { setView } = useApp();
  const { settings } = useSettings();
  const [state, setState] = useState&lt;AssistantState&gt;("idle");
  const [recognition, setRecognition] = useState&lt;SpeechRecognitionLike | null&gt;(null);
  const [supported, setSupported] = useState&lt;{ stt: boolean; tts: boolean; zaiTts: boolean }&gt;(
{ stt: false, tts: false, zaiTts: false });
  const [transcript, setTranscript] = useState("");
  const [interim, setInterim] = useState("");
  const [reply, setReply] = useState("");
  const [tools, setTools] = useState&lt;{ tool: string; input: Record&lt;string, unknown&gt; }[]&gt;
([]);
  const [listening, setListening] = useState(false);
  const [speaking, setSpeaking] = useState(false);
  const [voices, setVoices] = useState&lt;SpeechSynthesisVoice[]&gt;([]);
  const [rate, setRate] = useState(1);
  const [selectedVoice, setSelectedVoice] = useState("default");
  const [useZaiTts, setUseZaiTts] = useState(false);
  const abortRef = useRef&lt;AbortController | null&gt;(null);
  const audioRef = useRef&lt;HTMLAudioElement | null&gt;(null);
  const [zaiStatus, setZaiStatus] = useState&lt;string | null&gt;(null);

  useEffect(() =&gt; {
    const SR = (window as any).SpeechRecognition || (window as any).webkitSpeechRecognition;
    const stt = !!SR;
    const tts = typeof window !== "undefined" &amp;&amp; "speechSynthesis" in window;
    setSupported({ stt, tts, zaiTts: true });
    if (stt) {
      const rec = new SR();
      rec.lang = "en-US";
      rec.continuous = false;
      rec.interimResults = true;
      rec.onresult = (e: any) =&gt; {
        let finalText = "";
        let interimText = "";
        for (let i = e.resultIndex; i &lt; e.results.length; i++) {
          const r = e.results[i];
          if (r.isFinal) finalText += r[0].transcript;
          else interimText += r[0].transcript;
        }
        if (finalText) setTranscript((t) =&gt; (t ? t + " " : "") + finalText.trim());
        setInterim(interimText);
      };
      rec.onerror = (e: any) =&gt; {
        if (e.error === "not-allowed" || e.error === "service-not-allowed") {
          toast.error("Microphone permission denied. You can still type below.");
          setState("error");
        } else if (e.error === "no-speech") {
          toast.message("No speech detected. Try again.");
        } else {
          toast.error("Speech recognition error: " + e.error);
        }
        setListening(false);
        setState("idle");
      };
      rec.onend = () =&gt; {
        setListening(false);
        if (state !== "processing") setState("idle");
      };
      setRecognition(rec);
    }
```

```
    if (tts) {
      const load = () =&gt; setVoices(window.speechSynthesis.getVoices());
      load();
      window.speechSynthesis.onvoiceschanged = load;
    }
  }, []);

  useEffect(() =&gt; {
    if (settings?.ttsVoice &amp;&amp; settings.ttsVoice !== "default") setSelectedVoice(settings.
ttsVoice);
    if (settings?.ttsRate) setRate(settings.ttsRate);
  }, [settings?.ttsVoice, settings?.ttsRate]);

  const startListening = useCallback(() =&gt; {
    if (!recognition) {
      toast.error("Speech recognition is not available in this browser. Use typed input below.");
      return;
    }
    try {
      recognition.start();
      setListening(true);
      setState("listening");
    } catch (e) {
      toast.error("Could not start microphone. " + (e instanceof Error ? e.message : ""));
      setState("error");
      setTimeout(() =&gt; setState("idle"), 1200);
    }
  }, [recognition]);

  const stopListening = useCallback(() =&gt; {
    recognition?.stop();
    setListening(false);
    setState("idle");
  }, [recognition]);

  const stopSpeaking = useCallback(() =&gt; {
    if (supported.tts &amp;&amp; !useZaiTts) window.speechSynthesis.cancel();
    if (audioRef.current) { audioRef.current.pause(); audioRef.current.currentTime = 0; }
    setSpeaking(false);
    setState("idle");
  }, [supported.tts, useZaiTts]);

  const speak = useCallback(async (text: string) =&gt; {
    if (!text.trim()) return;
    setState("speaking");
    setSpeaking(true);
    // If browser TTS selected and available, use it; else fall back to z.ai TTS.
    if (supported.tts &amp;&amp; !useZaiTts) {
      window.speechSynthesis.cancel();
      const u = new SpeechSynthesisUtterance(text);
      u.rate = rate;
      const v = voices.find((x) =&gt; x.name === selectedVoice);
      if (v) u.voice = v;
      u.onend = () =&gt; { setSpeaking(false); setState("idle"); };
      u.onerror = () =&gt; { setSpeaking(false); setState("error"); setTimeout(() =&gt; setState(
"idle"), 1200); };
      window.speechSynthesis.speak(u);
      return;
    }
    // z.ai TTS fallback: fetch audio/wav and play it.
    try {
      setZaiStatus("Synthesizing speech…");
      const r = await fetch("/api/voice/tts", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ text, voice: selectedVoice, rate }),
      });
      if (!r.ok) {
        const j = await r.json().catch(() =&gt; ({}));
        throw new Error(j.error || `TTS failed (${r.status})`);
      }
      const blob = await r.blob();
      const url = URL.createObjectURL(blob);
      if (audioRef.current) { audioRef.current.src = url; audioRef.current.onended = () =&gt; {
setSpeaking(false); setState("idle"); URL.revokeObjectURL(url); }; audioRef.current.play(); }
      setZaiStatus(null);
    } catch (e) {
      setZaiStatus(null);
      toast.error(e instanceof Error ? e.message : "TTS failed.");
      setState("error");
      setTimeout(() =&gt; setState("idle"), 1200);
```

```
      setSpeaking(false);
    }
  }, [supported.tts, useZaiTts, rate, selectedVoice, voices]);

  async function send(text?: string) {
    const content = (text ?? transcript).trim();
    if (!content || state === "processing") return;
    setReply("");
    setTools([]);
    setTranscript("");
    setInterim("");
    setState("processing");
    const abort = new AbortController();
    abortRef.current = abort;
    try {
      const r = await fetch("/api/chat/send", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ message: content }),
        signal: abort.signal,
      });
      if (!r.ok || !r.body) throw new Error("Chat stream unavailable.");
      const reader = r.body.getReader();
      const decoder = new TextDecoder();
      let buffer = "";
      let prose = "";
      const collected: { tool: string; input: Record&lt;string, unknown&gt; }[] = [];
      while (true) {
        const { done, value } = await reader.read();
        if (done) break;
        buffer += decoder.decode(value, { stream: true });
        let nl: number;
        while ((nl = buffer.indexOf("\n\n")) !== -1) {
          const block = buffer.slice(0, nl).trim();
          buffer = buffer.slice(nl + 2);
          if (!block.startsWith("data:")) continue;
          const payload = block.slice(5).trim();
          if (!payload) continue;
          let ev: any;
          try { ev = JSON.parse(payload); } catch { continue; }
          if (ev.type === "delta" &amp;&amp; ev.text) { prose += ev.text; setReply(prose); }
          else if (ev.type === "tool" &amp;&amp; ev.tool) { collected.push({ tool: ev.tool, input:
ev.input ?? {} }); setTools([...collected]); }
          else if (ev.type === "error") { toast.error(ev.message); setState("error"); setTimeout(()
=&gt; setState("idle"), 1500); return; }
          else if (ev.type === "done") { /* finalize */ }
        }
      }
      setReply(prose);
      // Speak the final prose (excluding any tool-call text, which is empty).
      if (prose.trim() &amp;&amp; settings?.voiceEnabled !== false) {
        await speak(prose);
      } else {
        setState("idle");
      }
    } catch (e) {
      if (!abort.signal.aborted) {
        toast.error(e instanceof Error ? e.message : "Voice request

… (truncated for brevity; see the full file in the project repository) …
```

src/components/jarvis/views/tasks.tsx

```
"use client";

import { useEffect, useMemo, useState } from "react";
import { Card } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Textarea } from "@/components/ui/textarea";
import { Badge } from "@/components/ui/badge";
import { Skeleton } from "@/components/ui/skeleton";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/
select";
import { Dialog, DialogContent, DialogFooter, DialogHeader, DialogTitle, DialogTrigger } from "@/
components/ui/dialog";
import { Plus, Search, Trash2, Pencil, ListChecks, Circle, CheckCircle2, Clock } from "lucide-
react";
```

```
import { toast } from "sonner";
import { useApp } from "@/store/app-store";
import { useTasks, useCreateTask, useUpdateTask, useDeleteTask, useProjects } from "@/hooks/use-
data";
import type { Priority, Task, TaskStatus } from "@/lib/types";

const VIEWS = [
  { id: "active", label: "Open" },
  { id: "today", label: "Today" },
  { id: "upcoming", label: "Upcoming" },
  { id: "completed", label: "Completed" },
  { id: "all", label: "All" },
] as const;

export function TasksView() {
  const { quickAction, clearQuick } = useApp();
  const [view, setView] = useState&lt;(typeof VIEWS)[number]["id"]&gt;("active");
  const [q, setQ] = useState("");
  const [open, setOpen] = useState(false);
  const [editing, setEditing] = useState&lt;Task | null&gt;(null);

  const { data, isLoading, refetch } = useTasks(view, q);
  const { projects } = useProjects("").data ?? {};
  const createM = useCreateTask();
  const updateM = useUpdateTask();
  const deleteM = useDeleteTask();

  useEffect(() =&gt; {
    if (quickAction?.target === "tasks" &amp;&amp; quickAction.action === "new") {
      // eslint-disable-next-line react-hooks/set-state-in-effect
      setEditing(null);
      setOpen(true);
      clearQuick();
    }
  }, [quickAction, clearQuick]);

  const tasks = data?.tasks ?? [];
  const counts = useMemo(() =&gt; ({
    open: tasks.filter((t) =&gt; t.status !== "done").length,
    done: tasks.filter((t) =&gt; t.status === "done").length,
    high: tasks.filter((t) =&gt; t.priority === "high" &amp;&amp; t.status !== "done").length,
  }), [tasks]);

  async function toggle(t: Task) {
    await updateM.mutateAsync({ id: t.id, patch: { status: t.status === "done" ? "todo" : "done" }
});
  }

  async function remove(t: Task) {
    if (!confirm(`Delete task "${t.title}"?`)) return;
    await deleteM.mutateAsync(t.id);
    toast.success("Task deleted.");
  }

  return (
    &lt;div className="h-full overflow-y-auto jarvis-scrollbar"&gt;
      &lt;div className="max-w-5xl mx-auto px-4 sm:px-6 py-6 grid gap-4"&gt;
        {/* Header */}
        &lt;div className="flex flex-col sm:flex-row sm:items-center gap-3"&gt;
          &lt;div className="flex-1"&gt;
            &lt;h2 className="text-xl font-semibold flex items-center gap-2"&gt;&lt;ListChecks
className="size-5 text-primary" /&gt; Tasks&lt;/h2&gt;
            &lt;p className="text-sm text-muted-foreground"&gt;{counts.open} open · {counts.high}
high priority · {counts.done} completed&lt;/p&gt;
          &lt;/div&gt;
          &lt;div className="flex gap-2"&gt;
            &lt;div className="relative"&gt;
              &lt;Search className="absolute left-2.5 top-1/2 -translate-y-1/2 size-4 text-muted-
foreground" /&gt;
              &lt;Input value={q} onChange={(e) =&gt; setQ(e.target.value)} placeholder="Search
tasks…" className="pl-8 w-56 bg-background/60" /&gt;
            &lt;/div&gt;
            &lt;Dialog open={open} onOpenChange={setOpen}&gt;
              &lt;DialogTrigger asChild&gt;
                &lt;Button onClick={() =&gt; { setEditing(null); setOpen(true); }}&gt;&lt;Plus
className="size-4" /&gt; New&lt;/Button&gt;
              &lt;/DialogTrigger&gt;
              &lt;TaskDialog
                editing={editing}
                projects={projects ?? []}
                onClose={() =&gt; setOpen(false)}
```

```
                onSave={async (payload) =&gt; {
                  if (editing) await updateM.mutateAsync({ id: editing.id, patch: payload });
                  else await createM.mutateAsync(payload as Task);
                  setOpen(false);
                  refetch();
                }}
              /&gt;
            &lt;/Dialog&gt;
          &lt;/div&gt;
        &lt;/div&gt;

        {/* View tabs */}
        &lt;div className="flex flex-wrap gap-1.5"&gt;
          {VIEWS.map((v) =&gt; (
            &lt;button
              key={v.id}
              onClick={() =&gt; setView(v.id)}
              className={"px-3 py-1.5 rounded-full text-xs font-medium border transition " + (view
=== v.id ? "bg-primary/15 border-primary/40 text-primary" : "border-border/60 text-muted-foreground
hover:bg-accent/50")}
            &gt;
              {v.label}
            &lt;/button&gt;
          ))}
        &lt;/div&gt;

        {/* List */}
        &lt;div className="grid gap-2"&gt;
          {isLoading ? (
            &lt;div className="grid gap-2"&gt;{Array.from({ length: 4 }).map((_, i) =&gt; &lt;
Skeleton key={i} className="h-14 w-full" /&gt;)}&lt;/div&gt;
          ) : tasks.length === 0 ? (
            &lt;Card className="jarvis-panel p-8 text-center"&gt;
              &lt;ListChecks className="size-8 mx-auto text-muted-foreground mb-2" /&gt;
              &lt;p className="text-sm text-muted-foreground"&gt;No tasks here. Create one to get
started.&lt;/p&gt;
            &lt;/Card&gt;
          ) : (
            tasks.map((t) =&gt; &lt;TaskRow key={t.id} t={t} onToggle={() =&gt; toggle(t)} onEdit={(
) =&gt; { setEditing(t); setOpen(true); }} onDelete={() =&gt; remove(t)} /&gt;)
          )}
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
}

function priorityBadge(p: Priority) {
  const cls = p === "high" ? "bg-destructive/20 text-destructive" : p === "medium" ? "bg-amber-500/
20 text-amber-300" : "bg-emerald-500/20 text-emerald-300";
  return &lt;span className={"text-[10px] px-1.5 py-0.5 rounded " + cls}&gt;{p}&lt;/span&gt;;
}

function TaskRow({ t, onToggle, onEdit, onDelete }: { t: Task; onToggle: () =&gt; void; onEdit: ()
=&gt; void; onDelete: () =&gt; void }) {
  const done = t.status === "done";
  return (
    &lt;Card className="jarvis-panel p-3 flex items-start gap-3"&gt;
      &lt;button onClick={onToggle} className="mt-0.5" aria-label={done ? "Mark not done" : "Mark
done"}&gt;
        {done ? &lt;CheckCircle2 className="size-5 text-emerald-400" /&gt; : &lt;Circle
className="size-5 text-muted-foreground hover:text-primary" /&gt;}
      &lt;/button&gt;
      &lt;div className="flex-1 min-w-0"&gt;
        &lt;div className="flex items-center gap-2"&gt;
          &lt;span className={"font-medium truncate " + (done ? "line-through text-muted-
foreground" : "")}&gt;{t.title}&lt;/span&gt;
          {priorityBadge(t.priority)}
          {t.category &amp;&amp; &lt;Badge variant="outline" className="text-[10px]"&gt;{t.
category}&lt;/Badge&gt;}
        &lt;/div&gt;
        {t.description &amp;&amp; &lt;p className="text-xs text-muted-foreground mt-0.5 line-clamp-
2"&gt;{t.description}&lt;/p&gt;}
        {t.dueDate &amp;&amp; (
          &lt;div className={"text-[11px] mt-1 flex items-center gap-1 " + (new Date(t.dueDate) &lt;
 new Date() &amp;&amp; !done ? "text-destructive" : "text-muted-foreground")}&gt;
            &lt;Clock className="size-3" /&gt; {new Date(t.dueDate).toLocaleString(undefined, {
month: "short", day: "numeric", hour: "numeric", minute: "2-digit" })}
          &lt;/div&gt;
        )}
      &lt;/div&gt;
```

```
      &lt;div className="flex items-center gap-1"&gt;
        &lt;Button size="icon" variant="ghost" className="size-8" onClick={onEdit} aria-
label="Edit"&gt;&lt;Pencil className="size-3.5" /&gt;&lt;/Button&gt;
        &lt;Button size="icon" variant="ghost" className="size-8 text-destructive" onClick={
onDelete} aria-label="Delete"&gt;&lt;Trash2 className="size-3.5" /&gt;&lt;/Button&gt;
      &lt;/div&gt;
    &lt;/Card&gt;
  );
}

function TaskDialog({ editing, projects, onClose, onSave }: {
  editing: Task | null;
  projects: { id: string; name: string }[];
  onClose: () =&gt; void;
  onSave: (p: Partial&lt;Task&gt;) =&gt; Promise&lt;void&gt;;
}) {
  const [title, setTitle] = useState(editing?.title ?? "");
  const [description, setDescription] = useState(editing?.description ?? "");
  const [priority, setPriority] = useState&lt;Priority&gt;((editing?.priority as Priority) ??
"medium");
  const [category, setCategory] = useState(editing?.category ?? "");
  const [status, setStatus] = useState&lt;TaskStatus&gt;((editing?.status as TaskStatus) ?? "todo");
  const [dueDate, setDueDate] = useState(editing?.dueDate ? editing.dueDate.slice(0, 16) : "");
  const [projectId, setProjectId] = useState(editing?.projectId ?? "");
  const [busy, setBusy] = useState(false);

  useEffect(() =&gt; {
    setTitle(editing?.title ?? "");
    setDescription(editing?.description ?? "");
    setPriority((editing?.priority as Priority) ?? "medium");
    setCategory(editing?.category ?? "");
    setStatus((editing?.status as TaskStatus) ?? "todo");
    setDueDate(editing?.dueDate ? editing.dueDate.slice(0, 16) : "");
    setProjectId(editing?.projectId ?? "");
  }, [editing]);

  async function submit() {
    if (!title.trim()) { toast.error("Title is required."); return; }
    setBusy(true);
    try {
      await onSave({
        title, description, priority, category, status,
        dueDate: dueDate ? new Date(dueDate).toISOString() : nu

… (truncated for brevity; see the full file in the project repository) …
```

src/components/jarvis/views/notes.tsx

```
"use client";

import { useEffect, useMemo, useState } from "react";
import { Card } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Textarea } from "@/components/ui/textarea";
import { Label } from "@/components/ui/label";
import { Skeleton } from "@/components/ui/skeleton";
import { NotebookPen, Plus, Search, Trash2, Save, Sparkles, Eye, Pencil, X } from "lucide-react";
import { toast } from "sonner";
import { useApp } from "@/store/app-store";
import { useNotes, useCreateNote, useUpdateNote, useDeleteNote, useProjects } from "@/hooks/use-
data";
import { Markdown } from "@/components/jarvis/markdown";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/
select";
import type { Note } from "@/lib/types";

export function NotesView() {
  const { quickAction, clearQuick } = useApp();
  const [q, setQ] = useState("");
  const [selected, setSelected] = useState&lt;Note | null&gt;(null);
  const [editorOpen, setEditorOpen] = useState(false);
  const [editing, setEditing] = useState&lt;Note | null&gt;(null);

  const { data, isLoading, refetch } = useNotes(q);
  const { projects } = useProjects("").data ?? {};
  const createM = useCreateNote();
  const updateM = useUpdateNote();
  const deleteM = useDeleteNote();
```

```
  useEffect(() =&gt; {
    if (quickAction?.target === "notes" &amp;&amp; quickAction.action === "new") {
      // eslint-disable-next-line react-hooks/set-state-in-effect
      setEditing(null);
      setEditorOpen(true);
      clearQuick();
    }
  }, [quickAction, clearQuick]);

  const notes = data?.notes ?? [];

  return (
    &lt;div className="h-full grid lg:grid-cols-[320px_1fr]"&gt;
      {/* List */}
      &lt;div className={"border-r border-border/70 bg-card/40 flex flex-col " + (selected &amp;
&amp; "hidden lg:flex")}&gt;
        &lt;div className="p-3 border-b border-border/70 flex gap-2"&gt;
          &lt;div className="relative flex-1"&gt;
            &lt;Search className="absolute left-2.5 top-1/2 -translate-y-1/2 size-4 text-muted-
foreground" /&gt;
            &lt;Input value={q} onChange={(e) =&gt; setQ(e.target.value)} placeholder="Search
notes…" className="pl-8 bg-background/60" /&gt;
          &lt;/div&gt;
          &lt;Button size="icon" onClick={() =&gt; { setEditing(null); setEditorOpen(true); }} aria-
label="New note"&gt;&lt;Plus className="size-4" /&gt;&lt;/Button&gt;
        &lt;/div&gt;
        &lt;div className="flex-1 overflow-y-auto jarvis-scrollbar p-2"&gt;
          {isLoading ? (
            &lt;div className="grid gap-2"&gt;{Array.from({ length: 4 }).map((_, i) =&gt; &lt;
Skeleton key={i} className="h-16 w-full" /&gt;)}&lt;/div&gt;
          ) : notes.length === 0 ? (
            &lt;div className="p-6 text-center"&gt;
              &lt;NotebookPen className="size-7 mx-auto text-muted-foreground mb-2" /&gt;
              &lt;p className="text-sm text-muted-foreground"&gt;No notes yet.&lt;/p&gt;
            &lt;/div&gt;
          ) : (
            &lt;ul className="grid gap-1"&gt;
              {notes.map((n) =&gt; (
                &lt;li key={n.id}&gt;
                  &lt;button
                    onClick={() =&gt; setSelected(n)}
                    className={"w-full text-left rounded-lg p-2.5 transition " + (selected?.id ===
n.id ? "bg-primary/15 border border-primary/30" : "hover:bg-accent/50 border border-transparent")}
                  &gt;
                    &lt;div className="font-medium text-sm truncate"&gt;{n.title || "Untitled"}&lt;/
div&gt;
                    &lt;div className="text-xs text-muted-foreground line-clamp-2 mt-0.5"&gt;{
stripMd(n.content)}&lt;/div&gt;
                    &lt;div className="text-[10px] text-muted-foreground mt-1"&gt;{new Date(n.
updatedAt).toLocaleString()}&lt;/div&gt;
                  &lt;/button&gt;
                &lt;/li&gt;
              ))}
            &lt;/ul&gt;
          )}
        &lt;/div&gt;
      &lt;/div&gt;

      {/* Detail / editor */}
      &lt;div className="overflow-y-auto jarvis-scrollbar"&gt;
        {editorOpen ? (
          &lt;NoteEditor
            note={editing}
            projects={projects ?? []}
            onClose={() =&gt; { setEditorOpen(false); setEditing(null); }}
            onSave={async (payload) =&gt; {
              if (editing) await updateM.mutateAsync({ id: editing.id, patch: payload });
              else { const created = await createM.mutateAsync(payload as Note); setSelected(
created); }
              setEditorOpen(false);
              setEditing(null);
              refetch();
            }}
          /&gt;
        ) : selected ? (
          &lt;NoteDetail
            note={selected}
            onEdit={() =&gt; { setEditing(selected); setEditorOpen(true); }}
            onBack={() =&gt; setSelected(null)}
            onDelete={async () =&gt; {
              if (!confirm("Delete this note?")) return;
```

```
              await deleteM.mutateAsync(selected.id);
              setSelected(null);
              refetch();
              toast.success("Note deleted.");
            }}
          /&gt;
        ) : (
          &lt;div className="h-full grid place-items-center p-8 text-center"&gt;
            &lt;div&gt;
              &lt;NotebookPen className="size-10 mx-auto text-muted-foreground mb-3" /&gt;
              &lt;p className="text-sm text-muted-foreground"&gt;Select a note, or create a new one.
&lt;/p&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        )}
      &lt;/div&gt;
    &lt;/div&gt;
  );
}

function stripMd(md: string) {
  return md.replace(/[#*`&gt;\-]/g, "").replace(/\s+/g, " ").trim().slice(0, 120);
}

function NoteDetail({ note, onEdit, onBack, onDelete }: { note: Note; onEdit: () =&gt; void; onBack:
 () =&gt; void; onDelete: () =&gt; void }) {
  const [summary, setSummary] = useState&lt;string | null&gt;(null);
  const [busy, setBusy] = useState(false);
  const [err, setErr] = useState&lt;string | null&gt;(null);

  async function summarize() {
    setBusy(true); setErr(null);
    try {
      const r = await fetch(`/api/notes/${note.id}/summarize`, { method: "POST" });
      const j = await r.json();
      if (!j.ok) throw new Error(j.error);
      setSummary(j.summary);
    } catch (e) {
      setErr(e instanceof Error ? e.message : "Summarization failed.");
    } finally {
      setBusy(false);
    }
  }

  return (
    &lt;div className="max-w-3xl mx-auto p-4 sm:p-6"&gt;
      &lt;div className="flex items-start gap-2 mb-3"&gt;
        &lt;Button variant="ghost" size="icon" className="lg:hidden" onClick={onBack} aria-
label="Back to notes"&gt;&lt;X className="size-4" /&gt;&lt;/Button&gt;
        &lt;div className="flex-1"&gt;
          &lt;h2 className="text-xl font-semibold"&gt;{note.title || "Untitled"}&lt;/h2&gt;
          &lt;p className="text-xs text-muted-foreground"&gt;Updated {new Date(note.updatedAt).
toLocaleString()}&lt;/p&gt;
        &lt;/div&gt;
        &lt;Button variant="outline" size="sm" onClick={summarize} disabled={busy}&gt;&lt;Sparkles
className="size-4" /&gt; {busy ? "Summarizing…" : "Summarize"}&lt;/Button&gt;
        &lt;Button variant="ghost" size="icon" onClick={onEdit} aria-label="Edit"&gt;&lt;Pencil
className="size-4" /&gt;&lt;/Button&gt;
        &lt;Button variant="ghost" size="icon" className="text-destructive" onClick={onDelete} aria-
label="Delete"&gt;&lt;Trash2 className="size-4" /&gt;&lt;/Button&gt;
      &lt;/div&gt;
      {err &amp;&amp; &lt;p className="text-sm text-destructive mb-3"&gt;{err}&lt;/p&gt;}
      {summary &amp;&amp; (
        &lt;Card className="jarvis-panel p-4 mb-3 border-primary/30"&gt;
          &lt;div className="text-[10px] uppercase tracking-wider text-primary mb-1 flex items-
center gap-1"&gt;&lt;Sparkles className="size-3" /&gt; AI summary&lt;/div&gt;
          &lt;Markdown content={summary} /&gt;
          &lt;button className="text-xs text-muted-foreground hover:text-foreground mt-2" onClick={(
) =&gt; setSummary(null)}&gt;Dismiss&lt;/button&gt;
        &lt;/Card&gt;
      )}
      &lt;Card className="jarvis-panel p-4"&gt;
        &lt;Markdown content={note.content || "_Empty note._"} /&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
}

function NoteEditor({ note, projects, onClose, onSave }: {
  note: Note | null;
  projects: { id: string; name: string }[];
```

```
  onClose: () =&gt; void;
  onSave: (p: { title: string; content: string; projectId?: string | null }) =&gt; Promise&lt;
void&gt;;
}) {
  const [title, setTitle] = useState(note?.title ?? "");
  const [content, setContent] = useState(note?.content ?? "");
  const [projectId, setProjectId] = useState(note?.projectId ?? "");
  const [preview, setPreview] = useState(false);
  const [busy, setBusy] = useState(false);

  useEffect(() =&gt; {
    setTitle(note?.title ?? "");
    setContent(note?.content ?? "");
    setProjectId(note?.projectId ?? "");
    setPreview(false);
  }, [note]);

  async function save() {
    if (!title.trim()) { toast.error("Title is required."); return; }
    setBusy(true);
    try {
      await onSave({ title, content, projectId: projectId || null });
      toast.success(note ? "Note updated." : "Note saved.");
      onClose();
    } finally {
      setBusy(false);
    }
  }

  return (
    &lt;div className="max-w-3xl mx-auto p-4 sm:p-6 grid gap-3"&gt;
      &lt;div className="flex items-center gap-2"&gt;
        &lt;Button variant="ghost" size="icon" onClick={onClose} aria-label="Close"&gt;&lt;X
className="size-4" /&gt;&lt;/Button&gt;
        &lt;h2 className="text-xl font-semibold flex-1"&gt;{note ? "Edit note" : "New note"}&lt;/
h2&gt;
        &lt;Button variant="

… (truncated for brevity; see the full file in the project repository) …
```

src/components/jarvis/views/projects.tsx

```
"use client";

import { useEffect, useState } from "react";
import { Card } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Textarea } from "@/components/ui/textarea";
import { Label } from "@/components/ui/label";
import { Skeleton } from "@/components/ui/skeleton";
import { Dialog, DialogContent, DialogFooter, DialogHeader, DialogTitle, DialogTrigger } from "@/
components/ui/dialog";
import { Plus, FolderKanban, Pencil, Trash2, ArrowLeft, Sparkles, ListChecks, NotebookPen } from
"lucide-react";
import { toast } from "sonner";
import { useApp } from "@/store/app-store";
import { useProjects, useCreateProject } from "@/hooks/use-data";
import { Markdown } from "@/components/jarvis/markdown";
import type { Project, Task, Note as NoteT } from "@/lib/types";

export function ProjectsView() {
  const { quickAction, clearQuick, activeProjectId, setActiveProject } = useApp();
  const [open, setOpen] = useState(false);
  const [editing, setEditing] = useState&lt;Project | null&gt;(null);
  const [detail, setDetail] = useState&lt;(Project &amp; { tasks: Task[]; notes: NoteT[] }) |
null&gt;(null);
  const [loadingDetail, setLoadingDetail] = useState(false);
  const [plan, setPlan] = useState&lt;string | null&gt;(null);
  const [planning, setPlanning] = useState(false);
  const [planErr, setPlanErr] = useState&lt;string | null&gt;(null);

  const { data, isLoading, refetch } = useProjects("");
  const createM = useCreateProject();

  useEffect(() =&gt; {
    if (quickAction?.target === "projects" &amp;&amp; quickAction.action === "new") {
      setEditing(null);
      setOpen(true);
```

```
      clearQuick();
    }
  }, [quickAction, clearQuick]);

  useEffect(() =&gt; {
    if (!activeProjectId) { setDetail(null); return; }
    let alive = true;
    (async () =&gt; {
      setLoadingDetail(true);
      try {
        const r = await fetch(`/api/projects/${activeProjectId}`, { cache: "no-store" });
        const j = await r.json();
        if (!j.ok) { toast.error(j.error); return; }
        if (alive) setDetail(j.project);
      } finally {
        if (alive) setLoadingDetail(false);
      }
    })();
    return () =&gt; { alive = false; };
  }, [activeProjectId]);

  if (detail) {
    return (
      &lt;&gt;
        &lt;ProjectDetail
          project={detail}
          loading={loadingDetail}
          plan={plan}
          planning={planning}
          planErr={planErr}
          onBack={() =&gt; { setActiveProject(null); setDetail(null); setPlan(null); }}
          onPlan={async () =&gt; {
            setPlanning(true); setPlanErr(null);
            try {
              const r = await fetch(`/api/projects/${detail.id}/plan`, { method: "POST" });
              const j = await r.json();
              if (!j.ok) throw new Error(j.error);
              setPlan(j.plan);
            } catch (e) {
              setPlanErr(e instanceof Error ? e.message : "Planning failed.");
            } finally {
              setPlanning(false);
            }
          }}
          onEdit={() =&gt; { setEditing(detail); setOpen(true); }}
          onDeleted={async () =&gt; {
            if (!confirm("Delete this project? Tasks and notes will be detached (not deleted).")) 
return;
            const r = await fetch(`/api/projects/${detail.id}`, { method: "DELETE" });
            const j = await r.json();
            if (j.ok) { toast.success("Project deleted."); setActiveProject(null); setDetail(null); 
refetch(); }
            else toast.error(j.error);
          }}
        /&gt;
        {/* Dialog is always mounted so the Edit button from the detail view works. */}
        &lt;Dialog open={open} onOpenChange={setOpen}&gt;
          &lt;ProjectDialog editing={editing} onClose={() =&gt; setOpen(false)} onSave={async (p) 
=&gt; {
            if (editing) {
              const r = await fetch(`/api/projects/${editing.id}`, { method: "PATCH", headers: { 
"Content-Type": "application/json" }, body: JSON.stringify(p) });
              const j = await r.json();
              if (!j.ok) { toast.error(j.error); return; }
              toast.success("Project updated.");
              // Refresh detail in place.
              setDetail(j.project);
            } else {
              const r = await fetch("/api/projects", { method: "POST", headers: { "Content-Type": 
"application/json" }, body: JSON.stringify(p) });
              const j = await r.json();
              if (!j.ok) { toast.error(j.error); return; }
              toast.success("Project created.");
            }
            setOpen(false);
            refetch();
          }} /&gt;
        &lt;/Dialog&gt;
      &lt;/&gt;
    );
  }
```

```
  const projects = data?.projects ?? [];

  return (
    &lt;div className="h-full overflow-y-auto jarvis-scrollbar"&gt;
      &lt;div className="max-w-5xl mx-auto px-4 sm:px-6 py-6 grid gap-4"&gt;
        &lt;div className="flex items-center gap-3"&gt;
          &lt;div className="flex-1"&gt;
            &lt;h2 className="text-xl font-semibold flex items-center gap-2"&gt;&lt;FolderKanban
classNAme="size-5 text-primary" /&gt; Projects&lt;/h2&gt;
            &lt;p className="text-sm text-muted-foreground"&gt;{projects.length} projects&lt;/p&gt;
          &lt;/div&gt;
          &lt;Dialog open={open} onOpenChange={setOpen}&gt;
            &lt;DialogTrigger asChild&gt;
              &lt;Button onClick={() =&gt; { setEditing(null); setOpen(true); }}&gt;&lt;Plus
classNAme="size-4" /&gt; New&lt;/Button&gt;
            &lt;/DialogTrigger&gt;
            &lt;ProjectDialog editing={editing} onClose={() =&gt; setOpen(false)} onSave={async (p)
=&gt; {
              if (editing) {
                await fetch(`/api/projects/${editing.id}`, { method: "PATCH", headers: { "Content-
Type": "application/json" }, body: JSON.stringify(p) });
                toast.success("Project updated.");
              } else {
                const r = await fetch("/api/projects", { method: "POST", headers: { "Content-Type":
"application/json" }, body: JSON.stringify(p) });
                const j = await r.json();
                if (j.ok) toast.success("Project created."); else throw new Error(j.error);
              }
              setOpen(false);
              refetch();
            }} /&gt;
          &lt;/Dialog&gt;
        &lt;/div&gt;

        {isLoading ? (
          &lt;div className="grid sm:grid-cols-2 lg:grid-cols-3 gap-3"&gt;{Array.from({ length: 3
}).map((_, i) =&gt; &lt;Skeleton key={i} className="h-32" /&gt;)}&lt;/div&gt;
        ) : projects.length === 0 ? (
          &lt;Card className="jarvis-panel p-8 text-center"&gt;
            &lt;FolderKanban className="size-8 mx-auto text-muted-foreground mb-2" /&gt;
            &lt;p className="text-sm text-muted-foreground"&gt;No projects yet. Create one to group
tasks and notes.&lt;/p&gt;
          &lt;/Card&gt;
        ) : (
          &lt;div className="grid sm:grid-cols-2 lg:grid-cols-3 gap-3"&gt;
            {projects.map((p) =&gt; (
              &lt;button key={p.id} onClick={() =&gt; setActiveProject(p.id)} className="jarvis-
panel p-4 text-left hover:jarvis-glow transition group"&gt;
                &lt;div className="flex items-center justify-between mb-2"&gt;
                  &lt;span className="font-medium truncate group-hover:text-primary transition"&gt;{
p.name}&lt;/span&gt;
                  &lt;span className="text-xs text-muted-foreground"&gt;{p.progress}%&lt;/span&gt;
                &lt;/div&gt;
                &lt;div className="h-2 rounded-full bg-muted overflow-hidden mb-3"&gt;
                  &lt;div className="h-full bg-primary" style={{ width: `${p.progress}%` }} /&gt;
                &lt;/div&gt;
                &lt;p className="text-xs text-muted-foreground line-clamp-2"&gt;{p.description ||
"No description."}&lt;/p&gt;
                &lt;div className="flex gap-3 text-[11px] text-muted-foreground mt-3"&gt;
                  &lt;span className="flex items-center gap-1"&gt;&lt;ListChecks className="size-3"
/&gt; {p.completedTaskCount}/{p.taskCount} tasks&lt;/span&gt;
                  &lt;span className="flex items-center gap-1"&gt;&lt;NotebookPen className="size-
3" /&gt; {p.noteCount} notes&lt;/span&gt;
                &lt;/div&gt;
              &lt;/button&gt;
            ))}
          &lt;/div&gt;
        )}
      &lt;/div&gt;
    &lt;/div&gt;
  );
}

function ProjectDetail({ project, loading, plan, planning, planErr, onBack, onPlan, onEdit,
onDeleted }: {
  project: Project &amp;amp; { tasks: Task[]; notes: NoteT[] };
  loading: boolean;
  plan: string | null;
  planning: boolean;
  planErr: string | null;
  onBack: () =&gt; void;
```

```
  onPlan: () =&gt; void;
  onEdit: () =&gt; void;
  onDeleted: () =&gt; void;
}) {
  return (
    &lt;div className="h-full overflow-y-auto jarvis-scrollbar"&gt;
      &lt;div className="max-w-4xl mx-auto px-4 sm:px-6 py-6 grid gap-4"&gt;
        &lt;div className="flex items-center gap-2"&gt;
          &lt;Button variant="ghost" size="icon" onClick={onBack} aria-label="Back"&gt;&lt;
ArrowLeft className="size-4" /&gt;&lt;/Button&gt;
          &lt;div className="flex-1"&gt;
            &lt;h2 className="text-xl font-semibold"&gt;{project.name}&lt;/h2&gt;
            &lt;p className="text-xs text-muted-foreground"&gt;{project.completedTaskCount}/{
project.taskCount} tasks · {project.progress}% complete&lt;/p&gt;
          &lt;/div&gt;
          &lt;Button variant="outline" size="sm" onClick={onEdit}&gt;&lt;Pencil className="size-4" /
&gt; Edit&lt;/Button&gt;


… (truncated for brevity; see the full file in the project repository) …
```

src/components/jarvis/views/image-studio.tsx

```
"use client";

import { useState } from "react";
import { useMutation } from "@tanstack/react-query";
import { Card } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/
select";
import { Wand2, Loader2, Download, Trash2, Sparkles, ImageIcon, Save } from "lucide-react";
import { toast } from "sonner";
import { useImagesStore } from "@/store/images-store";
import type { ImageSize } from "@/lib/types";

const SIZES: { id: ImageSize; label: string }[] = [
  { id: "1024x1024", label: "Square · 1024×1024" },
  { id: "1344x768", label: "Landscape · 1344×768" },
  { id: "768x1344", label: "Portrait · 768×1344" },
  { id: "1440x720", label: "Wide · 1440×720" },
  { id: "720x1440", label: "Tall · 720×1440" },
];

const PRESETS = [
  "A serene mountain lake at sunset, cinematic, ultra detailed",
  "Futuristic command center, holographic UI, cyan glow, dark graphite",
  "A robot butler serving tea in a sunlit conservatory, studio lighting",
  "Minimalist poster of a fox made of circuit boards",
  "Anime-style city street at night after rain, neon reflections",
];

export function ImageStudioView() {
  const [prompt, setPrompt] = useState("");
  const [size, setSize] = useState&lt;ImageSize&gt;("1024x1024");
  const { images, removeImage, clear } = useImagesStore();

  const generate = useMutation({
    mutationFn: async () =&gt; {
      const r = await fetch("/api/images/generate", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ prompt, size }),
      });
      const j = await r.json();
      if (!j.ok) throw new Error(j.error);
      return j.images as string[];
    },
    onSuccess: (imgs) =&gt; {
      for (const src of imgs) useImagesStore.getState().addImage({ src, prompt, size });
      toast.success(`Generated ${imgs.length} image${imgs.length &gt; 1 ? "s" : ""}.`);
    },
    onError: (e) =&gt; toast.error(e instanceof Error ? e.message : "Generation failed."),
  });

  async function saveAsNote(src: string, p: string) {
    try {
```

```
      const r = await fetch("/api/notes", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ title: `Image: ${p.slice(0, 50)}`, content: `![${p}](${src})\n\
n_Prompt:_ ${p}` }),
      });
      const j = await r.json();
      if (j.ok) toast.success("Saved as note.");
      else toast.error(j.error || "Could not save note (database schema not applied?).");
    } catch (e) {
      toast.error(e instanceof Error ? e.message : "Save failed.");
    }
  }

  return (
    &lt;div className="h-full overflow-y-auto jarvis-scrollbar"&gt;
      &lt;div className="max-w-5xl mx-auto px-4 sm:px-6 py-6 grid gap-5"&gt;
        &lt;div&gt;
          &lt;h2 className="text-xl font-semibold flex items-center gap-2"&gt;&lt;Wand2
classNa
me="size-5 text-primary" /&gt; Image Studio&lt;/h2&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;Generate images from text with z.ai.
Generated images stay in this session; save the ones you like to a note.&lt;/p&gt;
        &lt;/div&gt;

        &lt;Card className="jarvis-panel p-5"&gt;
          &lt;div className="grid gap-3"&gt;
            &lt;div className="grid gap-1.5"&gt;
              &lt;Label htmlFor="prompt"&gt;Prompt&lt;/Label&gt;
              &lt;Input id="prompt" value={prompt} onChange={(e) =&gt; setPrompt(e.target.value)}
placeholder="Describe the image you want…" className="bg-background/60" /&gt;
            &lt;/div&gt;
            &lt;div className="grid sm:grid-cols-[1fr_auto] gap-3"&gt;
              &lt;div className="grid gap-1.5"&gt;
                &lt;Label&gt;Size&lt;/Label&gt;
                &lt;Select value={size} onValueChange={(v) =&gt; setSize(v as ImageSize)}&gt;
                  &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
                  &lt;SelectContent&gt;
                    {SIZES.map((s) =&gt; &lt;SelectItem key={s.id} value={s.id}&gt;{s.label}&lt;/
SelectItem&gt;)}
                  &lt;/SelectContent&gt;
                &lt;/Select&gt;
              &lt;/div&gt;
              &lt;div className="flex items-end"&gt;
                &lt;Button onClick={() =&gt; generate.mutate()} disabled={generate.isPending ||
prompt.trim().length &lt; 3} className="h-9"&gt;
                  {generate.isPending ? &lt;Loader2 className="size-4 animate-spin" /&gt; : &lt;
Sparkles className="size-4" /&gt;}
                  {generate.isPending ? "Generating…" : "Generate"}
                &lt;/Button&gt;
              &lt;/div&gt;
            &lt;/div&gt;
            &lt;div className="flex flex-wrap gap-2 pt-1"&gt;
              {PRESETS.map((p) =&gt; (
                &lt;button key={p} onClick={() =&gt; setPrompt(p)} className="text-xs rounded-full
border border-border/60 px-2.5 py-1 text-muted-foreground hover:bg-accent/50 hover:text-foreground
transition"&gt;
                  {p.slice(0, 40)}…
                &lt;/button&gt;
              ))}
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/Card&gt;

        {images.length === 0 ? (
          &lt;Card className="jarvis-panel p-10 text-center"&gt;
            &lt;ImageIcon className="size-10 mx-auto text-muted-foreground mb-3" /&gt;
            &lt;p className="text-sm text-muted-foreground"&gt;No images yet. Enter a prompt and
hit Generate.&lt;/p&gt;
          &lt;/Card&gt;
        ) : (
          &lt;div className="grid sm:grid-cols-2 gap-4"&gt;
            {images.map((img) =&gt; (
              &lt;Card key={img.id} className="jarvis-panel p-3 overflow-hidden"&gt;
                &lt;img src={img.src} alt={img.prompt} className="rounded-lg w-full" /&gt;
                &lt;div className="px-1 pt-3"&gt;
                  &lt;p className="text-xs text-muted-foreground line-clamp-2"&gt;{img.prompt}&lt;/
p&gt;
                  &lt;div className="flex items-center justify-between mt-2"&gt;
                    &lt;span className="text-[10px] text-muted-foreground"&gt;{img.size} · {new
Date(img.createdAt).toLocaleTimeString()}&lt;/span&gt;
                    &lt;div className="flex gap-1"&gt;
```

```
                      &lt;a href={img.src} download={`jarvis-${img.id}.png`} className="rounded-md
border border-border/60 p-1.5 hover:bg-accent/50" title="Download"&gt;&lt;Download className="size-
3.5" /&gt;&lt;/a&gt;
                      &lt;button onClick={() =&gt; saveAsNote(img.src, img.prompt)}
className="rounded-md border border-border/60 p-1.5 hover:bg-accent/50" title="Save as note"&gt;&lt;
Save className="size-3.5" /&gt;&lt;/button&gt;
                      &lt;button onClick={() =&gt; removeImage(img.id)} className="rounded-md
border border-border/60 p-1.5 hover:bg-accent/50 text-destructive" title="Delete"&gt;&lt;Trash2
className="size-3.5" /&gt;&lt;/button&gt;
                    &lt;/div&gt;
                  &lt;/div&gt;
                &lt;/div&gt;
              &lt;/Card&gt;
            ))}
          &lt;/div&gt;
        )}
        {images.length &gt; 0 &amp;&amp; (
          &lt;div className="flex justify-end"&gt;
            &lt;Button variant="ghost" size="sm" className="text-destructive" onClick={() =&gt; {
if (confirm("Clear all generated images from this session?")) clear(); }}&gt;Clear session&lt;/
Button&gt;
          &lt;/div&gt;
        )}
      &lt;/div&gt;
    &lt;/div&gt;
  );
}
```

src/components/jarvis/views/memory.tsx

```
"use client";

import { useState } from "react";
import { useQueryClient, useQuery, useMutation } from "@tanstack/react-query";
import { Card } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Textarea } from "@/components/ui/textarea";
import { Label } from "@/components/ui/label";
import { Skeleton } from "@/components/ui/skeleton";
import { Brain, Plus, Trash2, ShieldCheck, Pencil } from "lucide-react";
import { toast } from "sonner";
import { useAuth } from "@/components/providers/auth-provider";
import { useSettings } from "@/hooks/use-settings";
import type { MemoryEntry } from "@/lib/types";

export function MemoryView() {
  const qc = useQueryClient();
  const { settings } = useSettings();
  const memoryEnabled = settings?.memoryEnabled !== false;
  const [editing, setEditing] = useState&lt;MemoryEntry | null&gt;(null);
  const [key, setKey] = useState("");
  const [value, setValue] = useState("");
  const [showForm, setShowForm] = useState(false);

  const q = useQuery&lt;{ memory: MemoryEntry[] }&gt;({
    queryKey: ["memory"],
    queryFn: async () =&gt; {
      const r = await fetch("/api/memory", { cache: "no-store" });
      const j = await r.json();
      if (!j.ok) throw new Error(j.error);
      return j;
    },
    enabled: memoryEnabled,
  });

  const save = useMutation({
    mutationFn: async (payload: { key: string; value: string }) =&gt; {
      const r = await fetch("/api/memory", { method: "POST", headers: { "Content-Type":
"application/json" }, body: JSON.stringify(payload) });
      const j = await r.json();
      if (!j.ok) throw new Error(j.error);
      return j.entry as MemoryEntry;
    },
    onSuccess: () =&gt; { qc.invalidateQueries({ queryKey: ["memory"] }); setKey(""); setValue("");
setEditing(null); setShowForm(false); toast.success("Saved."); },
  });

  const remove = useMutation({
```

```
    mutationFn: async (id: string) =&gt; {
      const r = await fetch(`/api/memory/${id}`, { method: "DELETE" });
      const j = await r.json();
      if (!j.ok) throw new Error(j.error);
    },
    onSuccess: () =&gt; { qc.invalidateQueries({ queryKey: ["memory"] }); toast.success("Deleted.");
 },
  });

  const clearAll = useMutation({
    mutationFn: async () =&gt; {
      const r = await fetch("/api/memory", { method: "DELETE" });
      const j = await r.json();
      if (!j.ok) throw new Error(j.error);
    },
    onSuccess: () =&gt; { qc.invalidateQueries({ queryKey: ["memory"] }); toast.success("All memory
cleared."); },
  });

  const memory = q.data?.memory ?? [];

  return (
    &lt;div className="h-full overflow-y-auto jarvis-scrollbar"&gt;
      &lt;div className="max-w-3xl mx-auto px-4 sm:px-6 py-6 grid gap-4"&gt;
        &lt;div className="flex items-center gap-3"&gt;
          &lt;div className="flex-1"&gt;
            &lt;h2 className="text-xl font-semibold flex items-center gap-2"&gt;&lt;Brain
className="size-5 text-primary" /&gt; Memory&lt;/h2&gt;
            &lt;p className="text-sm text-muted-foreground"&gt;Preferences JARVIS remembers across
conversations. You’re in full control.&lt;/p&gt;
          &lt;/div&gt;
          {!memoryEnabled ? (
            &lt;Button variant="outline" size="sm" onClick={async () =&gt; { await fetch("/api/
settings", { method: "PATCH", headers: { "Content-Type": "application/json" }, body: JSON.stringify(
{ memoryEnabled: true }) }); qc.invalidateQueries({ queryKey: ["settings"] }); }}&gt;Enable
memory&lt;/Button&gt;
          ) : (
            &lt;Button size="sm" onClick={() =&gt; { setEditing(null); setKey(""); setValue("");
setShowForm(true); }}&gt;&lt;Plus className="size-4" /&gt; Add&lt;/Button&gt;
          )}
        &lt;/div&gt;

        {!memoryEnabled ? (
          &lt;Card className="jarvis-panel p-6 text-center"&gt;
            &lt;ShieldCheck className="size-8 mx-auto text-emerald-400 mb-2" /&gt;
            &lt;p className="text-sm text-muted-foreground"&gt;Memory is disabled. JARVIS won’t
persist any preferences between sessions. You can enable it any time.&lt;/p&gt;
          &lt;/Card&gt;
        ) : showForm || editing ? (
          &lt;Card className="jarvis-panel p-4"&gt;
            &lt;h3 className="font-semibold text-sm mb-3"&gt;{editing ? "Edit preference" : "New
preference"}&lt;/h3&gt;
            &lt;div className="grid gap-3"&gt;
              &lt;div className="grid gap-1.5"&gt;
                &lt;Label htmlFor="m-key"&gt;Key&lt;/Label&gt;
                &lt;Input id="m-key" value={editing ? editing.key : key} onChange={(e) =&gt;
editing ? setEditing({ ...editing, key: e.target.value }) : setKey(e.target.value)} placeholder="e.
g. preferred name, timezone, dietary" /&gt;
              &lt;/div&gt;
              &lt;div className="grid gap-1.5"&gt;
                &lt;Label htmlFor="m-val"&gt;Value&lt;/Label&gt;
                &lt;Textarea id="m-val" value={editing ? editing.value : value} onChange={(e) =&gt;
editing ? setEditing({ ...editing, value: e.target.value }) : setValue(e.target.value)} rows={3} /
&gt;
              &lt;/div&gt;
              &lt;div className="flex justify-end gap-2"&gt;
                &lt;Button variant="ghost" size="sm" onClick={() =&gt; { setShowForm(false);
setEditing(null); }}&gt;Cancel&lt;/Button&gt;
                &lt;Button size="sm" onClick={() =&gt; save.mutate({ key: editing?.key ?? key,
value: editing?.value ?? value })} disabled={save.isPending}&gt;Save&lt;/Button&gt;
              &lt;/div&gt;
            &lt;/div&gt;
          &lt;/Card&gt;
        ) : q.isLoading ? (
          &lt;div className="grid gap-2"&gt;{Array.from({ length: 3 }).map((_, i) =&gt; &lt;
Skeleton key={i} className="h-16" /&gt;)}&lt;/div&gt;
        ) : memory.length === 0 ? (
          &lt;Card className="jarvis-panel p-6 text-center"&gt;
            &lt;Brain className="size-8 mx-auto text-muted-foreground mb-2" /&gt;
            &lt;p className="text-sm text-muted-foreground"&gt;No saved preferences yet. Add one,
or ask JARVIS in chat to remember something — it will always ask first.&lt;/p&gt;
```

```
          &lt;/Card&gt;
        ) : (
          &lt;div className="grid gap-2"&gt;
            {memory.map((m) =&gt; (
              &lt;Card key={m.id} className="jarvis-panel p-3 flex items-start gap-3"&gt;
                &lt;div className="flex-1 min-w-0"&gt;
                  &lt;div className="text-sm font-medium"&gt;{m.key}&lt;/div&gt;
                  &lt;div className="text-sm text-muted-foreground whitespace-pre-wrap break-
words"&gt;{m.value}&lt;/div&gt;
                  &lt;div className="text-[10px] text-muted-foreground mt-1"&gt;Updated {new Date(m.
updatedAt).toLocaleString()}&lt;/div&gt;
                &lt;/div&gt;
                &lt;Button size="icon" variant="ghost" className="size-8" onClick={() =&gt;
setEditing(m)} aria-label="Edit"&gt;&lt;Pencil className="size-3.5" /&gt;&lt;/Button&gt;
                &lt;Button size="icon" variant="ghost" className="size-8 text-destructive" onClick={
() =&gt; { if (confirm("Delete this preference?")) remove.mutate(m.id); }} aria-label="Delete"&gt;
&lt;Trash2 className="size-3.5" /&gt;&lt;/Button&gt;
              &lt;/Card&gt;
            ))}
            &lt;div className="flex justify-end pt-2"&gt;
              &lt;Button variant="ghost" size="sm" className="text-destructive" onClick={() =&gt; {
if (confirm("Delete ALL saved preferences? This cannot be undone.")) clearAll.mutate(); }}&gt;Clear
all memory&lt;/Button&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        )}

        &lt;Card className="jarvis-panel p-4"&gt;
          &lt;div className="flex items-start gap-2"&gt;
            &lt;ShieldCheck className="size-4 text-primary mt-0.5" /&gt;
            &lt;div className="text-xs text-muted-foreground"&gt;
              &lt;p className="font-medium text-foreground mb-1"&gt;How JARVIS uses memory&lt;/p&gt;
              &lt;ul className="list-disc pl-4 space-y-1"&gt;
                &lt;li&gt;Only entries you explicitly save (or approve in chat) are stored.&lt;/
li&gt;
                &lt;li&gt;JARVIS never infers sensitive attributes (health, finances, identity
documents).&lt;/li&gt;
                &lt;li&gt;Memory is scoped to your account only — never shared across users.&lt;/
li&gt;
                &lt;li&gt;Conversation context is temporary; this list is the persistent layer.&lt;/
li&gt;
              &lt;/ul&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/Card&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
}
```

src/components/jarvis/views/settings.tsx

```
"use client";

import { useEffect, useState } from "react";
import { Card } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Switch } from "@/components/ui/switch";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/
select";
import { Slider } from "@/components/ui/slider";
import { Skeleton } from "@/components/ui/skeleton";
import { Settings as SettingsIcon, User, Cpu, Volume2, Palette, ShieldCheck, Trash2, RefreshCw,
CheckCircle2, XCircle, Loader2 } from "lucide-react";
import { toast } from "sonner";
import { useSettings } from "@/hooks/use-settings";
import { useAuth } from "@/components/providers/auth-provider";
import { useQuery } from "@tanstack/react-query";
import type { ConnectionStatus } from "@/lib/types";

export function SettingsView({ micGranted }: { micGranted: "unknown" | "granted" | "denied" |
"unsupported" }) {
  const { settings, models, update, isLoading } = useSettings();
  const { user } = useAuth();
  const [displayName, setDisplayName] = useState("");
  const [saving, setSaving] = useState&lt;string | null&gt;(null);
  useEffect(() =&gt; { if (settings) setDisplayName(settings.displayName); }, [settings]);
```

```
  const status = useQuery&lt;{ connections: ConnectionStatus[] }&gt;({
    queryKey: ["status"],
    queryFn: async () =&gt; {
      const r = await fetch("/api/status", { cache: "no-store" });
      const j = await r.json();
      if (!j.ok) throw new Error(j.error);
      return j;
    },
  });

  async function patch(key: string, value: any) {
    setSaving(key);
    try {
      await update({ [key]: value });
      toast.success("Saved.");
    } catch (e) {
      toast.error(e instanceof Error ? e.message : "Failed to save.");
    } finally {
      setSaving(null);
    }
  }

  if (isLoading || !settings) {
    return (
      &lt;div className="h-full overflow-y-auto jarvis-scrollbar p-6 max-w-3xl mx-auto grid gap-
3"&gt;
        {Array.from({ length: 5 }).map((_, i) =&gt; &lt;Skeleton key={i} className="h-28" /&gt;)}
      &lt;/div&gt;
    );
  }

  return (
    &lt;div className="h-full overflow-y-auto jarvis-scrollbar"&gt;
      &lt;div className="max-w-3xl mx-auto px-4 sm:px-6 py-6 grid gap-4"&gt;
        &lt;div&gt;
          &lt;h2 className="text-xl font-semibold flex items-center gap-2"&gt;&lt;SettingsIcon
className="size-5 text-primary" /&gt; Settings&lt;/h2&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;Personalize JARVIS. Changes apply
immediately.&lt;/p&gt;
        &lt;/div&gt;

        {/* Profile */}
        &lt;Section icon={User} title="Profile"&gt;
          &lt;div className="grid gap-3"&gt;
            &lt;div className="grid gap-1.5"&gt;
              &lt;Label htmlFor="dn"&gt;Display name&lt;/Label&gt;
              &lt;div className="flex gap-2"&gt;
                &lt;Input id="dn" value={displayName} onChange={(e) =&gt; setDisplayName(e.target.
value)} /&gt;
                &lt;Button onClick={async () =&gt; { await update({ displayName }); toast.success(
"Name updated."); }}&gt;Save&lt;/Button&gt;
              &lt;/div&gt;
            &lt;/div&gt;
            &lt;div className="grid gap-1.5"&gt;
              &lt;Label&gt;Email&lt;/Label&gt;
              &lt;Input value={user?.email ?? ""} disabled className="bg-muted/40" /&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/Section&gt;

        {/* AI */}
        &lt;Section icon={Cpu} title="AI brain"&gt;
          &lt;div className="grid gap-3"&gt;
            &lt;div className="grid gap-1.5"&gt;
              &lt;Label&gt;Model&lt;/Label&gt;
              &lt;Select value={settings.model} onValueChange={(v) =&gt; patch("model", v)}&gt;
                &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
                &lt;SelectContent&gt;
                  {models?.map((m) =&gt; &lt;SelectItem key={m.id} value={m.id}&gt;{m.label}&lt;/
SelectItem&gt;)}
                &lt;/SelectContent&gt;
              &lt;/Select&gt;
              &lt;p className="text-[11px] text-muted-foreground"&gt;Availability depends on the
configured provider. If a model is unavailable the API will surface a real error.&lt;/p&gt;
            &lt;/div&gt;
            &lt;div className="grid gap-1.5"&gt;
              &lt;Label&gt;Response style&lt;/Label&gt;
              &lt;Select value={settings.responseStyle} onValueChange={(v) =&gt; patch(
"responseStyle", v)}&gt;
                &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
                &lt;SelectContent&gt;
```

```
                  &lt;SelectItem value="concise"&gt;Concise&lt;/SelectItem&gt;
                  &lt;SelectItem value="balanced"&gt;Balanced&lt;/SelectItem&gt;
                  &lt;SelectItem value="detailed"&gt;Detailed&lt;/SelectItem&gt;
                &lt;/SelectContent&gt;
              &lt;/Select&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/Section&gt;

        {/* Voice */}
        &lt;Section icon={Volume2} title="Voice"&gt;
          &lt;div className="grid gap-3"&gt;
            &lt;Row label="Enable voice (spoken answers)" desc="JARVIS will read replies aloud in
the Voice view."&gt;
              &lt;Switch checked={settings.voiceEnabled} onCheckedChange={(v) =&gt; patch(
"voiceEnabled", v)} /&gt;
            &lt;/Row&gt;
            &lt;div className="grid gap-1.5"&gt;
              &lt;Label&gt;Speech rate&lt;/Label&gt;
              &lt;div className="flex items-center gap-3"&gt;
                &lt;Slider value={[settings.ttsRate]} min={0.5} max={2} step={0.05} onValueChange={(
v) =&gt; patch("ttsRate", v[0])} className="flex-1" /&gt;
                &lt;span className="text-xs text-muted-foreground w-12 text-right"&gt;{settings.
ttsRate.toFixed(2)}×&lt;/span&gt;
              &lt;/div&gt;
            &lt;/div&gt;
            &lt;div className="text-xs text-muted-foreground border border-border/60 rounded-lg p-
3"&gt;
              &lt;div className="flex items-center gap-2 mb-1"&gt;
                &lt;MicStatus granted={micGranted} /&gt;
                &lt;span&gt;Microphone permission: {micGranted}&lt;/span&gt;
              &lt;/div&gt;
              Listening is push-to-talk only — JARVIS never records audio in the background and
stores no recordings.
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/Section&gt;

        {/* Appearance */}
        &lt;Section icon={Palette} title="Appearance"&gt;
          &lt;div className="grid gap-3"&gt;
            &lt;Row label="Theme" desc="Dark is the canonical JARVIS look."&gt;
              &lt;Select value={settings.theme} onValueChange={(v) =&gt; patch("theme", v)}&gt;
                &lt;SelectTrigger className="w-32"&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
                &lt;SelectContent&gt;
                  &lt;SelectItem value="dark"&gt;Dark&lt;/SelectItem&gt;
                  &lt;SelectItem value="light"&gt;Light&lt;/SelectItem&gt;
                &lt;/SelectContent&gt;
              &lt;/Select&gt;
            &lt;/Row&gt;
            &lt;Row label="Reduce motion" desc="Disable nonessential animations."&gt;
              &lt;Switch checked={settings.reducedMotion} onCheckedChange={(v) =&gt; patch(
"reducedMotion", v)} /&gt;
            &lt;/Row&gt;
          &lt;/div&gt;
        &lt;/Section&gt;

        {/* Memory */}
        &lt;Section icon={ShieldCheck} title="Memory &amp; data"&gt;
          &lt;div className="grid gap-3"&gt;
            &lt;Row label="Enable memory" desc="Let JARVIS remember preferences across sessions.
"&gt;
              &lt;Switch checked={settings.memoryEnabled} onCheckedChange={(v) =&gt; patch(
"memoryEnabled", v)} /&gt;
            &lt;/Row&gt;
            &lt;Row label="Delete all conversations" desc="Permanently removes every chat session
and message."&gt;
              &lt;Button variant="destructive" size="sm" onClick={async () =&gt; {
                if (!confirm("Permanently delete ALL conversations?")) return;
                // No bulk endpoint exists; iterate sessions.
                const r = await fetch("/api/chat/sessions", { cache: "no-store" });
                const j = await r.json();
                for (const c of j.conversations ?? []) {
                  await fetch(`/api/chat/sessions/${c.id}`, { method: "DELETE" });
                }
                toast.success("All conversations deleted.");
              }}&gt;
                &lt;Trash2 className="size-4" /&gt; Delete
              &lt;/Button&gt;
            &lt;/Row&gt;
          &lt;/div&gt;
```

```
        &lt;/Section&gt;

        {/* Connected services */}
        &lt;Section icon={RefreshCw} title="Connected services"&gt;
          &lt;div className="grid gap-2"&gt;
            &lt;div className="flex justify-end"&gt;
              &lt;Button variant="outline" size="sm" onClick={() =&gt; status.refetch()} disabled={
status.isFetching}&gt;
                &lt;RefreshCw className={"size-4 " + (status.isFetching ? "animate-spin" : "")} /
&gt; Re-check
              &lt;/Button&gt;
            &lt;/div&gt;
            {status.data?.connections.map((c) =&gt; &lt;ConnectionRow key={c.id} c={c} /&gt;)}
            {!status.data &amp;&amp; status.isFetching &amp;&amp; &lt;Skeleton className="h-14" /
&gt;}
            &lt;p className="text-[11px] text-muted-foreground pt-1"&gt;
              JARVIS shows the real status of each integration. It never claims a connection that
isn’t actually configured.
            &lt;/p&gt;
          &lt;/div&gt;
        &lt;/Section&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
}

function Section({ icon: Icon, title, children }: { icon: React.ElementType; title: string;
children: React.ReactNode }) {
  return (
    &lt;Card className="jarvis-panel p-5"&gt;
      &lt;div className="flex items-center gap-2

… (truncated for brevity; see the full file in the project repository) …
```

API Routes — Auth & Status

src/app/api/auth/me/route.ts

```
import { NextResponse } from "next/server";
import { ensureDefaultUser } from "@/lib/auth";

// With the sign-in page removed, this endpoint auto-provisions the default
// local Operator account on first contact so the app boots straight into the
// shell. The returned user is used by the AuthProvider on the client.
export async function GET() {
  const user = await ensureDefaultUser();
  return NextResponse.json({ ok: true, user });
}
```

src/app/api/status/route.ts

```
import { NextResponse } from "next/server";
import { getCurrentUser } from "@/lib/auth";
import { pingZai, AVAILABLE_MODELS } from "@/lib/jarvis-ai";
import { isSupabaseConfigured, OPERATOR_ID } from "@/lib/supabase";
import { probeTables } from "@/lib/data";
import type { ConnectionStatus } from "@/lib/types";

// Honest connectivity + capability status. ?probe=1 performs a live z.ai test.
export async function GET(req: Request) {
  const user = await getCurrentUser();
  const probe = new URL(req.url).searchParams.get("probe") === "1";

  let zai: ConnectionStatus = {
    id: "zai",
    name: "z.ai Model (chat, TTS, ASR)",
    description: "Conversational brain via z-ai-web-dev-sdk.",
    status: "connected",
    detail: "Configured via /etc/.z-ai-config (SDK auto-loaded).",
  };
  if (probe) {
    const p = await pingZai();
    zai = { ...zai, status: p.ok ? "connected" : "error", detail: p.ok ? "Responded to a live test
prompt." : p.detail };
  }

  // Supabase: reflect the REAL state of the connection + schema.
  const sbConfigured = isSupabaseConfigured();
  let supabase: ConnectionStatus;
  if (!sbConfigured) {
    supabase = {
      id: "supabase",
      name: "Supabase",
      description: "Database backend (profiles, tasks, notes, projects, conversations, memory,
audit).",
      status: "not_configured",
      detail: "NEXT_PUBLIC_SUPABASE_URL / NEXT_PUBLIC_SUPABASE_ANON_KEY are not set.",
    };
  } else {
    let schemaReady = false;
    let tables: string[] = [];
    try {
      const probeRes = await probeTables();
      schemaReady = probeRes.ready;
      tables = probeRes.tables;
    } catch {
      /* ignore */
    }
    supabase = {
      id: "supabase",
      name: "Supabase",
      description: "Database backend (profiles, tasks, notes, projects, conversations, memory,
audit).",
      status: schemaReady ? "connected" : "error",
      detail: schemaReady
        ? `Project ${process.env.NEXT_PUBLIC_SUPABASE_URL?.replace("https://", "").replace(".
supabase.co", "")} — all 8 tables ready. Operator id ${OPERATOR_ID}.`
        : `Project reachable but schema not applied. Run supabase/schema.sql in the Supabase SQL
editor. (Found ${tables.length}/8 tables.)`,
```

```
    };
  }

  const connections: ConnectionStatus[] = [
    zai,
    supabase,
    {
      id: "browser_stt",
      name: "Browser Speech Recognition",
      description: "Push-to-talk microphone input via Web Speech API.",
      status: "connected",
      detail: "Provided by the browser when supported. Permission requested on first use.",
    },
    {
      id: "browser_tts",
      name: "Browser Speech Synthesis",
      description: "Spoken answers via Web Speech API.",
      status: "connected",
      detail: "Provided by the browser when supported. A z.ai TTS fallback is also available.",
    },
  ];

  return NextResponse.json({ ok: true, user: user ?? null, connections, models: AVAILABLE_MODELS });
}
```

src/app/api/settings/route.ts

```
import { NextResponse } from "next/server";
import { getCurrentUser, requireUser } from "@/lib/auth";
import { loadSettings, saveSetting } from "@/lib/data";
import { AVAILABLE_MODELS } from "@/lib/jarvis-ai";

export async function GET() {
  const user = await getCurrentUser();
  if (!user) return NextResponse.json({ ok: false, error: "Unauthorized" }, { status: 401 });
  try {
    const settings = await loadSettings();
    return NextResponse.json({ ok: true, settings, models: AVAILABLE_MODELS });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to load
settings." }, { status: 500 });
  }
}

export async function PATCH(req: Request) {
  await requireUser();
  let body: any;
  try { body = await req.json(); } catch { return NextResponse.json({ ok: false, error: "Invalid
JSON." }, { status: 400 }); }
  const allowed: Record&lt;string, (v: unknown) =&gt; string&gt; = {
    model: (v) =&gt; {
      const s = String(v);
      if (!AVAILABLE_MODELS.some((m) =&gt; m.id === s)) throw new Error("Unsupported model");
      return s;
    },
    responseStyle: (v) =&gt; ["concise", "balanced", "detailed"].includes(String(v)) ? String(v) :
"balanced",
    voiceEnabled: (v) =&gt; (v ? "true" : "false"),
    ttsVoice: (v) =&gt; String(v).slice(0, 60),
    ttsRate: (v) =&gt; {
      const n = Number(v);
      if (!isFinite(n) || n &lt; 0.5 || n &gt; 2) return "1";
      return String(n);
    },
    theme: (v) =&gt; (String(v) === "light" ? "light" : "dark"),
    reducedMotion: (v) =&gt; (v ? "true" : "false"),
    memoryEnabled: (v) =&gt; (v ? "true" : "false"),
  };

  if (body.displayName) {
    await saveSetting("displayName", String(body.displayName).slice(0, 60) || "Operator");
  }
  const updates = body.settings ?? body;
  for (const [key, raw] of Object.entries(updates ?? {})) {
    if (key in allowed) {
      try {
        const value = allowed[key](raw);
        await saveSetting(key, value);
      } catch (e) {
```

```
        return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Invalid
setting" }, { status: 400 });
      }
    }
  }
  try {
    const settings = await loadSettings();
    return NextResponse.json({ ok: true, settings, models: AVAILABLE_MODELS });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to reload
settings." }, { status: 500 });
  }
}
```

API Routes — Chat (streaming z.ai)

src/app/api/chat/sessions/route.ts

```
import { NextResponse } from "next/server";
import { getCurrentUser, requireUser } from "@/lib/auth";
import { listConversations, createConversation } from "@/lib/data";

export async function GET() {
  const user = await getCurrentUser();
  if (!user) return NextResponse.json({ ok: false, error: "Unauthorized" }, { status: 401 });
  try {
    const conversations = await listConversations();
    return NextResponse.json({ ok: true, conversations });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to load
conversations." }, { status: 500 });
  }
}

export async function POST(req: Request) {
  await requireUser();
  let body: any = {};
  try { body = await req.json(); } catch { /* empty body is fine */ }
  const title = (body.title &amp;&amp; String(body.title)) || "New conversation";
  try {
    const conversation = await createConversation(title);
    return NextResponse.json({ ok: true, conversation });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to create
conversation." }, { status: 500 });
  }
}
```

src/app/api/chat/sessions/[id]/route.ts

```
import { NextResponse } from "next/server";
import { requireUser } from "@/lib/auth";
import { renameConversation, deleteConversation, getConversation } from "@/lib/data";

export async function PATCH(req: Request, { params }: { params: Promise&lt;{ id: string }&gt; }) {
  await requireUser();
  const { id } = await params;
  let body: any;
  try { body = await req.json(); } catch { return NextResponse.json({ ok: false, error: "Invalid
JSON." }, { status: 400 }); }
  const title = body.title ? String(body.title).slice(0, 120) : undefined;
  try {
    const conv = await renameConversation(id, title ?? "Untitled");
    if (!conv) return NextResponse.json({ ok: false, error: "Not found" }, { status: 404 });
    return NextResponse.json({ ok: true, conversation: conv });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to rename.
" }, { status: 500 });
  }
}

export async function DELETE(_req: Request, { params }: { params: Promise&lt;{ id: string }&gt; }) {
  await requireUser();
  const { id } = await params;
  const conv = await getConversation(id);
  if (!conv) return NextResponse.json({ ok: false, error: "Not found" }, { status: 404 });
  try {
    await deleteConversation(id);
    return NextResponse.json({ ok: true });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to delete.
" }, { status: 500 });
  }
}
```

src/app/api/chat/sessions/[id]/messages/route.ts

```
import { NextResponse } from "next/server";
import { getCurrentUser } from "@/lib/auth";
import { getConversation, listMessages } from "@/lib/data";
import type { ChatMessage } from "@/lib/types";

export async function GET(_req: Request, { params }: { params: Promise&lt;{ id: string }&gt; }) {
  const user = await getCurrentUser();
  if (!user) return NextResponse.json({ ok: false, error: "Unauthorized" }, { status: 401 });
  const { id } = await params;
  try {
    const conv = await getConversation(id);
    if (!conv) return NextResponse.json({ ok: false, error: "Not found" }, { status: 404 });
    const messages: ChatMessage[] = await listMessages(id);
    return NextResponse.json({ ok: true, conversationId: conv.id, title: conv.title, messages });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to load
messages." }, { status: 500 });
  }
}
```

src/app/api/chat/send/route.ts

```
import { requireUser } from "@/lib/auth";
import { streamChatCompletion } from "@/lib/jarvis-ai";
import { buildSystemPrompt } from "@/lib/prompt";
import { parseToolCalls } from "@/lib/prompt";
import type { ZaiChatMessage } from "@/lib/jarvis-ai";
import { createConversation, addMessage, listMessages, touchConversation, getConversation } from "@/
lib/data";

// POST /api/chat/send  (Server-Sent Events stream)
// Body: { conversationId?: string, message: string }
export async function POST(req: Request) {
  await requireUser();
  let body: any;
  try { body = await req.json(); } catch {
    return new Response(JSON.stringify({ ok: false, error: "Invalid JSON." }), { status: 400,
headers: { "Content-Type": "application/json" } });
  }
  const { conversationId, message } = body ?? {};
  const userText = typeof message === "string" ? message.slice(0, 8000) : "";
  if (!userText.trim()) {
    return new Response(JSON.stringify({ ok: false, error: "Empty message." }), { status: 400,
headers: { "Content-Type": "application/json" } });
  }

  // Load or create the conversation.
  let convId = typeof conversationId === "string" ? conversationId : null;
  if (convId) {
    const owned = await getConversation(convId);
    if (!owned) convId = null;
  }
  if (!convId) {
    const title = userText.slice(0, 50).replace(/\s+/g, " ").trim() || "New conversation";
    try {
      const conv = await createConversation(title);
      convId = conv.id;
    } catch (e) {
      const msg = e instanceof Error ? e.message : "Failed to start conversation.";
      return new Response(JSON.stringify({ ok: false, error: msg }), { status: 500, headers: {
"Content-Type": "application/json" } });
    }
  }

  // Persist the user's message.
  try {
    await addMessage(convId, "user", userText);
  } catch (e) {
    const msg = e instanceof Error ? e.message : "Failed to persist message.";
    return new Response(JSON.stringify({ ok: false, error: msg }), { status: 500, headers: {
"Content-Type": "application/json" } });
  }

  // Load recent history for context (last ~12 messages).
  const history = await listMessages(convId);
  const recent = history.slice(-13);
  const { system, model } = await buildSystemPrompt("operator");

  const msgs: ZaiChatMessage[] = [{ role: "system", content: system }];
```

```
  for (const m of recent) {
    msgs.push({ role: m.role as "user" | "assistant", content: m.content });
  }
  if (msgs[msgs.length - 1].role !== "user" || msgs[msgs.length - 1].content !== userText) {
    msgs.push({ role: "user", content: userText });
  }

  const abort = new AbortController();
  req.signal?.addEventListener("abort", () =&gt; abort.abort());

  const encoder = new TextEncoder();
  const stream = new ReadableStream({
    async start(controller) {
      const send = (obj: Record&lt;string, unknown&gt;) =&gt; {
        controller.enqueue(encoder.encode(`data: ${JSON.stringify(obj)}\n\n`));
      };
      send({ type: "conv", conversationId: convId });

      let full = "";
      let started = 0;
      try {
        await streamChatCompletion(msgs, { model, temperature: 0.7 }, {
          onDelta(delta) {
            full += delta;
            const { prose, calls } = parseToolCalls(full);
            if (calls.length) {
              for (const c of calls) send({ type: "tool", tool: c.tool, input: c.input });
              if (prose &amp;&amp; prose.length &gt; started) {
                send({ type: "delta", text: prose.slice(started) });
                started = prose.length;
              }
              full = "";
            } else {
              if (full.length &gt; started) {
                let safe = full;
                const tagIdx = full.lastIndexOf("&lt;&lt;&lt;TOOL_CALL");
                if (tagIdx &gt;= 0 &amp;&amp; tagIdx &gt;= started) safe = full.slice(0, tagIdx);
                if (safe.length &gt; started) {
                  send({ type: "delta", text: safe.slice(started) });
                  started = safe.length;
                }
              }
            }
          },
          onDone() {
            const { prose, calls } = parseToolCalls(full);
            for (const c of calls) send({ type: "tool", tool: c.tool, input: c.input });
            const finalProse = prose.length &gt; started ? prose.slice(started) : "";
            if (finalProse) send({ type: "delta", text: finalProse });
            void addMessage(convId!, "assistant", prose).then(() =&gt; {
              void touchConversation(convId!);
            });
            send({ type: "done" });
            try { controller.close(); } catch { /* already closed */ }
          },
          onError(err) {
            send({ type: "error", message: err.message });
            try { controller.close(); } catch { /* noop */ }
          },
          signal: abort.signal,
        });
      } catch (e) {
        const msg = e instanceof Error ? e.message : String(e);
        send({ type: "error", message: msg });
        try { controller.close(); } catch { /* noop */ }
      }
    },
    cancel() {
      abort.abort();
    },
  });

  return new Response(stream, {
    headers: {
      "Content-Type": "text/event-stream; charset=utf-8",
      "Cache-Control": "no-cache, no-transform",
      Connection: "keep-alive",
    },
  });
}
```

API Routes — Tasks / Notes / Projects

src/app/api/tasks/route.ts

```
import { NextResponse } from "next/server";
import { getCurrentUser, requireUser } from "@/lib/auth";
import { listTasks, createTask } from "@/lib/data";

export async function GET(req: Request) {
  const user = await getCurrentUser();
  if (!user) return NextResponse.json({ ok: false, error: "Unauthorized" }, { status: 401 });
  const url = new URL(req.url);
  const view = url.searchParams.get("view") ?? "all";
  const q = url.searchParams.get("q") ?? "";
  const projectId = url.searchParams.get("projectId") ?? undefined;
  try {
    const tasks = await listTasks({ view, q: q || undefined, projectId });
    return NextResponse.json({ ok: true, tasks });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to load
tasks." }, { status: 500 });
  }
}

export async function POST(req: Request) {
  await requireUser();
  let body: any;
  try { body = await req.json(); } catch { return NextResponse.json({ ok: false, error: "Invalid
JSON." }, { status: 400 }); }
  const { title, description, priority, status, category, dueDate, projectId } = body ?? {};
  if (!title || typeof title !== "string") {
    return NextResponse.json({ ok: false, error: "Title is required." }, { status: 400 });
  }
  try {
    const task = await createTask({
      title, description, priority, status, category, dueDate, projectId,
    });
    return NextResponse.json({ ok: true, task });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to create
task." }, { status: 500 });
  }
}
```

src/app/api/tasks/[id]/route.ts

```
import { NextResponse } from "next/server";
import { getCurrentUser, requireUser } from "@/lib/auth";
import { getTask, updateTask, deleteTask } from "@/lib/data";

export async function GET(_req: Request, { params }: { params: Promise&lt;{ id: string }&gt; }) {
  const user = await getCurrentUser();
  if (!user) return NextResponse.json({ ok: false, error: "Unauthorized" }, { status: 401 });
  const { id } = await params;
  try {
    const task = await getTask(id);
    if (!task) return NextResponse.json({ ok: false, error: "Not found" }, { status: 404 });
    return NextResponse.json({ ok: true, task });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to load
task." }, { status: 500 });
  }
}

export async function PATCH(req: Request, { params }: { params: Promise&lt;{ id: string }&gt; }) {
  await requireUser();
  const { id } = await params;
  let body: any;
  try { body = await req.json(); } catch { return NextResponse.json({ ok: false, error: "Invalid
JSON." }, { status: 400 }); }
  try {
    const task = await updateTask(id, body ?? {});
    if (!task) return NextResponse.json({ ok: false, error: "Not found" }, { status: 404 });
    return NextResponse.json({ ok: true, task });
  } catch (e) {
```

```
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to update
task." }, { status: 500 });
  }
}

export async function DELETE(_req: Request, { params }: { params: Promise&lt;{ id: string }&gt; }) {
  await requireUser();
  const { id } = await params;
  try {
    await deleteTask(id);
    return NextResponse.json({ ok: true });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to delete
task." }, { status: 500 });
  }
}
```

src/app/api/notes/route.ts

```
import { NextResponse } from "next/server";
import { getCurrentUser, requireUser } from "@/lib/auth";
import { listNotes, createNote } from "@/lib/data";

export async function GET(req: Request) {
  const user = await getCurrentUser();
  if (!user) return NextResponse.json({ ok: false, error: "Unauthorized" }, { status: 401 });
  const url = new URL(req.url);
  const q = url.searchParams.get("q") ?? "";
  const projectId = url.searchParams.get("projectId") ?? undefined;
  try {
    const notes = await listNotes({ q: q || undefined, projectId });
    return NextResponse.json({ ok: true, notes });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to load
notes." }, { status: 500 });
  }
}

export async function POST(req: Request) {
  await requireUser();
  let body: any;
  try { body = await req.json(); } catch { return NextResponse.json({ ok: false, error: "Invalid
JSON." }, { status: 400 }); }
  const { title, content, projectId } = body ?? {};
  if (!title || typeof title !== "string") {
    return NextResponse.json({ ok: false, error: "Title is required." }, { status: 400 });
  }
  try {
    const note = await createNote({ title, content, projectId });
    return NextResponse.json({ ok: true, note });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to create
note." }, { status: 500 });
  }
}
```

src/app/api/notes/[id]/route.ts

```
import { NextResponse } from "next/server";
import { getCurrentUser, requireUser } from "@/lib/auth";
import { getNote, updateNote, deleteNote } from "@/lib/data";

export async function GET(_req: Request, { params }: { params: Promise&lt;{ id: string }&gt; }) {
  const user = await getCurrentUser();
  if (!user) return NextResponse.json({ ok: false, error: "Unauthorized" }, { status: 401 });
  const { id } = await params;
  try {
    const note = await getNote(id);
    if (!note) return NextResponse.json({ ok: false, error: "Not found" }, { status: 404 });
    return NextResponse.json({ ok: true, note });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to load
note." }, { status: 500 });
  }
}

export async function PATCH(req: Request, { params }: { params: Promise&lt;{ id: string }&gt; }) {
```

```
  await requireUser();
  const { id } = await params;
  let body: any;
  try { body = await req.json(); } catch { return NextResponse.json({ ok: false, error: "Invalid
JSON." }, { status: 400 }); }
  try {
    const note = await updateNote(id, body ?? {});
    if (!note) return NextResponse.json({ ok: false, error: "Not found" }, { status: 404 });
    return NextResponse.json({ ok: true, note });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to update
note." }, { status: 500 });
  }
}

export async function DELETE(_req: Request, { params }: { params: Promise&lt;{ id: string }&gt; }) {
  await requireUser();
  const { id } = await params;
  try {
    await deleteNote(id);
    return NextResponse.json({ ok: true });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to delete
note." }, { status: 500 });
  }
}
```

src/app/api/notes/[id]/summarize/route.ts

```
import { NextResponse } from "next/server";
import { requireUser } from "@/lib/auth";
import { getNote } from "@/lib/data";
import { chatCompletion } from "@/lib/jarvis-ai";

export async function POST(_req: Request, { params }: { params: Promise&lt;{ id: string }&gt; }) {
  await requireUser();
  const { id } = await params;
  try {
    const note = await getNote(id);
    if (!note) return NextResponse.json({ ok: false, error: "Not found" }, { status: 404 });
    if (!note.content.trim()) return NextResponse.json({ ok: false, error: "Note is empty." }, {
status: 400 });
    const summary = await chatCompletion(
      [
        { role: "system", content: "You are JARVIS. Summarize the user's note in 3-5 concise bullet
points in markdown. Preserve key facts." },
        { role: "user", content: `Title: ${note.title}\n\n${note.content}` },
      ],
      { maxTokens: 800 }
    );
    return NextResponse.json({ ok: true, summary });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Summarization
failed." }, { status: 502 });
  }
}
```

src/app/api/projects/route.ts

```
import { NextResponse } from "next/server";
import { getCurrentUser, requireUser } from "@/lib/auth";
import { listProjects, createProject } from "@/lib/data";

export async function GET(req: Request) {
  const user = await getCurrentUser();
  if (!user) return NextResponse.json({ ok: false, error: "Unauthorized" }, { status: 401 });
  const url = new URL(req.url);
  const q = url.searchParams.get("q") ?? "";
  try {
    const projects = await listProjects({ q: q || undefined });
    return NextResponse.json({ ok: true, projects });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to load
projects." }, { status: 500 });
  }
}
export async function POST(req: Request) {
```

```
  await requireUser();
  let body: any;
  try { body = await req.json(); } catch { return NextResponse.json({ ok: false, error: "Invalid
JSON." }, { status: 400 }); }
  const { name, description, goal } = body ?? {};
  if (!name || typeof name !== "string") return NextResponse.json({ ok: false, error: "Name is
required." }, { status: 400 });
  try {
    const project = await createProject({ name, description, goal });
    return NextResponse.json({ ok: true, project });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to create
project." }, { status: 500 });
  }
}
```

src/app/api/projects/[id]/route.ts

```
import { NextResponse } from "next/server";
import { getCurrentUser, requireUser } from "@/lib/auth";
import { getProject, updateProject, deleteProject } from "@/lib/data";

export async function GET(_req: Request, { params }: { params: Promise&lt;{ id: string }&gt; }) {
  const user = await getCurrentUser();
  if (!user) return NextResponse.json({ ok: false, error: "Unauthorized" }, { status: 401 });
  const { id } = await params;
  try {
    const project = await getProject(id);
    if (!project) return NextResponse.json({ ok: false, error: "Not found" }, { status: 404 });
    return NextResponse.json({ ok: true, project });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to load
project." }, { status: 500 });
  }
}

export async function PATCH(req: Request, { params }: { params: Promise&lt;{ id: string }&gt; }) {
  await requireUser();
  const { id } = await params;
  let body: any;
  try { body = await req.json(); } catch { return NextResponse.json({ ok: false, error: "Invalid
JSON." }, { status: 400 }); }
  try {
    const project = await updateProject(id, body ?? {});
    if (!project) return NextResponse.json({ ok: false, error: "Not found" }, { status: 404 });
    return NextResponse.json({ ok: true, project });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to update
project." }, { status: 500 });
  }
}

export async function DELETE(_req: Request, { params }: { params: Promise&lt;{ id: string }&gt; }) {
  await requireUser();
  const { id } = await params;
  try {
    await deleteProject(id);
    return NextResponse.json({ ok: true });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to delete
project." }, { status: 500 });
  }
}
```

src/app/api/projects/[id]/plan/route.ts

```
import { NextResponse } from "next/server";
import { requireUser } from "@/lib/auth";
import { getProject } from "@/lib/data";
import { chatCompletion } from "@/lib/jarvis-ai";

export async function POST(_req: Request, { params }: { params: Promise&lt;{ id: string }&gt; }) {
  await requireUser();
  const { id } = await params;
  try {
    const project = await getProject(id);
    if (!project) return NextResponse.json({ ok: false, error: "Not found" }, { status: 404 });
```

```
    const taskLines = project.tasks.length
      ? project.tasks.map((t) =&gt; `- [${t.status}] ${t.title}${t.dueDate ? ` (due ${t.dueDate.
slice(0, 10)})` : ""}`).join("\n")
      : "(no tasks yet)";
    const plan = await chatCompletion(
      [
        {
          role: "system",
          content:
            "You are JARVIS. The user wants a milestone plan for a project. Produce a concrete,
ordered plan in markdown. Suggest 3-6 milestones. For each, give a short name, a one-line objective,
 and 1-3 suggested tasks. Mark which tasks already exist. Do not invent external deadlines or
people. End with a one-line 'Next step' suggestion. This is a draft — do not claim it has been
saved.",
        },
        {
          role: "user",
          content:
            `Project: ${project.name}\nGoal: ${project.goal || "(not set)"}\nDescription: ${project.
description || "(none)"}\n\nExisting tasks:\n${taskLines}`,
        },
      ],
      { maxTokens: 1200 }
    );
    return NextResponse.json({ ok: true, plan });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Planning failed.
" }, { status: 502 });
  }
}
```

API Routes — Memory / Briefing / Tools

src/app/api/memory/route.ts

```
import { NextResponse } from "next/server";
import { getCurrentUser, requireUser } from "@/lib/auth";
import { getMemory, saveMemory as saveMemoryRow, clearMemory } from "@/lib/data";

export async function GET() {
  const user = await getCurrentUser();
  if (!user) return NextResponse.json({ ok: false, error: "Unauthorized" }, { status: 401 });
  try {
    const memory = await getMemory();
    return NextResponse.json({ ok: true, memory });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to load
memory." }, { status: 500 });
  }
}

export async function POST(req: Request) {
  await requireUser();
  let body: any;
  try { body = await req.json(); } catch { return NextResponse.json({ ok: false, error: "Invalid
JSON." }, { status: 400 }); }
  const { key, value } = body ?? {};
  try {
    const k = String(key ?? "").trim().slice(0, 80);
    const v = String(value ?? "").trim().slice(0, 2000);
    if (!k || !v) return NextResponse.json({ ok: false, error: "Key and value are required." }, {
status: 400 });
    const entry = await saveMemoryRow(k, v);
    return NextResponse.json({ ok: true, entry });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to save."
}, { status: 500 });
  }
}

export async function DELETE() {
  await requireUser();
  try {
    await clearMemory();
    return NextResponse.json({ ok: true });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to clear.
" }, { status: 500 });
  }
}
```

src/app/api/memory/[id]/route.ts

```
import { NextResponse } from "next/server";
import { requireUser } from "@/lib/auth";
import { deleteMemory } from "@/lib/data";

export async function DELETE(_req: Request, { params }: { params: Promise&lt;{ id: string }&gt; }) {
  await requireUser();
  const { id } = await params;
  try {
    await deleteMemory(id);
    return NextResponse.json({ ok: true });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to delete.
" }, { status: 500 });
  }
}
```

src/app/api/briefing/route.ts

```
import { NextResponse } from "next/server";
import { getCurrentUser } from "@/lib/auth";
import { listTasks, countTasks, listProjects } from "@/lib/data";
```

```
import { chatCompletion } from "@/lib/jarvis-ai";
import type { Task } from "@/lib/types";

// GET /api/briefing — daily briefing from the user's REAL stored data.
export async function GET() {
  const user = await getCurrentUser();
  if (!user) return NextResponse.json({ ok: false, error: "Unauthorized" }, { status: 401 });

  const now = new Date();
  const startOfDay = new Date(now); startOfDay.setHours(0, 0, 0, 0);
  const endOfDay = new Date(now); endOfDay.setHours(23, 59, 59, 999);
  const in7 = new Date(now.getTime() + 7 * 86400000);

  try {
    const tasksToday = (await listTasks({ view: "today" })).slice(0, 8);
    const upcoming = (await listTasks({ view: "upcoming" })).slice(0, 8);
    const overdue = (await listTasks({})).filter((t) =&gt; t.status !== "done" &amp;&amp; t.dueDate
&amp;&amp; new Date(t.dueDate) &lt; startOfDay).slice(0, 8);
    const totalOpen = await countTasks({ statusNot: "done" });
    const totalDone = await countTasks({ status: "done" });
    const projects = await listProjects({});

    const projectLines = projects.length
      ? projects.map((p) =&gt; `- ${p.name}: ${p.completedTaskCount}/${p.taskCount} tasks (${p.
progress}%)`).join("\n")
      : "Projects: none";
    const taskLines = (label: string, list: Task[]) =&gt;
      list.length ? `${label}:\n` + list.map((t) =&gt; `- ${t.title}${t.priority === "high" ? " (
high)" : ""}`).join("\n") : `${label}: none`;
    const summary = [
      taskLines("Today", tasksToday),
      taskLines("Upcoming (7d)", upcoming),
      taskLines("Overdue", overdue),
      `Open tasks: ${totalOpen} • Completed: ${totalDone}`,
      projectLines,
    ].join("\n\n");

    let narrative: string | null = null;
    let narrativeError: string | null = null;
    try {
      narrative = await chatCompletion(
        [
          {
            role: "system",
            content:
              "You are JARVIS giving the user a concise, motivating daily briefing based ONLY on
the real task/project data provided. Do not invent calendar events, news, weather, or any data not
present. If a section is empty, say so briefly. Keep it under 120 words. Use a calm, capable tone.",
          },
          { role: "user", content: `User display name: ${user.displayName}\n\n${summary}` },
        ],
        { maxTokens: 500 }
      );
    } catch (e) {
      narrativeError = e instanceof Error ? e.message : String(e);
    }

    return NextResponse.json({
      ok: true,
      briefing: {
        greeting: `Good ${hourGreeting(now)}, ${user.displayName}.`,
        narrative,
        narrativeError,
        counts: { totalOpen, totalDone, overdue: overdue.length, today: tasksToday.length, upcoming:
 upcoming.length },
        tasksToday: tasksToday.map(serializeTask),
        upcoming: upcoming.map(serializeTask),
        overdue: overdue.map(serializeTask),
        projects: projects.map((p) =&gt; ({ id: p.id, name: p.name, progress: p.progress ?? 0 })),
      },
    });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Briefing failed.
" }, { status: 500 });
  }
}

function hourGreeting(d: Date): string {
  const h = d.getHours();
  if (h &lt; 12) return "morning";
  if (h &lt; 18) return "afternoon";
```

```
  return "evening";
}

function serializeTask(t: Task) {
  return { id: t.id, title: t.title, priority: t.priority, status: t.status, category: t.category,
dueDate: t.dueDate };
}
```

src/app/api/tools/route.ts

```
import { NextResponse } from "next/server";
import { getCurrentUser, requireUser } from "@/lib/auth";
import { listTools, runTool } from "@/lib/tools/registry";

export async function GET() {
  const user = await getCurrentUser();
  if (!user) return NextResponse.json({ ok: false, error: "Unauthorized" }, { status: 401 });
  return NextResponse.json({ ok: true, tools: listTools() });
}

export async function POST(req: Request) {
  await requireUser();
  let body: any;
  try { body = await req.json(); } catch { return NextResponse.json({ ok: false, error: "Invalid
JSON." }, { status: 400 }); }
  const { name, input } = body ?? {};
  if (!name || typeof name !== "string") {
    return NextResponse.json({ ok: false, error: "Tool name required." }, { status: 400 });
  }
  try {
    const result = await runTool(name, input ?? {}, "operator");
    return NextResponse.json({ ok: result.ok, result });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Tool failed." },
{ status: 500 });
  }
}
```

API Routes — Advanced AI (images, search, vision, voice)

src/app/api/images/generate/route.ts

```
import { NextResponse } from "next/server";
import { requireUser } from "@/lib/auth";
import { generateImage, type ImageSize } from "@/lib/jarvis-ai";

// POST /api/images/generate  { prompt, size? } -&gt; { ok, images: [dataUrl...] }
export async function POST(req: Request) {
  await requireUser();
  let body: any;
  try { body = await req.json(); } catch { return NextResponse.json({ ok: false, error: "Invalid
JSON." }, { status: 400 }); }
  const prompt = typeof body.prompt === "string" ? body.prompt.trim() : "";
  if (prompt.length &lt; 3) return NextResponse.json({ ok: false, error: "Prompt must be at least 3
characters." }, { status: 400 });
  const size = (body.size as ImageSize) ?? "1024x1024";
  const allowed = ["1024x1024", "768x1344", "864x1152", "1344x768", "1152x864", "1440x720",
"720x1440"];
  if (!allowed.includes(size)) return NextResponse.json({ ok: false, error: "Unsupported size." }, {
 status: 400 });
  try {
    const images = await generateImage(prompt, size);
    if (images.length === 0) return NextResponse.json({ ok: false, error: "No image returned." }, {
status: 502 });
    return NextResponse.json({ ok: true, images });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Image generation
failed." }, { status: 502 });
  }
}
```

src/app/api/search/route.ts

```
import { NextResponse } from "next/server";
import { requireUser } from "@/lib/auth";
import { webSearch, type WebSearchResult } from "@/lib/jarvis-ai";

// POST /api/search  { query, num?, recencyDays? } -&gt; { ok, results: [...] }
export async function POST(req: Request) {
  await requireUser();
  let body: any;
  try { body = await req.json(); } catch { return NextResponse.json({ ok: false, error: "Invalid
JSON." }, { status: 400 }); }
  const query = typeof body.query === "string" ? body.query.trim() : "";
  if (query.length &lt; 2) return NextResponse.json({ ok: false, error: "Query too short." }, {
status: 400 });
  const num = Math.min(Math.max(Number(body.num) || 5, 1), 10);
  const recencyDays = body.recencyDays ? Math.min(Math.max(Number(body.recencyDays), 1), 365) :
undefined;
  try {
    const results: WebSearchResult[] = await webSearch(query, num, recencyDays);
    return NextResponse.json({ ok: true, results });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Web search
failed." }, { status: 502 });
  }
}
```

src/app/api/vision/route.ts

```
import { NextResponse } from "next/server";
import { requireUser } from "@/lib/auth";
import { visionChat } from "@/lib/jarvis-ai";

// POST /api/vision  { image (data URL or http URL), prompt? } -&gt; { ok, description }
export async function POST(req: Request) {
  await requireUser();
  let body: any;
```

```
  try { body = await req.json(); } catch { return NextResponse.json({ ok: false, error: "Invalid
JSON." }, { status: 400 }); }
  const image = typeof body.image === "string" ? body.image.trim() : "";
  if (!image) return NextResponse.json({ ok: false, error: "Image is required." }, { status: 400 });
  // Validate it's a URL or a reasonable base64 data URL (cap size to ~6MB).
  if (!image.startsWith("http") &amp;&amp; !image.startsWith("data:image/")) {
    return NextResponse.json({ ok: false, error: "Image must be a URL or a data:image/... URL." }, {
 status: 400 });
  }
  if (image.startsWith("data:") &amp;&amp; image.length &gt; 6_000_000) {
    return NextResponse.json({ ok: false, error: "Image too large (max ~4.5MB)." }, { status: 413
});
  }
  const prompt = typeof body.prompt === "string" &amp;&amp; body.prompt.trim() ? body.prompt.trim()
: "Describe this image in detail.";
  try {
    const description = await visionChat(prompt, image);
    return NextResponse.json({ ok: true, description });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Vision failed."
}, { status: 502 });
  }
}
```

src/app/api/voice/tts/route.ts

```
import { NextResponse } from "next/server";
import { requireUser } from "@/lib/auth";
import { getZAI } from "@/lib/jarvis-ai";

// POST /api/voice/tts  { text, voice?, rate? } -&gt; audio/wav blob.
// Server-side fallback for when the browser lacks speechSynthesis or the user
// explicitly chooses the z.ai voice in Settings.
export async function POST(req: Request) {
  const user = await requireUser();
  let body: any;
  try { body = await req.json(); } catch { return NextResponse.json({ ok: false, error: "Invalid
JSON." }, { status: 400 }); }
  const { text, voice, rate } = body ?? {};
  if (!text || typeof text !== "string") {
    return NextResponse.json({ ok: false, error: "Text is required." }, { status: 400 });
  }
  const trimmed = text.slice(0, 2000);
  if (!trimmed.trim()) return NextResponse.json({ ok: false, error: "Empty text." }, { status: 400
});

  try {
    const zai = await getZAI();
    const response = (await zai.audio.tts.create({
      input: trimmed,
      voice: voice &amp;&amp; voice !== "default" ? String(voice) : "tongtong",
      speed: typeof rate === "number" &amp;&amp; rate &gt;= 0.5 &amp;&amp; rate &lt;= 2 ? rate : 1,
      response_format: "wav",
      stream: false,
    } as never)) as Response;
    if (!response.ok) {
      const errTxt = await response.text().catch(() =&gt; "");
      return NextResponse.json({ ok: false, error: `TTS failed: ${response.status} ${errTxt.slice(0,
 200)}` }, { status: 502 });
    }
    const buf = await response.arrayBuffer();
    // Mark as used by user (light audit only — we do not store the audio or text).
    void user.id;
    return new NextResponse(buf, {
      status: 200,
      headers: { "Content-Type": "audio/wav", "Cache-Control": "no-store" },
    });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "TTS failed." }, {
 status: 502 });
  }
}
```

API Routes — Supabase Setup

src/app/api/supabase/setup/route.ts

```
import { NextResponse } from "next/server";
import { requireUser } from "@/lib/auth";
import { probeTables } from "@/lib/data";
import { isSupabaseConfigured } from "@/lib/supabase";

// GET /api/supabase/setup — reports whether the schema is ready and what to do
// if it isn't. Used by the client to show a setup banner.
export async function GET() {
  await requireUser();
  const configured = isSupabaseConfigured();
  if (!configured) {
    return NextResponse.json({ ok: true, configured: false, ready: false, tables: [] });
  }
  try {
    const probe = await probeTables();
    return NextResponse.json({ ok: true, configured: true, ready: probe.ready, tables: probe.tables
});
  } catch (e) {
    return NextResponse.json({ ok: true, configured: true, ready: false, tables: [], error: e
instanceof Error ? e.message : "probe failed" });
  }
}
```

src/app/api/supabase/schema/route.ts

```
import { NextResponse } from "next/server";
import { readFileSync } from "fs";
import { join } from "path";

// GET /api/supabase/schema — returns the raw schema SQL so the UI can display
// and let the user copy it into the Supabase SQL editor.
export async function GET() {
  try {
    const sqlPath = join(process.cwd(), "supabase", "schema.sql");
    const sql = readFileSync(sqlPath, "utf-8");
    const editorUrl = `https://supabase.com/dashboard/project/${process.env.
NEXT_PUBLIC_SUPABASE_URL?.replace("https://", "").replace(".supabase.co", "") ?? ""}/sql/new`;
    return NextResponse.json({ ok: true, sql, editorUrl });
  } catch (e) {
    return NextResponse.json({ ok: false, error: e instanceof Error ? e.message : "Failed to read
schema." }, { status: 500 });
  }
}
```

Appendix A — shadcn/ui Components

These are standard shadcn/ui (New York style) primitives installed via the shadcn CLI. They are open-source boilerplate; the latest versions are available at https://ui.shadcn.com. Each file lives at `src/components/ui/<name>.tsx`.

accordion.tsx	alert-dialog.tsx	alert.tsx
aspect-ratio.tsx	avatar.tsx	badge.tsx
breadcrumb.tsx	button.tsx	calendar.tsx
card.tsx	carousel.tsx	chart.tsx
checkbox.tsx	collapsible.tsx	command.tsx
context-menu.tsx	dialog.tsx	drawer.tsx
dropdown-menu.tsx	form.tsx	hover-card.tsx
input-otp.tsx	input.tsx	label.tsx
menubar.tsx	navigation-menu.tsx	pagination.tsx
popover.tsx	progress.tsx	radio-group.tsx
resizable.tsx	scroll-area.tsx	select.tsx
separator.tsx	sheet.tsx	sidebar.tsx
skeleton.tsx	slider.tsx	sonner.tsx
switch.tsx	table.tsx	tabs.tsx
textarea.tsx	toast.tsx	toaster.tsx
toggle-group.tsx	toggle.tsx	tooltip.tsx

Appendix B — Environment & Deployment

Environment variables (stored in `.env`, gitignored — not reproduced here as they contain credentials):

Variable	Purpose
DATABASE_URL	SQLite path for the Prisma fallback store
NEXT_PUBLIC_SUPABASE_URL	Supabase project URL (browser-safe)
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase anon/publishable key (RLS-protected, browser-safe)
SUPABASE_SERVICE_ROLE_KEY	Optional server-only key (bypasses RLS); empty = use anon+RLS

Running locally. `bun run dev` starts the Next.js dev server on port 3000. `bun run db:push` syncs the Prisma schema to SQLite. `bun run lint` runs ESLint. The z.ai SDK auto-loads its config from `/etc/.z-ai-config`.

Supabase setup. The app works immediately via the Prisma fallback. To store data in your Supabase project, run `supabase/schema.sql` once in the Supabase SQL editor — the data layer then switches to

Supabase automatically.